



Trabajo de Fin de Máster

"Máster Universitario en Microelectrónica: Diseño y Aplicaciones de Sistemas Micro/Nanométricos"

Diseño y caracterización de aleatoriedad de un TRNG basado en osciladores en anillos

Design and randomness assessment of a Ring Oscillator-based TRNG

Autor / Author: Mario García Jiménez

Tutores / Advisors

Fecha / Date: Junio de 2027 / June 2027

Abstract

Cryptographic keys are only as good as the entropy behind them, yet most embedded key generators are evaluated as black boxes: statistical test suites are run on their output and a pass is taken as evidence of randomness. This thesis first shows, on the hardware random number generator of the ESP32, why that is not enough: its output passes NIST SP 800-22 and scores like noise under the SP 800-90B estimators even when, as a restart test reveals, the generator receives no noise at all and only continues a pseudo-random sequence seeded at boot — and NIST itself has since rejected the first suite for assessing cryptographic generators. It then takes the opposite route: a white-box key-generation engine whose entropy is justified from the physics of the device. A ring-oscillator entropy source is designed at transistor level and its jitter is extracted from simulation and turned, through a validated stochastic model, into a lower bound on min-entropy per bit and into the design parameters of the sampler. Around it, a fully portable digital core — SP 800-90B health tests, an AES-based conditioning and CTR_DRBG with derivation function, AES-CMAC authentication and an I2C register interface — is written once and verified three ways: against the official test vectors, in simulation on a Basys 3 FPGA acting as the verification vehicle, and against an ESP32 that recomputes every result with an independent implementation (mbedtls). The same VHDL is finally synthesized on an open 130 nm standard-cell library to give the area in gate equivalents and the path to a dedicated integrated circuit. All code, models, vectors and data are released.

Keywords true random number generator, ring oscillator, jitter, min-entropy, stochastic model, AIS 20/31, SP 800-90B, CTR_DRBG, AES-CMAC, secure element, FPGA, ASIC, ESP32.

Resumen

Las claves criptográficas valen lo que vale la entropía que hay detrás, y sin embargo la mayoría de los generadores embebidos se evalúan como cajas negras: se pasan baterías de tests estadísticos sobre su salida y un aprobado se toma como prueba de aleatoriedad. Este TFM muestra primero, sobre el generador hardware del ESP32, por qué no basta: su salida pasa NIST SP 800-22 y puntúa como ruido en los estimadores de SP 800-90B incluso cuando, como revela una prueba de reinicios, el generador no recibe ningún ruido y solo continúa una secuencia pseudoaleatoria sembrada al arrancar — y el propio NIST ha rechazado después la primera batería para evaluar generadores criptográficos. Después toma el camino contrario: un motor de generación de claves de caja blanca cuya entropía se justifica desde la física del dispositivo. Se diseña a nivel de transistor una fuente de entropía por oscilador de anillo, se extrae su jitter por simulación y se convierte, mediante un modelo estocástico validado, en una cota inferior de min-entropía por bit y en los parámetros de diseño del muestreador. A su alrededor, un núcleo digital totalmente portable — tests de salud SP 800-90B, acondicionado y CTR_DRBG con función de derivación basados en AES, autenticación AES-CMAC e interfaz de registros I2C — se escribe una sola vez y se verifica de tres maneras: contra los vectores oficiales, en simulación sobre una FPGA Basys 3 que hace de vehículo de verificación, y frente a un ESP32 que recalcula cada resultado con una implementación independiente (mbedtls). El mismo VHDL se sintetiza por último sobre una biblioteca abierta de celdas estándar de 130 nm para dar el área en puertas equivalentes y el camino hacia un circuito integrado dedicado. Todo el código, modelos, vectores y datos se publican.

Acknowledgements

To my family, my colleagues at Airzone, and the faculty of the Máster en Microelectrónica at IMSE-CNM, for their support during this work.

Contents

Abstract **1**

Resumen **2**

Acknowledgements **3**

List of Figures **7**

List of Tables **9**

List of Acronyms **11**

1 Introduction **13**

1.1 Motivation: keys are only as good as their entropy 13
1.2 From a black-box study of the ESP32 to a white-box engine 14
1.3 Objectives 14
1.4 Scope and exclusions 15
1.5 Methodology and reproducibility conventions 16
1.6 Structure of the document 16

2 State of the Art **17**

2.1 Ring-oscillator TRNG architectures 17
2.2 Stochastic models and entropy bounds 18
2.3 Embedded jitter measurement 19
2.4 Standards: AIS 20/31 v3.0 and NIST SP 800-90B 20
2.5 Generators of commodity microcontrollers 21
2.6 Second sources and entropy harvesting 23
2.7 Symmetric cores for constrained hardware 25
2.8 Commercial secure elements 26
2.9 From FPGA to ASIC: what changes in the entropy source 26
2.10 Gap addressed by this work 27

3 The ESP32 Generator as a Black Box **29**
3.1 The peripheral and what Espressif documents . . . 29
3.2 Experimental design and capture . . . 30
3.3 The SP 800-22 battery . . . 33
3.4 Shannon entropy, throughput and temperature . . . 35
3.5 A multi-source key harvester . . . 38
3.6 The ESP32 reference under SP 800-90B . . . 42
3.7 What a black box can and cannot tell . . . 47

4 Entropy Source: from Transistor to Min-Entropy **49**
4.1 The elementary ring oscillator . . . 49
4.2 Stochastic model and the quality factor . . . 50
4.3 Exact conditional entropy vs. the asymptotic bound . . . 50
4.3.1 Design target . . . 51
4.4 Embedded jitter estimator and its two corrections . . . 51
4.5 Transistor-level bench and its resolution . . . 52
4.6 Design target and divider selection . . . 54

5 Digital Engine **55**
5.1 Architecture and the single-primitive decision . . . 55
5.2 AES-128 encryption core . . . 56
5.3 AES-CMAC . . . 56
5.4 CTR_DRBG with derivation function . . . 57
5.5 Health tests . . . 57
5.6 Capture path and clock-domain crossings . . . 58
5.7 I2C register interface and command sequencer . . . 59

6 Verification **60**
6.1 Strategy: vectors, simulation, independent verifier . . . 60
6.2 Reference models in Python . . . 61
6.3 Testbenches and results . . . 61
6.4 ESP32 verifier firmware . . . 64
6.5 The engine in a product: signed readings . . . 64
6.6 FPGA bring-up . . . 66

7 Path to an Integrated Circuit **67**
7.1 Standard-cell synthesis flow . . . 67
7.2 Area per block in gate equivalents . . . 67
7.3 Optimizations and their expected savings . . . 69
7.4 What the ring oscillator needs in silicon . . . 70

7.5 Cost and schedule of an academic shuttle 70

8 Results and Comparison **72**

8.1 Status against the objectives 72
8.2 Transistor-level noise of the ring 72
8.3 The ring with the transistors of an open PDK 74
8.4 Operating point: divider and throughput 77
8.5 Functional verification, consolidated 77
8.6 Comparison with commercial secure elements 78
8.7 The ESP32 reference 79
8.8 The simulated source under the same battery 80
8.9 Uncertainty budget 82
8.10 The pending campaign 83

9 Conclusions and Future Work **85**

9.1 Conclusions 85
9.2 Contributions 85
9.3 Future work 85

Bibliography **86**

A Register Map **100**

B Official Test Vectors Used **104**

C Reproduction Guide **106**

List of Figures

3.1 Tasks and cores of the capture firmware. The campaign task reads the generator into RAM while the condition holds; the ADC task (conditions A and C) and the Wi-Fi driver (A and B) run on the other core. The dashed arrows are the paths by which the documented sources may reach the generator. The ADC task is stopped before the buffer is streamed. . . 31

3.2 SP 800-22 p values of the four sequences of the first campaign, one per condition and test, 10^6 bits each [MEASURED]. A test fails below 0.01; none does. Tests with several p values show their minimum. 34

3.3 Autocorrelation and periodogram of the four sequences of the first campaign [MEASURED], against white noise (dashed). (a) Distribution of $r_k\sqrt{n}$ for lags k from 1 to 1000, $n = 10^6$; for white noise it is standard normal. (b) Distribution of the periodogram bins of the first 65 536 bits, divided by their mean, on a logarithmic scale; for white noise it is exponential, which is why the spectral flatness of a single periodogram tends to $e^{-\gamma} \approx 0.561$ and not to 1. 36

3.4 Byte entropy of the four sequences of the first campaign against the reading of the die sensor [MEASURED], in its own unit, an uncalibrated integer in degrees Fahrenheit; a bar spans the readings before and after the sequence. The band is a perfect source of 125 000 bytes, mean and one standard deviation (calculated). The whole campaign is one step of the sensor, so the correlation of $r = 0.795$ computed over these points says nothing about temperature. 37

3.5 Yield of the von Neumann corrector on independent input bits (calculated). Even an unbiased input loses three quarters of its bits; the output is unbiased for every p in $(0, 1)$ only if the input bits are independent. 39

3.6 Off-line validation of the harvester construction (calculated, on measured inputs). (a) Hamming distance between two of the 1000 seeds derived from one SRAM pattern and the condition-D sequence, over 20 000 random pairs, against the binomial distribution of independent uniform seeds. (b) p values of the seven SP 800-22 tests that apply to the 256 000 bits of the seed stream. Both are passed by construction (see text) and say nothing about the entropy of the seeds. 40

5.1 Data path of the engine. Only the ring oscillators are technology dependent; every other block is portable RTL. 55

8.1 Min-entropy of the simulated source against the divider [SIMULATED]: the stochastic model (lines) and the SP 800-90B non-IID estimate on 10^6 bits (markers, mean and standard deviation of three seeds). The grey band is the estimate for uniform independent bits of the same length; the hatched band spans the thirty-six ESP32 captures of conditions A–D [MEASURED]. 81

List of Tables

2.1 AIS-compatible TRNG cores on Xilinx Spartan-6 (45 nm), after Petura *et al.* [8], Table II. Entropy is the T8 estimate; "feasibility" is the authors' rank (higher is easier). 18

2.2 Published assessments of the hardware generators of IoT-class parts. "Conditions" says whether the operating state was varied. 23

3.1 The four capture campaigns [MEASURED]. Each cycle is one cold boot followed by the four conditions in the order D, C, B, A, 10^6 bits each. . . . 32

3.2 Statistical metrics of the first campaign, one sequence of 10^6 bits per condition [MEASURED]: Shannon entropy per byte and per bit, largest autocorrelation $|r_k|$ for lags k from 1 to 1000, spectral flatness of one periodogram of the first 65 536 bits, zlib compression ratio and byte χ^2. The last row is what a perfect source returns at this length (calculated, see text). 35

3.3 The ESP32 generator under the SP 800-90B estimators [MEASURED]. Conditions A–D: nine repetitions of 10^6 bits each, three on each of the three chips, mean and range over the repetitions; E and F: single captures of 3.5×10^6 and 8.4×10^6 bits. The ceiling is the highest value the most-common-value (MCV) estimator can return for a perfectly uniform source of that length at 99 % confidence; the non-IID figure is the minimum of the ten estimators of Section 6.3 of the standard, and the last column names the estimator that set it (times out of nine). The IID-track verdicts are discussed in the text. 43

3.4 Restart test of the ESP32 generator [MEASURED]: 1000 cold restarts per variant, one chip. H_r and H_c are the minimum of the non-IID estimators over the row and column datasets of the standard (1000 restarts $\times$ 1000 bits); the test fails when either is below half the initial estimate H_I, the lowest non-IID figure of this chip in the same condition (0.838 bit per bit in D, 0.859 in B). The byte layout gives the same verdicts. Overlaps are restarts whose output appears shifted inside another's, all identical word for word. The late reads fall 352 to 412 ms after the application starts, most of them at 352 ms. 45

3.5 Which source injects noise [MEASURED], third chip, boot noise removed: a first read at boot, one source, and a second read at a fixed 800 ms. A source injects noise if the second reads stop being windows of the fixed sequence (overlaps) and never repeat. 300 restarts per variant, 1000 for the ADC and the boot source, which also pass the restart test of the standard (H_r / H_c 0.864 / 0.877 and 0.840 / 0.842). 47

4.1 Conditional Shannon entropy per bit: exact wrapped-Gaussian computation vs. truncated expansions [SIMULATED]. 51
4.2 Recovery of a known Q_{raw} by the corrected embedded estimator (K = 20 000 blocks of $N = 100$ pairs) [SIMULATED]. 52

6.1 Testbenches, what they check and their result (GHDL 6.0, VHDL-2008). . 62

7.1 Logic synthesis on IHP SG13G2, flattened, no routing. 1 GE = 7.2576 μm^2. Lower bound on area [SIMULATED]. 68

8.1 Objectives and the evidence supporting each. 73
8.2 Period jitter of the five-stage ring against injected noise density [SIMULATED]. $\sigma_{\text{rel}} = \sigma/T$. 73
8.3 Five-stage ring on IHP SG13G2 [SIMULATED]. The noise density is not assumed: it is the white floor of the transistors' own noise model at the switching point. The thermal point is the mean of eight independent realizations, ± one standard deviation of the mean; the other rows are single realizations. Power is the average over whole periods. 74
8.4 Divider and resulting throughput per ring for $Q \geq 0.2286$ [ESTIMATED]. . . 77
8.5 Verification totals [SIMULATED]. 78
8.6 The engine against commercial secure elements. Certification data from the manufacturers' datasheets and from the NIST validation records [7, 102, 98, 99]. 79

A.1 Registers. R: read, W: write. 101
A.2 STATUS (0x02). 101
A.3 DIAG (0x0E). 102
A.4 CONTROL (0x03). One command per write. 102

B.1 Reference values and their origin. Counts are checks passed in the last run (`results/tb_motor_top.log`, 177 in total, with 11 of 11 PSL assertions and 6 of 6 cover points). 105

C.1 What the workflow reproduces. 106

List of Acronyms

AES	Advanced Encryption Standard
AIS	Anwendungshinweise und Interpretationen zum Schema (BSI)
APT	Adaptive Proportion Test
ASIC	Application-Specific Integrated Circuit
BSI	Bundesamt für Sicherheit in der Informationstechnik
CAVP	Cryptographic Algorithm Validation Program
CDC	Clock Domain Crossing
CMAC	Cipher-based Message Authentication Code
CTR_DRBG	Counter-mode Deterministic Random Bit Generator
DRBG	Deterministic Random Bit Generator
ERO	Elementary Ring Oscillator
FPGA	Field-Programmable Gate Array
GE	Gate Equivalent
I2C	Inter-Integrated Circuit
KAT	Known-Answer Test
LUT	Look-Up Table
MEMS	Micro-Electro-Mechanical System
MPW	Multi-Project Wafer
NIST	National Institute of Standards and Technology
PDK	Process Design Kit

PLL	Phase-Locked Loop
PUF	Physically Unclonable Function
PSL	Property Specification Language
RCT	Repetition Count Test
RO	Ring Oscillator
RTL	Register-Transfer Level
VHDL	VHSIC Hardware Description Language
SPI	Serial Peripheral Interface
TRNG	True Random Number Generator
XADC	Xilinx Analog-to-Digital Converter

Chapter 1

Introduction

1.1 Motivation: keys are only as good as their entropy

Every secure connection an embedded device opens, every firmware image it authenticates and every credential it stores rests on a key, and every key rests on a source of randomness. If that source is weaker than assumed, no amount of cryptographic sophistication downstream recovers what was lost at the start: a 128-bit key drawn from a generator with effectively 40 bits of entropy is a 40-bit key. This is not a theoretical concern. Frequency injection on the supply rail of a commercial secure microcontroller collapsed its key space from 2^{64} to about 3300 [1]; a certified smart-card generator was found biased despite passing the standard statistical suites [2]; and the hardware generator of one of the most widely deployed IoT microcontrollers, the ESP32, silently returns pseudo-random values when its radio is off while still passing every black-box test [3], which this work measures directly with the restart test of SP 800-90B (Section 3.6).

The common thread is that randomness has been judged by looking at the output. Statistical batteries such as NIST SP 800-22 check whether a bit stream *looks* uniform and independent; they cannot tell a physical noise source from a deterministic generator seeded with a constant, and NIST said so formally in 2022 when it announced the revision of that publication, "rejecting its use for assessing cryptographic random number generators" [4]. The current standards — AIS 20/31 in Europe [5] and SP 800-90B in the United States [6] — ask for something different: a stochastic model of the noise source, an estimate of its min-entropy derived from that model, and health tests that watch the source itself rather than its output. That is a microelectronics problem before it is a cryptography problem, and it is the problem this thesis addresses.

1.2 From a black-box study of the ESP32 to a white-box engine

This work has two parts, and the first is the justification of the second. The first part characterizes the hardware random number generator of the ESP32 as a black box: the same peripheral is exercised under four operating conditions, its output is run through a self-validated implementation of the fifteen tests of SP 800-22 and through the SP 800-90B estimators, a key-harvesting scheme combining several on-chip sources is built, and a restart test with and without the documented noise sources opens the box as far as it can be opened from outside (Chapter 3). Its central finding is methodological: the tests pass under conditions in which the entropy cannot be what the tests imply. With the radio and the ADC off the generator receives no noise: it only continues a pseudo-random sequence seeded by the bootloader, and once that boot noise is removed its output is a fixed sequence read from a slightly different point at every boot. Every statistical test still scores the output like noise — which is exactly the blind spot described above. Two figures of the first campaign also had to be revised during the work — a temperature correlation was read from a sensor whose resolution was a single step over the observed range, and a stability figure was inferred where it could have been measured — and both lessons are turned into rules in Section 1.5.

The second part builds the generator instead of measuring someone else's: an engine whose entropy source is designed, whose jitter is measured inside the device, whose min-entropy follows from a model, and whose keys are produced and used by primitives verified against the standards. The ESP32 changes role from subject to judge. It has its own AES and SHA accelerators and a mature cryptographic library, so it can recompute everything the engine produces with an implementation that shares nothing with it, over a real bus.

The engine is designed for silicon. Secure elements — small integrated circuits that generate, store and use keys on behalf of a host microcontroller over I2C — are a mature product category; the *transparency* of their entropy is not. Of three widely used parts, one ships with a NIST validation of its entropy source and two do not mention SP 800-90 anywhere in their documentation [7]. The FPGA in this work is therefore the verification vehicle, not the target: it lets millions of test vectors run against the design in minutes, which no transistor-level simulator can, before the same description is synthesized on standard cells.

1.3 Objectives

O1 Design a ring-oscillator entropy source at transistor level, with enable and sampler, and simulate it with noise, stating the numerical resolution of the bench before any figure is read from it.

O2 Extract the jitter from the simulated waveforms with an estimator that separates quantization, thermal and flicker components, validated beforehand on synthetic data of known jitter.

O3 Turn the jitter into a min-entropy bound through a stochastic model checked against an exact computation, and derive the sampler divider that meets the AIS 20/31 target with margin.

O4 Write the digital engine — health tests, AES-based conditioning and CTR_DRBG, AES-CMAC and an I2C register interface — as technology-independent RTL verified against official vectors.

O5 Verify the engine functionally on an FPGA with the ESP32 as an independent oracle: known-answer tests, cross interoperability and challenge-response.

O6 Run an entropy campaign on real silicon (the FPGA's rings) with the SP 800-90B estimators and compare with the ESP32 generator under the same methodology.

O7 Synthesize the engine on an open standard-cell library and report its area in gate equivalents with the growth factors to core area.

O8 Compare the design with the ESP32 generator, with commercial secure elements and with published circuits, with stated uncertainties.

The minimum defensible result is O1–O3 together with O5: an entropy source designed from the transistor, with its jitter extracted and turned into a bound, and a digital engine verified against an independent implementation. A negative result that is well measured closes the work as well as a positive one: if the accumulation exponent on hardware is two rather than one, flicker dominates and the divider cannot grow indefinitely, and that is a finding.

1.4 Scope and exclusions

Four things are deliberately out of scope, each for a stated reason. *Fabrication*: an academic shuttle is affordable and the engine fits in the smallest block offered, but the schedule from tape-out to packaged and characterized samples exceeds a master's thesis; the cost and the path are documented, not executed. *Asymmetric cryptography*: an elliptic-curve multiplier is a project in itself and would consume the whole area budget; the engine provides symmetric keys and authentication. *Side-channel resistance*: the threat model is a logical, remote adversary; timing leaks are avoided by construction, but no masking or trace-based evaluation is attempted, and the institute's own laboratory for such work is noted as the natural continuation. *Formal certification*: the criteria of

AIS 20/31 and SP 800-90B and the official vectors are used throughout, and no validation is claimed.

1.5 Methodology and reproducibility conventions

Four conventions apply to every figure in this document. First, each number is labelled [MEASURED], [SIMULATED] or [ESTIMATED], and carries its uncertainty or the source it comes from. Second, **the resolution of the instrument is declared before any trend is interpreted**, whether the instrument is a sensor, an estimator or a simulator — the numerical noise floor of the SPICE bench and the quantization of a temperature sensor are treated alike. Third, quantities are measured rather than inferred whenever a direct measurement exists. Fourth, novelty is stated as "not found in the literature consulted", never as "does not exist"; one such claim in an early draft of this work had to be corrected when a 2024 paper measuring exactly the quantity in question was found, and the correction is left visible in the working notes.

Everything is released: the VHDL and its testbenches, the Python models and analysis, the firmware, the synthesis flow, the official test vectors used and the raw data of every campaign, with the commands that reproduce each table.

1.6 Structure of the document

Chapter 2 reviews ring-oscillator generators, their stochastic models, the standards and the commercial landscape. Chapter 3 is the first part of the work: it characterizes the generator of the ESP32 as a black box and shows what such a study can and cannot establish. Chapter 4 builds the chain from transistor to min-entropy and validates every link. Chapter 5 describes the digital engine and Chapter 6 its verification. Chapter 7 synthesizes the engine on standard cells and states the path and cost of an integrated circuit. Chapter 8 reports the hardware campaign and the comparisons, and Chapter 9 concludes.

Chapter 2

State of the Art

The review is organized along the chain the thesis builds: how ring oscillators are turned into random bits, how their entropy is modeled and bounded, how their jitter is measured from inside the device, what the two governing standards demand, what is known of the generators of commodity microcontrollers — the ESP32's above all — and of the second sources and harvesters built around them, which symmetric primitives fit a constrained engine, what the commercial parts offer, and what changes when the design leaves the FPGA for silicon. The last section states the gap this work addresses, in the guarded form the working rules require.

2.1 Ring-oscillator TRNG architectures

A ring oscillator is an odd chain of inverters closed on itself; its period fluctuates because each transition is delayed by a random amount, and that fluctuation — jitter — is the noise source of an entire family of generators. What distinguishes the architectures is how the jitter is accumulated and sampled [8, 9].

The **elementary ring oscillator** (ERO) samples one ring with a flip-flop clocked by a second ring divided by K_D; the divider sets the accumulation time. It has the most mature stochastic model [10, 11, 12] and an embedded jitter measurement [13], at the price of a very low output rate (K_D of 10^4 to 10^5: 80 000 in Spartan-6 and 135 000 in Cyclone V in Petura's implementation, about 430 000 in Fischer and Lubicz's design for 0.997 bit of entropy). The **multi-ring** generator of Sunar, Martin and Stinson [14] XORs many rings sampled by a reference clock; Wold and Tan [15] added a flip-flop per ring before the XOR and reported 100 Mbit/s in under 100 logic elements. Its weakness was exposed by Bochard *et al.* [16]: part of the apparent randomness is *pseudo*-randomness from the interaction of rings at different frequencies, and the independence the proof assumes does not hold on an FPGA. **Coherent sampling** (COSO) uses two rings of nearly equal period and counts the beat [17, 18]; its model gives min-entropy directly from the distribution of the count [19], and dynamic calibration of reconfigurable rings makes it portable across

devices and placements (3.30 Mbit/s in Spartan-6, 4.65 Mbit/s in Spartan-7) [20, 19]. The **transition-effect ring** (TERO) counts how many times a bistable structure oscillates before settling [21], with a full physical-to-stochastic model [22]; the **self-timed ring** (STR) circulates several events in an asynchronous ring to obtain phases spaced by the order of the jitter [23], reaching 154 Mbit/s; and the **ES-TRNG** of Yang *et al.* uses the carry chain as a time-to-digital converter to encode the edge position with 10 LUTs and 5 flip-flops [24, 25].

Table 2.1 reproduces the comparison Petura *et al.* made of the AIS-compliant cores on one device [8]. The entropy column is a statistical estimate (the AIS 31 T8 test), not a model bound; the feasibility column is the authors' qualitative note, where COSO and TERO score worst because they need manual adjustment per device. No comparable table exists for Artix-7; the open OpenTRNG project implements ERO, multi-ring and COSO on an Arty A7 with the same XC7A35T die as the Basys 3 but publishes neither entropy nor throughput figures [26].

Table 2.1: AIS-compatible TRNG cores on Xilinx Spartan-6 (45 nm), after Petura *et al.* [8], Table II. Entropy is the T8 estimate; "feasibility" is the authors' rank (higher is easier).

Core	LUT/reg.	Power (mW)	Rate (Mbit/s)	Entropy/bit	Feasib.
ERO	46/19	2.16	0.0042	0.999	5
COSO	18/3	1.22	0.54	0.999	1
MURO	521/131	54.72	2.57	0.999	4
PLL	34/14	10.6	0.44	0.981	3
TERO	39/12	3.31	0.625	0.999	1
STR	346/256	65.9	154	0.998	2

2.2 Stochastic models and entropy bounds

The requirement that a physical generator come with a *stochastic model* — a mathematical description of the noise source from which the entropy of the output can be derived — was introduced by Killmann and Schindler [11, 27] and is mandatory in AIS 20/31. For ring oscillators the reference model is Baudet *et al.* [10]: the phase of the sampled ring is a Wiener process with drift μ and volatility σ^2, the sampled bit is the half-period the phase falls in, and everything depends on the quality factor $Q = \sigma^2 \Delta t$ (accumulated phase variance between samples) and the frequency ratio $\nu = \mu \Delta t$. The model gives the bit probability conditioned on the previous phase, the probability of any output vector, and a lower bound on the Shannon entropy rate, $H \geq 1 - \frac{4}{\pi^2 \ln 2} e^{-4\pi^2 Q}$, whose exact status is examined in Chapter 4. Petura *et al.* rewrite it in design quantities, $H = 1 - \frac{4}{\pi^2 \ln 2} \exp(-\pi^2 \sigma_{th}^2 K T_2 / T_1^3)$, and Fischer and Lubicz invert it to obtain the divider

K_D from a measured jitter [13]. Ma *et al.* give an alternative model used by the Leuven group [12]; Haddad *et al.* question the mutual independence of jitter realizations on which these cancellations rest [28]; and Saarinen warns that Baudet's bound "is never lower than 0.415 even when Q approaches zero" and is safe only under additional assumptions, proposing a bound valid over the whole range [29].

Two refinements matter for a design. First, not all jitter is entropy. Fischer and Lubicz separate the accumulated phase variance into a random-walk term, linear in time, produced by thermal (white) noise, and flicker ($1/f^{\beta}$) terms whose variance grows quadratically and dominate at long accumulation times [13]; deterministic, global jitter from the supply and substrate behaves like the latter and can be known or induced by an adversary [30]. Only the linear term may be credited, which is why a divider cannot be made arbitrarily large to compensate a weak source. Second, every embedded measurement is *differential*: two rings sharing a supply see the same global jitter, which cancels in their relative phase, leaving the sum of their local jitters. Lubicz and Skórski formalize this "jitter transfer principle" and show that with two rings one obtains only the sum, while three rings and three pairwise measurements recover the individual variances; their experiment also shows the assumption of variance linear in period failing by a factor of two in one case [31].

2.3 Embedded jitter measurement

Oscilloscopes cannot resolve the picoseconds of jitter of a ring inside a chip without an external divider [32], so the useful methods measure from within. The **counter method** counts periods of one ring in a fixed window of the other; the variance of the count grows with the accumulated relative jitter [11, 33], with a quantization floor of one count and contamination by flicker at long windows. The method of **Fischer and Lubicz** [13] needs no divider: in $K \approx 10^4$ blocks of $N \approx 100$ bits sampled at $K_D = 1$, the fraction of pairs (b_j, b_{j+M}) that differ tracks the phase accumulated over M periods, and its variance across blocks, plotted against M between 200 and 1600, gives the jitter per period with an error under 5 % in simulation; on a Cyclone III they measured $\sigma = 5.01$ ps per 7.81 ns period, and showed that cooling the chip reduces σ — which makes the same measurement an online test. **Coherent sampling** yields the accumulated jitter from the variance of the beat count with no extra hardware [19]. Most recently Benea *et al.* characterize the jitter of rings with the **Allan variance**, fitting $\sigma_t^2(t) = a_0 + a_1 t + a_2 t^2$ to separate quantization, thermal and flicker terms, and do so both on an Artix-7 — the FPGA family of this work — and on a 28 nm FD-SOI circuit, with data and scripts released [32]. Published per-period jitter figures cluster at 2–4 ps for periods of 4–8 ns in Spartan-6 and Cyclone V [8], 5 ps in Cyclone III measured embedded [13], and 2–2.5 ps in Cyclone III and Virtex-5 for self-timed rings [23]; the Artix-7 coefficients of Benea *et al.* are published in counter

units, not seconds, so an absolute figure for the XC7A35T remains to be measured.

2.4 Standards: AIS 20/31 v3.0 and NIST SP 800-90B

Two documents govern how a physical generator is assessed. The German **AIS 20/31**, in its version 3.0 of September 2024 [5], replaces the 2011 requirement of 0.997 bit of Shannon entropy per internal bit [34] with a choice in class PTG.2: Shannon entropy ≥ 0.9998 and/or min-entropy ≥ 0.98, together with $P(1) \in (0.493, 0.507)$; the BSI's transition policy allows new evaluations under the old version only until March 2026 [35]. Its requirements are structural as much as numerical: a verifiable stochastic model is mandatory; the online test must be derived from the model, failing with high probability when the source parameter leaves the admissible set; a total-failure test must detect entropy collapsing to zero, and the repetition-count test of SP 800-90B is explicitly accepted for it; and the black-box suite T_{irn} now includes two of NIST's predictors (MultiMMC and LZ78Y), a deliberate harmonization [5, 36].

The American **SP 800-90B** [6] estimates *min-entropy* — the adversarial quantity, $-\log_2$ of the probability of the most probable value — from at least one million raw samples and a 1000×1000 restart matrix, using the most-common-value estimator if the source is shown to be i.i.d. and otherwise the minimum of ten estimators, four of them predictors that try to guess the next sample. It approves two continuous health tests and requires no others: the Repetition Count Test with cutoff $C = 1 + \lceil -\log_2 \alpha / H \rceil$ and the Adaptive Proportion Test over a window of 1024 samples for binary sources (512 for non-binary), $C = 1 + \text{CRITBINOM}(W, 2^{-H}, 1 - \alpha)$, with $\alpha = 2^{-20}$ recommended. It lists the conditioning components whose output may claim full entropy — HMAC, CMAC, CBC-MAC, approved hashes and the two derivation functions of SP 800-90A — and gives the formula for the entropy of a conditioned output as a function of input length, input entropy and internal width. The reference implementation of the estimators is public [37].

The two standards agree on what the older statistical suites cannot do. SP 800-22 [38] tests uniformity and independence of a bit stream; a deterministic generator with zero entropy passes it by construction, and so does a weak physical source behind a cryptographic post-processor [29]. NIST's 2022 decision to revise the publication states the point in so many words, "rejecting its use for assessing cryptographic random number generators" [4]. The criticism that led there is older: Saarinen calls the suite "clearly obsolete, possibly harmful", since generators with no secret at all pass it [39]; Iwasaki shows that the results of its tests are not mutually independent and that it lacks a principled global acceptance criterion [40]; and the public comments gathered in NIST's own review repeat

the redundancy of the tests and the misuse of the suite as a certification [41]. The empirical record agrees: a certified smart-card generator showed consistent biases while passing most tests [2, 42], and the ESP32's generator passes the same screenings with its radio off, when Espressif's own documentation says it returns pseudo-random values [43, 44, 3]. The academic literature on that generator applies only SP 800-22 or Dieharder [45, 46]; no SP 800-90B characterization of it was found (Section 2.5).

2.5 Generators of commodity microcontrollers

Why the generator of a microcontroller deserves a section of its own is shown by what happens when one fails in the field. Heninger *et al.* scanned the TLS and SSH hosts of the whole IPv4 space and found that 0.75 % of TLS certificates shared their keys with another host; they factored the RSA keys of real devices whenever two keys shared a prime, and traced the cause to headless embedded appliances — routers, firewalls — generating keys at first boot, before their entropy pool had filled: the *boot-time entropy hole* [47]. RFC 4086, the IETF's best current practice on randomness, states the principle that the security of keys, nonces and cookies rests on unpredictable values and that a poorly seeded pseudo-random process gives only "pseudo-security" [48]; and Tillmanns *et al.*, reverse-engineering the firmware of Broadcom and Cypress Bluetooth chips, found that the dedicated hardware generators present in some of them pass advanced tests while many popular devices have none and fall back on an insecure pseudo-random generator [49]. A connected thermostat or HVAC unit that wakes from deep sleep and opens a TLS session before its radio has associated is the same pattern in miniature.

The ESP32 chain. On the ESP32 the path from the peripheral to the protocol is short. The second-stage bootloader, and the application on demand through `bootloader_random_enable()`, put the SAR ADC into a noise-sampling mode (`bootloader_random.c` [50]); the entropy interface of mbedTLS in ESP-IDF, `mbedtls_hardware_poll()` in `esp_hardware.c`, is a thin wrapper over `esp_fill_random()` [51]; and the TLS stack built on it [52] draws from that function the client and server randoms, the ephemeral ECDHE scalars, the IVs and the session identifiers, with no conditioning of its own. Whatever regime the generator is in at the moment a handshake runs, the session keys inherit it. Benchmarking TLS 1.3 on the ESP32, Salles and Farias find a Curve25519 and RSA suite about four times faster than P-256 with ECDSA at equal security [53], so the handshake is where the time goes, and a generator that blocks or a slow software conditioner taxes the most expensive moment of the connection. The bootloader is also what seeds the generator before any radio is up: asked when the RF subsystem counts as "enabled" for `esp_random()` to be true random — whether initializing a Bluetooth stack is enough, and which cases saturate the high-speed ADC with Wi-Fi on — an Espressif

maintainer answered that the radio start-up enables the high-speed ADC and that the bootloader seeds the generator's entropy beforehand, and the documentation was rewritten as a result [54]. The bootloader announces the switching of that source in its log ("Enabling/Disabling RNG early entropy source"), which has itself puzzled users [55]. The boundary between true and pseudo-random operation is, in short, subtle even for the manufacturer's own users; the restart test of Section 3.6 measures how much the boot noise contributes.

The original ESP32 and its successors. The guides of the later chips, ESP32-S2, -C3 and -H2, document a secondary entropy source that the original ESP32 does not have: the sampling of an asynchronous 8 MHz internal oscillator, always enabled and mixed into the generator state by hardware, which on its own "was sufficient to pass the Dieharder random number test suite", although true random output is only guaranteed with the main source also enabled [56]. The guide of the original ESP32 names no such source [43]. Results obtained on a later chip therefore do not transfer to the original one, and the "pseudo-random only" caveat weighs more on it.

Earlier looks at the ESP32 generator. The closest peer-reviewed work, by Hidayatulloh *et al.*, uses `esp_random()` for the nonces of AES counter mode on point-to-point LoRa links and reports that the numbers pass the NIST suite, in a single operating condition and without the per-test pass proportions or the uniformity of the p-values [46]. Castillo *et al.* present an automated web framework for SP 800-22 and validate it on 2 GB from an ESP32-C3, which carries the secondary source; the contribution is the tool, not the device [45]. Espressif's own validation is the single favourable-condition Dieharder run of Section 3.1. The rest is informal: Cheich plots histograms of up to about 300 000 values with Wi-Fi on and off and across reboots and concludes that the output looks random, with no formal battery [57], and the AndroidCrypto tutorial reproduces the Espressif warning and shows how to call `bootloader_random_enable()`, with no characterization at all [58]. Blanco-Romero *et al.* make the point that the pseudo-random output with the radio off passes statistical screenings [3]. A characterization of the original ESP32 that varies the operating conditions and estimates the min-entropy of the output was not found in the literature consulted.

Other microcontrollers. Table 2.2 sets the ESP32 beside the generators of other parts of the same class. STMicroelectronics documents in application note AN4230 a validation of the STM32 generator against SP 800-22 and SP 800-90B and reports its evaluation against AIS-31 PTG.2, the most complete public vendor characterization found [59]; even so, Ngo and Dubrova recover the Hamming weight of its output words from the power consumption with a deep-learning model, with better than 60 % accuracy against a 14 %

baseline — side-channel leakage being a threat distinct from statistical weakness [60]. Nordic documents for the nRF52840 a generator based on internal thermal noise with an optional hardware bias corrector [61]. The RP2040 exposes one bit of a ring oscillator, `RANDOMBIT`, which its datasheet says does not meet the requirements of security systems [62, 63]. The Broadcom application processors of the Raspberry Pi expose a closed generator as `/dev/hwrng`, which community evaluations with ENT, `rngtest` and Dieharder find good at about 1 Mbit/s [64]; with the design and any filtering unpublished, no stochastic model can be built and "passes the tests" cannot be turned into an entropy figure. The ESP32 sits between the documented FPGA designs of Section 2.1 and these black boxes: its sources are named, not quantified.

Table 2.2: Published assessments of the hardware generators of IoT-class parts. "Conditions" says whether the operating state was varied.

Platform	Declared source	Assessment	Conditions	Result	Ref.
ESP32	RF/SAR ADC noise, asynchronous clocks	Dieharder, 2 GB (vendor)	Wi-Fi on only	Pass; data not public	[44]
ESP32	`esp_random()`	SP 800-22	One	Pass (LoRa nonces)	[46]
ESP32-C3	As ESP32, plus 8 MHz secondary	SP 800-22, 2 GB	One	Pass (focus: the framework)	[45]
ESP32 (blog)	`esp_random()`	Histograms	Wi-Fi on/off, reboots	"Looks random"	[57]
STM32	Analog noise, dedicated	SP 800-22, SP 800-90B; AIS-31 PTG.2	Not stated	Pass (vendor)	[59]
nRF52840	Thermal noise, bias corrector	None published	—	—	[61]
RP2040	Ring oscillator bit	Community analyses	—	Not fit for cryptography (datasheet)	[62]
BCM2835	Closed	ENT, `rngtest`, Dieharder	No	Pass	[64]
Broadcom/ Cypress Bluetooth	On-chip generator, if any	Advanced suites, reverse engineering	Many devices	Generator good; fallback insecure	[49]

2.6 Second sources and entropy harvesting

A node that keeps its radio off needs entropy from elsewhere, and two bodies of work bear on where to find it and how to combine it.

The SRAM start-up state. A six-transistor SRAM cell is a pair of cross-coupled inverters; at power-up both start near their metastable point and the cell resolves towards

one state or the other. In cells whose transistors are strongly mismatched the outcome is fixed by manufacture and repeats at every power-up: a device-specific fingerprint, the basis of the SRAM PUF. In closely matched cells thermal noise decides anew each time, and those cells are an entropy source. The same pattern is thus read in two ways, the stable cells as identity and the unstable ones as randomness, as Holcomb, Burleson and Fu showed with FERNS, extracting 128 random bits that pass the NIST runs, approximate-entropy and block-frequency tests from 512 bytes of power-up SRAM [65]. Guajardo *et al.* gave the first intrinsic SRAM PUF on FPGAs and the intra- and inter-device Hamming-distance methodology now standard for separating fingerprint from noise [66], and van der Leest *et al.* quantified the min-entropy of the noise in commercial SRAM start-up patterns and conditioned it into full-entropy 256-bit seeds [67]. The pattern is described by its uniformity (fraction of ones, ideally one half), its uniqueness (inter-device distance, ideally half the length) and its reliability, whose complement, the intra-device bit error rate between power-ups of one chip, is reported in a range of about 3 to 8 % at nominal conditions [65, 67]. The identity use corrects those errors away with a fuzzy extractor and public helper data; the entropy use seeks them out, and must remember that the stable majority is known to anyone who has once read the pattern. For the ESP32 no peer-reviewed estimate of the start-up min-entropy was found. The open `esp32_puflib` implements an SRAM PUF for the family as an ESP-IDF component, reading the memory from a deep-sleep wake stub and keeping helper data in flash [68]; and a forum thread reports that the boot ROM has already written the first 0x1320 bytes of SRAM1 and 0x6E0 bytes of SRAM2 when application code runs [69], so a reader must use regions the ROM leaves untouched. The two sources of a chip are complementary rather than redundant: the generator's noise is continuous but needs its injectors powered, while the SRAM pattern is one-shot, free at power-up, renewed only by a power cycle and dominated by its fixed fingerprint.

Conditioning and harvesting. The Leftover Hash Lemma of Impagliazzo, Levin and Luby bounds how close to uniform a 2-universal hash of a source of known min-entropy comes, but needs a uniform seed [70], which a device short of entropy does not have. In practice a fixed cryptographic function is used and its soundness argued computationally: Dodis *et al.* analyse when the CBC-MAC, cascade and HMAC constructions act as randomness extractors [71], SP 800-90B lists them as vetted conditioners (Section 2.4), and HKDF splits key derivation into an extract step that distils a pseudorandom key from non-uniform input and an expand step that stretches it [72]. Fusing several weak sources has precedent in constrained devices. Pawlowski, Jara and Ogorzalek harvest the least significant bits of temperature, humidity and light sensors on IoT nodes, the light sensors giving about 7.9 bits per byte after adjustment at roughly 1.3 ms per byte [73]; Zhang and Wu build an entropy pool that fuses chaotic circuits and environmental events

in multi-sensor networks [74]; Foreman, Yeung and Curchod show extractors (Toeplitz, Trevisan) raising the measured quality of commercial generators under extended batteries [75]; and the survey of Kietzmann, Schmidt and Wählisch maps radio, sensor, SRAM and jitter sources, generators and design guidelines for the constrained IoT and treats entropy scarcity there as structural [76]. The `pico_rand` of the RP2040 SDK is a working instance in silicon: because the ring-oscillator bit is autocorrelated when sampled quickly, it mixes ring samples (at most about 100 kbit/s, 10 μs apart), time and bus counters through a hash [77]. Two cautions apply to all of them. Min-entropy adds across sources only if they are independent, which sources sharing die, supply and temperature need not be; what concatenation followed by a hash guarantees without that assumption is only the min-entropy of the best source. And a hashed output looks uniform whatever went in, so statistical batteries on it validate the construction, not the entropy; the harvester of Section 3.5 is read in that light.

2.7 Symmetric cores for constrained hardware

The engine needs a block cipher and, if it uses one, a hash. For AES-128 (ten rounds [78]) the design space runs from fully unrolled pipelines for throughput to 8-bit datapaths for area: Good and Benaissa report 124 slices and two block RAMs at 2.2 Mbit/s on a Spartan-II at one extreme and 25 Gbit/s at the other [79]; Chodowiec and Gaj's compact design uses 222 slices and three block RAMs [80]; and the smallest ASIC realizations reach 2.4 kGE with an 8-bit datapath at 226 cycles per block [81], following the composite-field S-box line of Satoh [82], Canright [83] and Boyar–Peralta [84]. The composite-field constructions optimize ASIC gate counts and lose their advantage in an FPGA LUT, where a direct table is both faster and simpler. Open reference designs with published Artix-7 figures exist: a one-round-per-cycle encryptor at 1104 LUTs and 294 MHz [85], and a 32-bit iterative encryptor/decryptor at 2298 slices and 97 MHz [86]. For an engine whose output leaves through an I2C link, any of these is far faster than the link, and the choice is dictated by verifiability and area, not speed.

SHA-256 [87] in its iterative form costs about 750 slices at 108 MHz on Artix-7 [88], comparable to a compact AES; its role in a generator would be as the vetted conditioner of SP 800-90B. The deterministic generators of SP 800-90A [89] — Hash_DRBG, HMAC_DRBG and CTR_DRBG — differ mainly in cost: CTR_DRBG with AES-128 keeps a state of 256 bits plus a counter and produces 128 bits per block encryption, Hash_DRBG needs 440-bit modular additions, and HMAC_DRBG two hash compressions per output block. Without a derivation function CTR_DRBG requires full-entropy input of exactly the seed length; with one, the usual configuration in practice, it accepts arbitrary input. Woodage and Shumow's analysis warns that the standard's flexibility permits choices that worsen state leakage and recommends updating after every generate

and reseeding far below the 2^{48} limit [90]; SP 800-90C, final since 2025, frames how a physical source and a DRBG combine [91]. For authentication, AES-CMAC (SP 800-38B [92], RFC 4493 [93]) costs one block encryption per message block and is itself a vetted conditioner. The official vectors for all of these are public [94, 95, 96, 97].

2.8 Commercial secure elements

The product category this engine belongs to exists. A secure element is a small integrated circuit that generates and stores keys and performs symmetric and asymmetric operations for a host microcontroller, almost always over I2C. Four parts dominate the low-cost segment: Microchip's ATECC608B, NXP's SE050, Infineon's OPTIGA Trust M and ST's STSAFE-A110 [7, 98, 99, 100]. The ATECC608B, at 0.90 $ in single units [101], offers AES-128, SHA-256/HMAC, ECDSA/ECDH on P-256, sixteen key slots and an I2C interface, and its entropy source holds a NIST validation under SP 800-90B, obtained precisely with ring oscillators [102]. Two of its competitors make no reference to SP 800-90 in their datasheets; the third states that its generator is "designed to meet" the standard. The lesson for this work is where an academic design can and cannot compete: not on price, where a certified part costs less than a euro, but on the transparency of the entropy, which is exactly what a black-box part cannot offer and what the standards now demand.

2.9 From FPGA to ASIC: what changes in the entropy source

Digital logic ports from an FPGA to standard cells with a known penalty — Kuon and Rose measured 40× in area, 3.2× in delay and 12× in dynamic power for logic-only designs in 90 nm, and 27–49× in area for AES specifically [103]; the ring oscillator does not port at all, because in silicon it is built from inverter cells whose noise, variability and coupling differ from a LUT chain. Three published findings define what a silicon implementation must add. Benea *et al.* show that flicker noise, whose weight grows in newer nodes, changes the character of the jitter and, against expectation, that treating it as a legitimate source can raise throughput fourfold at equal entropy [32]. Yang, Blaauw and Sylvester show that process variation — larger in 40 nm than in 180 nm — must be absorbed by a configurable ring with a tuning loop for the generator to survive −40 to 120 °C and 0.6 to 0.9 V [104]. And two attacks show why a security device cannot ignore its environment: a frequency injected on the supply locks the rings and removed the jitter of a commercial secure microcontroller, reducing its key space from 2^{64} to 3300 [1], and a few hundred microwatts of radiated field biased a fifty-ring generator until it failed every test [105]; the published defence combines supply filtering with the same calibration

loop [104], and device aging under bias-temperature instability and hot-carrier injection degrades poorly-timed generators over years while leaving well-timed ones intact.

Access to silicon is more open than it appears. The University of Sevilla and IMSE-CNM are members of Europractice [106], whose academic price list is public: a mini@sic block costs 3 430 € in UMC 180 nm and 3 691 € in TSMC 65 nm, and IHP's SG13G2 is offered at 5 110 €/mm^2 with a free run for open-source designs under 2 mm^2 [107, 108, 109]. The institute's hardware-security area has already taken ring-oscillator primitives from Xilinx 7-series FPGAs, evaluated with SP 800-22 and SP 800-90B, to characterized 65 nm silicon [110, 111, 112, 113], and has fabricated cryptographic circuits before [114]. The open standard-cell libraries that make a synthesis chapter possible — IHP SG13G2, SkyWater SKY130, GlobalFoundries GF180MCU and the predictive Nangate 45 nm — come with liberty, LEF and GDS views, and the literature compares such designs in gate equivalents normalized to the minimum NAND2 of the library [115, 116, 117, 118, 119], with the caution that logic-synthesis area is a lower bound that grows by a factor of 1.7 to 4.6 to routed core area.

2.10 Gap addressed by this work

No absolute novelty is claimed. The ERO and its model, the embedded jitter measurement, the survey of AIS-compliant cores, the calibrated COSO on 7-series devices and an open implementation on the very die of the Basys 3 all exist, and the institute's own group has crossed from 7-series FPGAs to 65 nm silicon with ring-oscillator primitives; so do SRAM start-up entropy and multi-source harvesters with a cryptographic conditioner, on the RP2040 among others. What was *not found* in the literature consulted, and what this work sets out to provide in a modest but verifiable way, is the following.

1. A per-period jitter figure for the Artix-7 XC7A35T in absolute units, obtained inside the device with a differential method, with the thermal and flicker components separated and with stated uncertainty — the published Artix-7 coefficients being in counter units.

2. The complete chain *measured jitter* → *quality factor* → *min-entropy bound* → *divider* → *health-test cutoffs*, applied to one platform and confronted with the ten non-i.i.d. estimators of SP 800-90B and with the PTG.2 requirements of AIS 20/31 *version 3.0*, which most of the earlier literature could not use.

3. An SP 800-90B characterization of the ESP32 generator in its operating states, and a controlled comparison between a white-box FPGA source and a black-box commercial one under the same methodology.

4. The use of a commodity microcontroller with its own cryptographic accelerators as an independent oracle for an FPGA engine, over a real bus, with known-answer, interoperability and challenge-response tests.

5. A single-primitive key-generation engine — AES for conditioning, generation and authentication — written as portable RTL, verified against official vectors, synthesized on an open 130 nm library with the result stated as a lower bound, and costed for an academic shuttle.

Each item is formulated as "not found" rather than "does not exist". One earlier claim of this kind in the working notes — that no Artix-7 jitter measurement had been published — had to be corrected when Benea *et al.*'s paper was found, and the correction is kept visible there.

Chapter 3

The ESP32 Generator as a Black Box

This chapter is the first part of the work. It treats the hardware random number generator of the ESP32 as an integrator meets it: a register that returns 32-bit words, behind which the manufacturer names two noise sources and quantifies neither. The generator is run under four operating conditions in four capture campaigns on three chips; its output goes through a self-validated SP 800-22 battery, through Shannon and spectral metrics and through the SP 800-90B estimators; a key harvester is built around it; and a restart test opens the box as far as it can be opened from outside. Every figure is [MEASURED] on silicon unless it is said to be calculated. The chapter ends by stating why none of these measurements bounds the entropy of the generator, which is the reason for the rest of the thesis.

3.1 The peripheral and what Espressif documents

The device is the original ESP32, a dual-core Xtensa LX6 system-on-chip; the three chips measured here are ESP32-D0WD-V3, silicon revision 3.1. According to the Technical Reference Manual [44], every 32-bit word read from `RNG_DATA_REG` (address 0x3FF75144) is a true random number, generated from thermal noise and from the mismatch between asynchronous clocks; the noise comes from the high-speed ADC of the radio, from the SAR ADC or from both, and enters the generator through an XOR gate. The SAR ADC brings 2 bits per cycle of the 8 MHz `RC_FAST_CLK`, so the register should be read at no more than 500 kHz; the high-speed ADC brings 2 bits per cycle of the 80 MHz APB clock, which allows 5 MHz. With neither converter enabled, the manual states, pseudo-random numbers are returned. The software guide [43] puts it in operational terms: the output is true random while Wi-Fi or Bluetooth is enabled, while the SAR source switched on by `bootloader_random_enable()` is active — it must be switched off before the application

uses the ADC, I2S or the radio — or while the second-stage bootloader runs, and otherwise it "should be considered as pseudo-random only". Applications read the generator through `esp_random()`, which busy-waits between words so that fresh noise can enter, or through `esp_fill_random()`, which fills a buffer with it. For the ESP32 the wait is 16 APB cycles, a ceiling of 5 million words per second (`hw_random.c`, unchanged on this point from ESP-IDF 5.2.1 to 5.5.2, which bracket the 5.3.2 used here); the 15–75 kHz quoted by the guide match the longer waits coded for later chips.

The only validation Espressif publishes is a 2 GB sample, read at 5 MHz with only the high-speed ADC enabled, that passed the Dieharder suite 3.31.1 [44]; the data are not public, and the sample covers the favourable condition only. Nothing is published about the internal state the noise is mixed into, nor any model of the noise or min-entropy figure, and the literature consulted applies only SP 800-22 or Dieharder to the generator (Section 2.4). To an integrator it is a black box with two documented inputs that can be switched on and off, and that is how it is measured here.

3.2 Experimental design and capture

The two documented sources are crossed as two binary factors, which gives four conditions: A, Wi-Fi and ADC; B, Wi-Fi only; C, ADC only; D, neither. "Wi-Fi on" means the radio started in station mode with `esp_wifi_start()`, without associating to any network, since the documentation ties the noise to the RF subsystem being enabled, not to traffic. "ADC on" means a background task that keeps the SAR ADC converting on channel 0 of ADC1 (GPIO36, floating) as fast as the one-shot driver allows. That is not the documented SAR source, the noise-sampling mode of `bootloader_random_enable()`, and Espressif does not say whether ordinary conversions reach the generator; the statistics below cannot tell, and only the restart test of Section 3.6 shows that they do, without saying how much they bring. The conditions run in the order D, C, B, A from a cold boot: D first, because once the radio has run a radio-off state is no longer that of a device whose radio has never been on, and each later step adds one source. The firmware waits 500 ms after each change, and 200 ms more after starting the radio, so that each sequence belongs to one stationary condition.

Each condition yields 125 000 bytes, 10^6 bits, the minimum length SP 800-22 recommends for its full battery [38] and the minimum sample count of SP 800-90B [6]. The sequence is generated into RAM with `esp_fill_random()` while the condition holds and only then streamed, with the ADC task stopped, because ADC activity during long serial bursts was found to corrupt them; before it, the firmware times the generation of 64 KiB, and it reads the die temperature before and after. The stream is hexadecimal, in frames that carry the condition, the repetition, the temperature, the rate and a CRC-32 computed on the device; the host accepts a sequence only if the CRC matches and

the length is exactly 125 000 bytes. The console runs at 115 200 baud, the only rate at which the application output arrived reliably (about 25 s per condition), with logging at `ERROR` level because the Wi-Fi driver's messages broke frames. Before the cycle, `app_main` copies a 4 KiB buffer from the uninitialized `.noinit` section, the SRAM pattern of Section 3.5. The firmware is in `firmware/esp32_rng`, the tools in `analysis/esp32_rng` and `analysis/campana_rng.py`.

The firmware runs the CPU at 240 MHz, so that the rates of Section 3.4 refer to one fixed clock, with the FreeRTOS tick at 1 kHz. After the SRAM copy, `app_main` prints the MAC address and the ESP-IDF version and starts a single campaign task pinned to core 1. The ADC task of conditions A and C runs on core 0, at a lower priority, beside the Wi-Fi driver task, which ESP-IDF places on core 0 by default (Figure 3.1). The campaign task runs the four conditions, prints 100 harvester seeds and, after 2 s, starts the next cycle, indefinitely. Every artefact leaves the device as a frame between a `BEGIN` and an `END` line — `PUF` for the SRAM pattern, `RNG` for a sequence, `SEEDS` for the seeds, and `CYCLE` around a whole cycle — with its metadata as key–value pairs in the header, a payload of 64 bytes per hexadecimal line, and the CRC-32 of the raw bytes, from `esp_rom_crc32_le`, on the closing line. A sequence is streamed in 4 KiB chunks, yielding one scheduler tick between chunks to the watchdog and the UART driver, after a 100 ms pause once the ADC task has stopped.

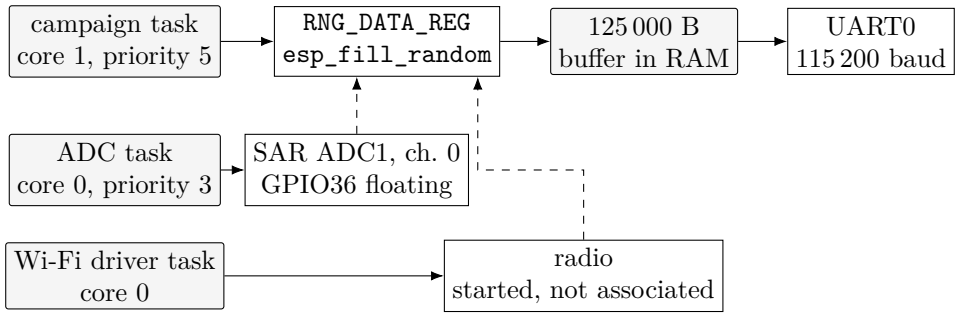


Figure 3.1: Tasks and cores of the capture firmware. The campaign task reads the generator into RAM while the condition holds; the ADC task (conditions A and C) and the Wi-Fi driver (A and B) run on the other core. The dashed arrows are the paths by which the documented sources may reach the generator. The ADC task is stopped before the buffer is streamed.

The serial rate was not the first choice. At 460 800 and 921 600 baud the banner of the second-stage bootloader still arrived, at the ROM default rate, but the application output never did; the cause was not investigated, and 115 200 baud was kept. At that rate the 250 000 hexadecimal characters of one sequence need about 22 s (calculated, ten line bits per character, newlines and frame lines left out), against the about 25 s observed per condition. On the host (`analysis/esp32_rng/capture_rng.py`), a small state machine collects the hexadecimal lines between the markers and ignores anything else outside a block; before decoding it removes any character that is not hexadecimal, as a stray log

fragment would leave, and drops an odd trailing nibble, so that an interleaved line ends as a CRC rejection rather than as a decoding failure. One more failure appeared on the bench: on current transients, most often with the radio and the ADC both active, the board restarted and its USB–serial bridge, a CP2102, re-enumerated, which invalidated the open port in the middle of a read. The cause was taken to be a brown-out and not measured. The capture tool then drops the block in progress, reopens the port until it reappears and goes on; and it numbers the repetitions itself, by the count of valid sequences it already holds for each condition, because the firmware's counter restarts with the board. A reconnection can therefore lose a block but never duplicate or overwrite one.

The firmware also contains an optional telemetry path, `mqtt_reporter`, implemented and never activated. A configuration menu (`Kconfig.projbuild`) takes the network name, its password and the broker URI, so that no credential is compiled from the source; after the first cycle the firmware tries to associate and connect only if the network name differs from its placeholder, and otherwise reports that MQTT is disabled and carries on over the serial link. Active, it would publish the condition, the die temperature and the rate on the topics `esp32/rng/condition`, `esp32/rng/temperature` and `esp32/rng/throughput`, and the last seed of each cycle on `esp32/harvester/seed`. No campaign set the credentials, so the radio was never associated, as the conditions require. Activating it would have changed the later cycles, with an associated radio, and the default broker URI is plain `mqtt://`, so the seed would have crossed the network in clear: as written it is a bench telemetry path, not one for key material. The sequences were never meant for it, since conditions C and D need the radio off.

Table 3.1: The four capture campaigns [MEASURED]. Each cycle is one cold boot followed by the four conditions in the order D, C, B, A, 10^6 bits each.

Campaign	Date	Chip (MAC)	Firmware built with	Cycles
First	2026-08-04	ec:e3:34:9a:d1:18	ESP-IDF 5.3	1
Second	2026-09-19	ec:e3:34:9a:d1:18	ESP-IDF 5.3.2	2
Third	2026-10-12	a0:dd:6c:0e:fb:4c	ESP-IDF 5.3.2	3
Fourth	2026-10-20/21	f8:b3:b7:2a:09:28	ESP-IDF 5.3.2	3 + 3

Four campaigns were run (Table 3.1). The first captured one cycle and the SRAM patterns on the chip of MAC address ec:e3:34:9a:d1:18, which the firmware reports at boot and which was noted with the results; its capture log has no MAC column. The same firmware, rebuilt unchanged, then ran two more cycles on that chip, three on a second one, a0:dd:6c:0e:fb:4c, and two batches of three on a third, f8:b3:b7:2a:09:28, the first batch with the very binary used on the second chip. On the third chip the serial link to the host occasionally corrupted a block on the host side; the CRC rejected all three such blocks, two were taken again from the next cycle of the same boot, and one incomplete cycle was set aside and repeated. Every sequence kept has the right length and CRC. The data are in `results/esp32_rng_agosto` and in `results/esp32_campana_rng` with

a subdirectory per chip, each with a log of the temperatures, rates and CRC verdicts. The first-level SP 800-22 battery, the Shannon metrics and the harvester of the next three sections use the first campaign; the second-level SP 800-22 analysis uses all four, and the SP 800-90B estimators of Section 3.6 all but the second batch of the fourth, which is kept for one check.

The firmware does not stop after one cycle: it comes back to condition D with the radio stopped but having run, and the first design meant to compare those later D sequences with the cold-boot one, to see whether the residual state of the radio changes the output. None was captured. Every condition D sequence kept, in the four campaigns, comes from the first cycle after a cold boot; the two blocks retaken from a second cycle on the third chip were of condition A. The restart test of Section 3.6 answers the question more directly than a later D could have: once the radio has run, the generator has received noise (Table 3.5), so a later D continues from a state that is no longer the fixed one, and the statistics below, which do not separate D from A, would not have separated the two Ds either.

3.3 The SP 800-22 battery

The fifteen tests of SP 800-22 Rev. 1a [38] were implemented from the definitions of the standard rather than taken from a library (`analysis/esp32_rng/nist_sp800_22.py`), with the bits taken most significant first and each test decided at $\alpha = 0.01$. The five tests that produce several p values — non-overlapping templates, serial, cumulative sums, random excursions and its variant — keep the minimum, so that a test passes only if all its parts do; the non-overlapping template test runs the first 8 of the 148 templates of length 9 that the standard lists. A self-test runs the battery on three inputs of 125 000 bytes: data from `os.urandom` must pass with at most one failure, an all-zero buffer must fail, and the alternating pattern `0x55` must be rejected by the runs or the approximate-entropy test. It passes: the zeros fail all fifteen tests, and the alternating pattern fails all but the frequency, block-frequency and cumulative-sums tests, which a perfectly balanced sequence must pass.

The length of 10^6 bits is also what lets every test run in the regime its approximations need (calculated from the parameters of the standard). The matrix-rank test uses 32×32 matrices of 1024 bits and needs at least 38 of them for its three-class χ^2; 10^6 bits give 976. Maurer's universal test reaches its setting of $L = 7$, with $Q = 1280$ initialization blocks, only from 904 960 bits. The two random-excursion tests apply only when the ± 1 walk of the sequence returns to zero at least $J = 500$ times. For a symmetric walk of n steps $E[J] \approx \sqrt{2n/\pi}$, so the threshold asks for about 3.9×10^5 bits on average, and at 10^6 bits $E[J] \approx 800$. The spread is wide: $J/\sqrt{n}$ tends to the absolute value of a standard normal variable, so a perfect source of 10^6 bits falls below 500 returns with probability

0.38. The standard then treats the excursion tests as not applicable; the implementation here records the case as a failure with a note, so a short walk cannot pass unnoticed, and the four sequences of the first campaign all had enough returns. The same spread is why only 34 of the 59 sequences of the second-level analysis below enter those tests.

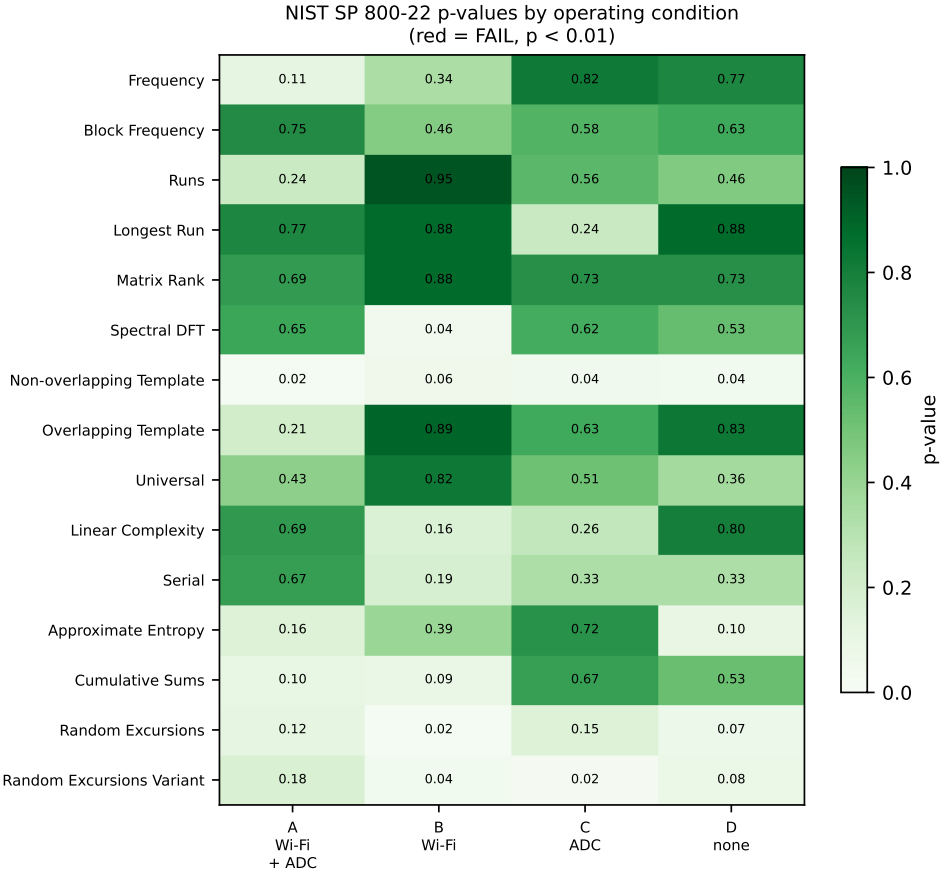


Figure 3.2: SP 800-22 p values of the four sequences of the first campaign, one per condition and test, 10^6 bits each [MEASURED]. A test fails below 0.01; none does. Tests with several p values show their minimum.

On the four sequences of the first campaign the battery passes all fifteen tests in all four conditions (Figure 3.2). The length bounds what that can mean. A perfect source fails at least one of fifteen tests with probability $1 - 0.99^{15} \approx 0.14$ (calculated), so only a failure that reproduces would indicate a defect, and with one sequence per condition the second-level analysis of the standard — the proportion of passing sequences and the uniformity of their p values — is not possible. The smallest of the sixty p values, 0.017, belongs to the template test in condition A, and it is expected: the minimum of k uniform p values has a median of $1 - 2^{-1/k}$, which is 0.083 for eight templates and 0.038 for the eighteen states of the excursions variant (calculated), and the template row (0.017 to 0.055) and the variant row (0.020 to 0.176) sit around those medians. Nothing separates the conditions.

With the later campaigns the battery can be read as the standard intends, at the

second level (Section 4.2 of SP 800-22): fifty-nine sequences of 10^6 bits — the forty-eight of conditions A–D on the three chips and eleven pieces of the two long captures E and F of Section 3.6 — with each p value of the multi-part tests taken separately, forty-eight statistics in all. Every statistic passes the proportion criterion (at least 0.951 of the sequences passing at $\alpha = 0.01$, 0.939 for the excursion tests, which apply to the 34 sequences with enough cycles), and the p values are uniform: the smallest $P\text{-value}_T$ is 0.0075, against a threshold of 10^{-4} (indicative for the excursion tests, below the 55 sequences the standard asks for). Eighteen of the 2182 p values fall below 0.01, where 21.8 are expected. The second level confirms what the first said, with more power, and adds nothing about predictability.

A pass says that fifteen fixed statistics do not distinguish these 10^6 bits from independent, unbiased bits at the 1 % level. It does not say that they are unpredictable: a deterministic generator with a good output function passes by design, which is why NIST decided in 2022 to revise SP 800-22 and rejected its use for assessing cryptographic generators [4]. Condition D is the case in point. The battery passes it, Espressif calls it pseudo-random, and the restart test of Section 3.6 shows that without the boot noise it is a fixed sequence. The battery serves here as a check for gross defects — bias, periodicity, short-range correlation — and finds none.

3.4 Shannon entropy, throughput and temperature

Six further metrics were computed on the same four sequences (`analysis/esp32_rng/entropy_analysis.py`). Table 3.2 gives them next to the value a perfect source returns at the same length, which, rather than the ideal of an infinite sequence, is the reference.

Table 3.2: Statistical metrics of the first campaign, one sequence of 10^6 bits per condition [MEASURED]: Shannon entropy per byte and per bit, largest autocorrelation $|r_k|$ for lags k from 1 to 1000, spectral flatness of one periodogram of the first 65 536 bits, zlib compression ratio and byte χ^2. The last row is what a perfect source returns at this length (calculated, see text).

Condition	H (bit/byte)	H (bit/bit)	$\max \lvert r_k \rvert$	Flatness	zlib	χ^2 p
A	7.99852	0.9999982	0.00375	0.564	1.00037	0.473
B	7.99852	0.9999993	0.00345	0.564	1.00037	0.470
C	7.99877	1.0000000	0.00332	0.563	1.00037	0.972
D	7.99836	0.9999999	0.00314	0.561	1.00037	0.109
Perfect source	7.99853 ± 0.00013	$1 - 7 \times 10^{-7}$	0.0034	0.561	> 1	uniform

Each metric has a floor of its own at 10^6 bits, and the reading starts there. The plug-in entropy of 125 000 bytes over 256 values falls short of 8 bits even for a perfect source, by $\chi^2_{255}/(2N \ln 2)$, that is 0.00147 ± 0.00013 bit; the four measured shortfalls, 0.0012 to 0.0016 bit, are that bias, and the byte entropy and the byte χ^2 are the same statistic in

two units. The entropy per bit restates the monobit test. The autocorrelation at each lag has a standard deviation of $1/\sqrt{n} = 0.001$, and the largest of a thousand such values has a median of 0.0034 for white noise. The flatness of a single unaveraged periodogram of white noise is not 1 but $e^{-\gamma} \approx 0.561$, with γ Euler's constant, because each bin is exponentially distributed. Figure 3.3 shows the two distributions behind those columns: in all four conditions the thousand standardized autocorrelations follow a standard normal distribution and the periodogram bins an exponential one, as for white noise. In every column the four conditions sit where a perfect source would.

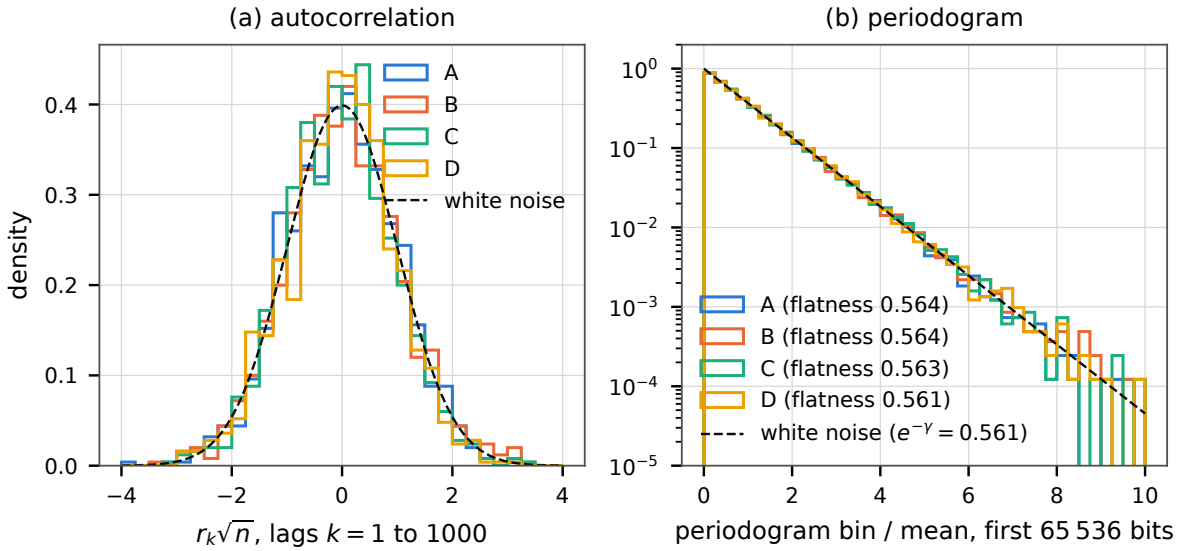


Figure 3.3: Autocorrelation and periodogram of the four sequences of the first campaign [MEASURED], against white noise (dashed). (a) Distribution of $r_k\sqrt{n}$ for lags k from 1 to 1000, $n = 10^6$; for white noise it is standard normal. (b) Distribution of the periodogram bins of the first 65 536 bits, divided by their mean, on a logarithmic scale; for white noise it is exponential, which is why the spectral flatness of a single periodogram tends to $e^{-\gamma} \approx 0.561$ and not to 1.

Like SP 800-22, these are averages over the output that a deterministic sequence can reach, and Shannon entropy is not the quantity a key needs: key generation counts min-entropy, the negative logarithm of the probability of the most likely outcome, which is never larger and is the subject of Section 3.6.

The rate is timed by the firmware with the 1 μs system timer over 64 KiB, about 12.6 ms, so its resolution is better than 0.01 % (calculated). In all four campaigns the twenty-four sequences without the ADC task were generated at 5.21 to 5.22 MB/s (10^6 bytes per second), and the twenty-four with it at 2.77 MB/s in the first two campaigns and 2.70 MB/s in the last two; the radio changes the rate by less than 0.1 %. The ADC task halves the rate, for a cause that was not isolated. Set against the manual, 1.30 million words per second without the task is under the 5 MHz of the radio path, and 0.68 to 0.69 million with it is above the 500 kHz of the SAR path: conditions A

and C read 21.6 to 22.2 Mbit/s while the SAR path injects at most 16 Mbit/s, so even if the conversions of condition C reached the generator and every injected bit were fully random, no more than 0.72 to 0.74 bit of fresh noise could stand behind each output bit (calculated). The radio path, at 160 Mbit/s, exceeds the highest read rate, 41.7 Mbit/s, nearly fourfold.

Temperature comes last because its instrument is the coarsest. The firmware reads the sensor through an undocumented ROM function that returns an uncalibrated 8-bit integer in degrees Fahrenheit, so one step is 5/9 °C, 0.556 °C. In the first campaign it returned only 52.2 and 52.8 °C, which are 126 and 127 °F: the whole temperature range of the campaign is one step of the sensor. The correlation between byte entropy and temperature over the four sequences, $r = 0.795$ with $p = 0.21$, was therefore computed against the least significant bit of the sensor, and it carries no information; no temperature effect is claimed, in either direction. Figure 3.4 shows the data in the sensor's own unit: one sequence read 126 °F throughout, two read 127 °F, one moved from the first value to the second while it was generated, and all four entropies lie within two standard deviations of a perfect source of the same length. The three identified chips later read 49.4 to 51.7, 61.7 to 64.4 and 42.8 to 45.0 °C on the same uncalibrated scale, so chip and temperature are confounded. A temperature study needs a calibrated sensor and a controlled sweep, and none was made.

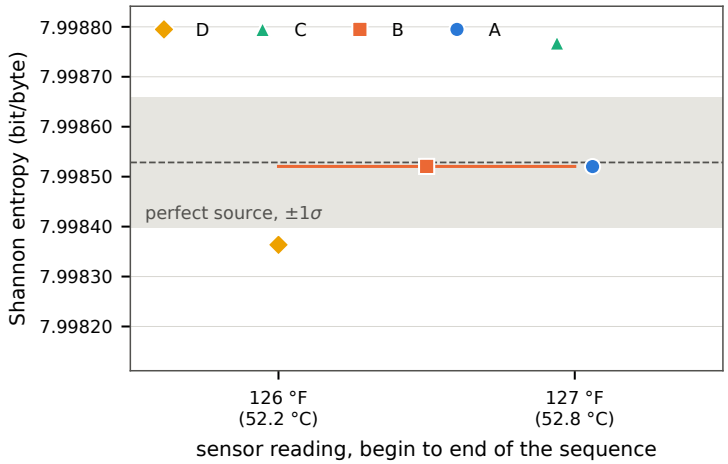


Figure 3.4: Byte entropy of the four sequences of the first campaign against the reading of the die sensor [MEASURED], in its own unit, an uncalibrated integer in degrees Fahrenheit; a bar spans the readings before and after the sequence. The band is a perfect source of 125 000 bytes, mean and one standard deviation (calculated). The whole campaign is one step of the sensor, so the correlation of $r = 0.795$ computed over these points says nothing about temperature.

3.5 A multi-source key harvester

If the generator cannot be trusted with the radio off, a node that keeps its radio off needs other sources. This part of the work built a harvester (`firmware/esp32_rng/main/entropy_harvester.c`) that adds two sources of the same chip whose availability complements that of the generator: the SRAM power-up pattern, present at every power-up before any peripheral starts [65], and the least significant bits of the ADC reading a floating input. These pass through the corrector of von Neumann [120], which discards the pairs 00 and 11 and maps 01 to 0 and 10 to 1, so that independent bits come out unbiased at the cost of at least three quarters of them. The contributions are conditioned with SHA-256 into a 256-bit pool,

$$\mathrm{pool}_0 = \mathrm{SHA256}\big(r \,\|\, \mathrm{SHA256}(S) \,\|\, a \,\|\, \texttt{"TFM0"}\big), \tag{3.1}$$

where r is 32 bytes of `esp_fill_random()`, S the 4 KiB SRAM pattern, a 32 bytes of debiased ADC bits and the last field a tag that keeps this use of the hash apart from the others. Each 32-byte output hashes the pool with a counter and a fresh read of the generator, and the pool is then ratcheted forward,

$$o_i = \mathrm{SHA256}(\mathrm{pool} \,\|\, i \,\|\, r_i), \qquad \mathrm{pool} \leftarrow \mathrm{SHA256}(\mathrm{pool} \,\|\, o_i), \tag{3.2}$$

so that a pool captured after output i does not give the earlier ones; a reseed hashes fresh generator and ADC data into the pool. Concatenating before hashing gives the one guarantee that needs no independence between sources sharing die, supply and temperature: $H_\infty(X_1 \| X_2 \| X_3) \geq \max_i H_\infty(X_i)$.

The corrector's claim rests on one line. For independent bits with an unknown probability p of a one, $\Pr[01] = (1-p)\,p = \Pr[10]$, so a kept pair yields 0 and 1 with probability $1/2$ each, whatever p is. A pair is kept with probability $2p(1-p)$ and gives one bit for two, so the yield is $p(1-p)$ output bits per input bit, at most $1/4$, at $p = 1/2$ (Figure 3.5). The corrector removes bias, not correlation: for dependent bits $\Pr[01]$ and $\Pr[10]$ need not be equal. The firmware takes the least significant bit of each 12-bit conversion, packs eight to a byte, corrects 2048 of them at a time and repeats until it has 32 bytes. No separate capture of these bits exists, so neither their bias nor their correlation was measured.

Hashing the concatenation is justified, under assumptions, by the theory of randomness extractors. The leftover hash lemma [70] states that a function drawn at random from a universal hash family, applied to an input of min-entropy k, gives an ℓ-bit output within statistical distance ε of uniform provided

$$k \geq \ell + 2\log_2(1/\varepsilon); \tag{3.3}$$

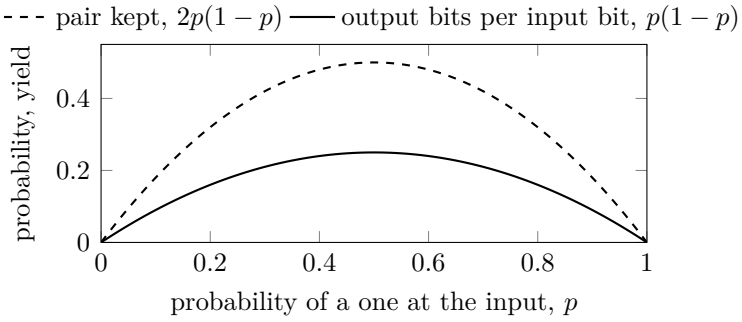


Figure 3.5: Yield of the von Neumann corrector on independent input bits (calculated). Even an unbiased input loses three quarters of its bits; the output is unbiased for every p in $(0, 1)$ only if the input bits are independent.

for $\ell = 256$ and $\varepsilon = 2^{-32}$ the input needs 320 bits of min-entropy. SHA-256 is one fixed function, not a member of a family drawn at random, so the lemma does not apply to it as stated; its use as an extractor rests on idealized models, or on analyses such as that of Dodis et al. [71] for the HMAC and cascade constructions under assumptions on the compression function, and SP 800-90B admits an approved hash function as a vetted conditioning component [6]. What matters here is the condition rather than the bound: the pool is close to uniform only if the inputs together hold some 320 bits of min-entropy, and the paragraphs below show that this was measured for none of them. The concatenation bound follows from an inequality that holds for any joint distribution, because the most likely triple is no more likely than the most likely value of any one component,

$$\max_{x_1,x_2,x_3} \Pr[X_1{=}x_1, X_2{=}x_2, X_3{=}x_3] \;\le\; \min_{i} \max_{x_i} \Pr[X_i{=}x_i], \tag{3.4}$$

and taking $-\log_2$ of both sides gives it. XOR-combining sources of equal width also keeps the best min-entropy, but only if the sources are independent, which three sources on one die cannot be assumed to be; concatenation needs no such assumption, at the cost of a longer hash input. If the ADC fails to deliver, its field is filled with zeros and the pool is formed from the other two sources.

The ratchet of Equation (3.2) gives forward secrecy: the pool after output i does not reveal earlier outputs as long as SHA-256 cannot be inverted. The reseed that follows the 100 seeds of each cycle hashes 32 fresh generator bytes and 32 ADC bytes into the pool, $\text{pool} \leftarrow \text{SHA256}(\text{pool} \,\|\, r \,\|\, a)$, and is meant to give backward secrecy, recovery after the pool has been exposed; it does so only to the extent that r and a carry entropy the attacker cannot predict, which is precisely what is not known with the radio off. The construction resembles Hash_DRBG of SP 800-90A [89] — a hashed state, outputs that hash the state, reseeds from a live source — but aims at something narrower: a hundred 256-bit seeds per cycle, each to be expanded by a key derivation function, instead of long

output streams, with every output absorbing a fresh generator read and the whole state in one 256-bit word. It is a small absorb-and-squeeze conditioner, without the reseed counters, health tests or validation of an SP 800-90A generator.

The harvester was validated offline. The seeds produced on the device were not captured reliably in the first campaign, and the later capture scripts stopped listening once the four conditions were in, before the seeds are printed; on the third chip, where listening went on to retake a corrupted block, the firmware did print them and started a second cycle, so seed generation runs on the device, but its output was not analysed. The validation therefore reproduces the construction in Python (`analysis/esp32_rng/harvester_from_sources.py`) over captured data: one SRAM pattern; the condition-D sequence as the generator contribution, the worst condition, on purpose; and 32 bytes of the same sequence standing in for the ADC field, of which no separate capture exists. Of the 1000 seeds of 32 bytes so derived, all are distinct and their bytes are uniform (χ^2 $p = 0.30$); the Hamming distance between two seeds, over 20 000 random pairs, averages 128.05 of 256 bits (50.02 %) with a standard deviation of 8.03, against 128 and 8 for independent uniform seeds; and the seven SP 800-22 tests that apply to the 256 000 bits of the seed stream pass, with p from 0.148 to 0.956 (Figure 3.6).

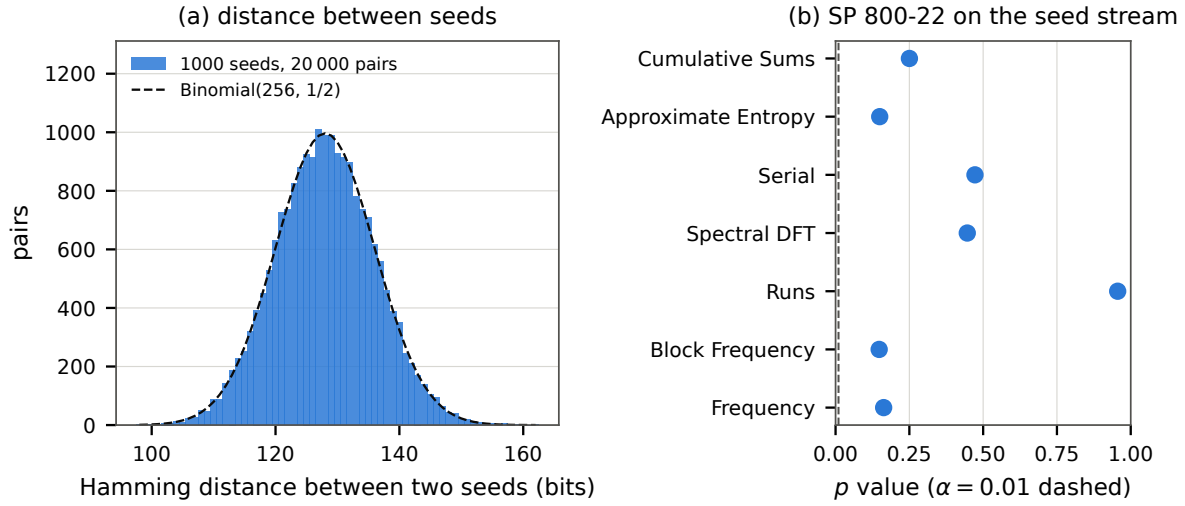


Figure 3.6: Off-line validation of the harvester construction (calculated, on measured inputs). (a) Hamming distance between two of the 1000 seeds derived from one SRAM pattern and the condition-D sequence, over 20 000 random pairs, against the binomial distribution of independent uniform seeds. (b) p values of the seven SP 800-22 tests that apply to the 256 000 bits of the seed stream. Both are passed by construction (see text) and say nothing about the entropy of the seeds.

The SRAM source was characterized on its own. Only a removal of power re-randomizes the cells, since a reset that keeps the supply leaves in the buffer whatever the previous run wrote, so a clean series of four patterns was captured with a power cycle before each. The nine patterns captured have a Hamming weight of 49.6 to 49.9 %, and the six distances of

the clean series span 6.45 to 6.81 %, with a mean of 6.63 %. The fraction of stable cells, those that read the same value in all four power-ups, was measured directly: 87.90 % in the clean series and 87.64 % in the four power-ups of the first, mixed series, so about 4000 of the 32 768 cells flipped at least once. The first analysis had inferred about 93 % from the mean distance, which counts how often two power-ups differ, not how many cells ever do; the counted figure is the one used (Section 1.5). The inter-boot distance of 23.9 % first computed for the source is not a clean figure: it averages ten pairs of a series that mixed power cycles with resets that kept the supply, in which one pattern differs from the other four in 49.6 to 49.7 % of its bits, as unrelated data would, and the other six pairs differ in 6.5 to 7.4 %, as power-ups of the same cells do. The clean mean of 6.63 % lies within the 3 to 8 % reported for the start-up noise of SRAM cells [65, 67]; with one board, four power-ups and one temperature, that says only that this SRAM behaves like those of the literature, not how much it varies.

The validation shows that the construction was implemented as designed, not that the seeds carry entropy. Every check is passed by construction: the counter makes every output distinct even if all other inputs were constant, and SHA-256 turns any distinct inputs into outputs that look uniform and lie half a word apart. The inputs of this validation make the point. The SRAM field was one pattern for all 1000 seeds — the first of the mixed series, which is not a power-up state — and the only input that changed between seeds was condition-D output, which the restart test shows to be a fixed sequence once the boot noise is removed. The entropy of a seed is at most $\min(256, H_\infty)$ bits, H_∞ being the min-entropy of the inputs, and that was measured for none of them. A weight of 50 % does not change this: 88 % of the SRAM cells are a fixed fingerprint, known to anyone who has once read the pattern, and bounding what the other 12 % contribute would take many more power-ups and a model of the cells. The harvester is an architecture whose output is at least as strong as its best source, not a measured source of entropy.

The intended use was demonstrated off-line (`analysis/esp32_rng/mqtt_tls_demo.py`) for a generic connected node, a thermostat publishing telemetry. A 32-byte seed is expanded into a 256-bit session key with HKDF-SHA256 [72], which first condenses the seed into a pseudorandom key and then expands it,

$$\mathrm{PRK} = \mathrm{HMAC}_H(\mathrm{salt}, \mathrm{seed}), \qquad K = \mathrm{HMAC}_H(\mathrm{PRK}, \mathrm{info} \,\|\, \texttt{0x01})_{1..L}, \tag{3.5}$$

with H = SHA-256, no salt (HKDF then uses a string of zeros of the hash length), info = `"tfm-mqtt-session"` and $L = 32$ bytes, one block because L equals the hash length; changing info derives independent keys for other purposes from the same seed. The key drives AES-256-GCM with a 96-bit nonce and the topic `esp32/hvac/telemetry` as associated data, so that the 128-bit tag covers ciphertext and topic. On the first two seeds of the validation set, the same seed gives the same key, the two keys differ

in 130 of 256 bits, a 61-byte message makes the round trip, and decryption rejects the message presented under another topic and the message with one ciphertext bit flipped (calculated, on the PC). Two limits belong with that result. GCM alone does not reject a message replayed under the same topic, which needs a counter in the authenticated data, and the demonstration has none. And it takes its nonce from the operating system of the PC, not from the harvester; a repeated pair of key and nonce would break both the confidentiality and the authenticity of GCM, so on a device the nonce generator needs the same care as the key. The case the harvester was built for is a node that provisions its key in condition D shortly after a cold boot, where Section 3.6 shows that, without the boot noise, the generator returns one of a few thousand outputs, the same on two chips; deriving keys through a key derivation function from a conditioned pool rather than from the raw register is the practice RFC 4086 recommends [48]. The demonstration does not close that case: it exercises the cryptography, and the key inherits the entropy of the seed without adding any. A second mode of the script, an MQTT session over TLS through the paho client that would take a live seed from the topic `esp32/harvester/seed`, was implemented and never run. It accepts any broker certificate and receives the seed over the network, so it shows the plumbing, not a design for establishing keys.

3.6 The ESP32 reference under SP 800-90B

O8 asks for a numerical comparison against the ESP32, and half of it can be closed without the board: the reference side. Sections 3.3 and 3.4 applied SP 800-22 and Shannon entropy to the first campaign; the SP 800-90B estimators are applied here to the first three and to the first batch of the fourth, which together give nine repetitions of 10^6 bits per condition, three on each of the three chips of the same model (ESP32-D0WD-V3). Two further captures with the radio off, of 3.52×10^6 and 8.38×10^6 bits, were taken with the verifier firmware of Chapter 6. All thirty-eight were run through the National Institute of Standards and Technology (NIST) reference implementation of the SP 800-90B estimators [37, 6] (version 1.1.8), on both the IID and the non-IID tracks, in the binary layout the standard prescribes for bit sources. Table 3.3 gives the outcome.

The resolution comes first, as everywhere in this work. The MCV estimator returns $-\log_2 p_u$ with $p_u = \hat{p} + 2.576\sqrt{\hat{p}(1-\hat{p})/N}$, so even a perfect source cannot score above 0.9963 with 10^6 bits, 0.9980 with 3.5×10^6 or 0.9987 with 8.4×10^6. Every one of the thirty-eight captures sits within 0.0037 of that ceiling, and none implies an imbalance between ones and zeros beyond 2.6σ, which is what thirty-eight draws from an unbiased source look like.

The IID track needs a calibration before its verdicts can be read. Of the seventy-six runs (thirty-eight captures in the bit and byte layouts), five fail a single test, never the same file in both layouts: one permutation test in each layout and three χ^2 tests in the byte

Table 3.3: The ESP32 generator under the SP 800-90B estimators [MEASURED]. Conditions A–D: nine repetitions of 10^6 bits each, three on each of the three chips, mean and range over the repetitions; E and F: single captures of 3.5×10^6 and 8.4×10^6 bits. The ceiling is the highest value the most-common-value (MCV) estimator can return for a perfectly uniform source of that length at 99 % confidence; the non-IID figure is the minimum of the ten estimators of Section 6.3 of the standard, and the last column names the estimator that set it (times out of nine). The IID-track verdicts are discussed in the text.

Capture	Condition	H (IID)	Ceiling	H (non-IID)	Limiting
A	Wi-Fi + ADC	0.9950	0.9963	0.843 (0.812–0.893)	Compr. 7, Lag 1, Coll. 1
B	Wi-Fi	0.9952	0.9963	0.857 (0.809–0.912)	Compression 8, LRS 1
C	ADC	0.9948	0.9963	0.847 (0.812–0.918)	Compression 9
D	none	0.9955	0.9963	0.853 (0.832–0.916)	Compression 8, Collision 1
E (verifier)	none	0.9977	0.9980	0.867	Compression
F (verifier)	none	0.9982	0.9987	0.911	Compression

layout. How often good data fail these tests at the same length is not the nominal 10^{-3}, so it was measured, running the same tool on 1000 files of 125 000 bytes and 200 files of 10^6 bits from the operating-system generator. In the byte layout the permutation battery fails on 1.9 % of good files and the χ^2 pair on 2.4 %; in the bit layout the permutation battery fails on 2.5 % and the χ^2 pair on none of the 200. The χ^2 figure was traced to its cause by reimplementing both tests, which reproduces the tool's fifty-two p values of the first twenty-six captures to the sixth decimal, and running them on 5000 good files. The byte-layout independence test spreads 62 500 non-overlapping pairs over 65 536 cells and merges cells until each group expects five counts, but at this length its p is not uniform: the median is 0.175 instead of 0.5, and it falls below 10^{-3} on 1.7 % of the files, seventeen times the nominal rate. The other three χ^2 tests are well calibrated, and the defect disappears from 10^6 bytes upwards. Against these rates the ESP32 failures are unremarkable: one permutation failure in thirty-eight byte runs where 0.72 are expected ($P = 0.52$), three χ^2 failures where 0.91 are expected ($P = 0.063$), and one bit-layout permutation failure where 0.95 are expected ($P = 0.62$). On one chip the byte independence test gave $p < 0.05$ in six captures of twelve, against 2.9 expected; pooling each chip's captures, where the test is calibrated and more powerful, gives empirical p values of 0.84 and 0.94 on that chip for byte pairs aligned with the 32-bit word and offset by one byte, and 0.53 and 0.66, and 0.42 and 0.22, on the other two, so no dependence between neighbouring bytes is present. On the third chip the byte goodness-of-fit p values sat low without any failing (median 0.21 against 0.51 for good files, one-sided Mann-Whitney $p = 0.0031$, at the limit of a Bonferroni correction for sixteen comparisons); the second batch of twelve captures on that chip gave a median of 0.34 and $p = 0.13$, so it is read as chance. The rarest event is the goodness-of-fit failure of the first capture of condition D ($p = 2.4 \times 10^{-4}$): that test fails on 0.04 % of good files, so one such failure in thirty-eight has a probability near 1.5 %.

It did not reproduce in the eight later captures of the same condition ($p = 0.14$ to 0.96) nor in E and F, and it is recorded as chance; Espressif's warning that the generator is not guaranteed random without the radio or the ADC stands as a warning that these tests do not see. Formally, a single failure withdraws the IID assumption for that capture; since the figure carried forward is the non-IID one, nothing below depends on it. Read with its calibration, the IID track finds the ESP32 output indistinguishable from a full-entropy binary source at this sample size. That is the same verdict SP 800-22 gave, reached by a different route, and it says the same thing: nothing wrong is visible.

The non-IID track says something else, and reading it correctly matters more than the number. In thirty-seven of the thirty-eight captures its minimum is set by one of three estimators — compression, collision or longest repeated substring — at 0.81 to 0.92 bit per bit; in the remaining one by the lag predictor, at 0.88; the MCV and Markov estimators never drop below 0.99. Those three estimators are known to be conservative on nearly uniform data [121]; the predictors of Kelsey, McKay and Sönmez Turan [122] were introduced precisely because the earlier estimators of the standard misjudge sources of that kind. The repetitions show that this is estimator behaviour and not a property of the conditions. The compression estimate has a standard deviation of 0.03 to 0.04 *within* a condition, larger than the 0.013 that separates the four condition means, and a one-way analysis of variance on each of the eleven estimators finds no difference between conditions: the smallest p is 0.045, for the MultiMCW predictor, whose condition means follow no physical order, and one value below 0.05 among eleven tests is what chance provides; it is far above the 0.0045 that eleven tests demand. The same holds between chips. Grouping the conditions, which do not differ, the twelve captures of each chip are separated by no estimator at the corrected level; the smallest p, 0.032 for MCV, corresponds to a difference of about 0.001 bit per bit next to the resolution ceiling. The non-IID minimum is 0.852, 0.847 and 0.851 on the three chips ($p = 0.92$): with three devices, the reference does not depend on which one is measured, at the resolution of these estimators. The largest capture adds a second illustration of the resolution effect: in the byte layout its limiting estimator is the t-tuple test at $t = 2$, which returns 7.39 bits per byte because 10^6 bytes hold too few of the 65 536 possible pairs for the most frequent one to be representative, exactly as the MCV ceiling does at $t = 1$. No difference between Wi-Fi on, ADC on and neither is resolvable by any estimator with nine repetitions, and none is claimed.

The restart test of Section 3.1.4 of the standard answers what the statistics above cannot: if a generator has no noise of its own, its output after a cold restart must repeat. A dedicated firmware read 1024 bytes with `esp_fill_random` as the first action of the application, with the radio and the ADC off (condition D), and the chip of MAC a0:dd:6c:0e:fb:4c was reset through its enable pin 1000 times; the boot ROM reported a power-on reset every time. Two variants share the application and differ only in the second-stage bootloader: the stock one, which feeds SAR ADC noise into the generator

for about 126 ms while it runs, and the same bootloader with that call diverted to an empty function at link time, which the disassembly confirms. Three further variants read a second time after starting the Wi-Fi as in conditions A and B, without associating to any network, and waiting 200 ms, with each of the two bootloaders; the third is a control that waits the same time without touching the radio, so that the read falls on the same scheduler tick. Table 3.4 gives the outcome.

Table 3.4: Restart test of the ESP32 generator [MEASURED]: 1000 cold restarts per variant, one chip. H_r and H_c are the minimum of the non-IID estimators over the row and column datasets of the standard (1000 restarts × 1000 bits); the test fails when either is below half the initial estimate H_I, the lowest non-IID figure of this chip in the same condition (0.838 bit per bit in D, 0.859 in B). The byte layout gives the same verdicts. Overlaps are restarts whose output appears shifted inside another's, all identical word for word. The late reads fall 352 to 412 ms after the application starts, most of them at 352 ms.

Bootloader	Read	H_r / H_c	Distinct	Overlaps	Verdict
Stock	at boot, radio off	0.898 / 0.854	1000	0	passes
Boot noise removed	at boot, radio off	0.024 / 0.813	759	5424	fails
Boot noise removed	late, radio off (control)	0.024 / 0.808	760	5632	fails
Boot noise removed	late, Wi-Fi on	0.831 / 0.845	1000	0	passes
Stock	late, Wi-Fi on	0.844 / 0.807	1000	0	passes

With the stock bootloader the generator passes in both layouts, and the 1000 outputs differ from one another exactly as random data would: the mean Hamming distance between two restarts is 4096.0 bits of 8192, against 4096 ± 45 expected. Without the boot noise it fails decisively, and the data show why. Each restart is a window on one fixed sequence, read from a slightly different starting point: wherever the beginning of one restart's output appears inside another's, displaced by a few 32-bit words, the whole overlap matches word for word, 5424 times without a single difference. The only thing that changes between restarts is the instant at which the application starts reading, which varies by a few thousand clock cycles from one boot to the next. That is worth between 6.5 bits per restart, the bound of the standard from the most repeated output, and 10.8 bits if every starting point were equally likely, out of 8192 bits read. With the radio and the ADC off the generator has no noise of its own, and Espressif's statement that its output is then pseudo-random only [43] is confirmed directly, which no statistical test on a continuous stream could do. Two consequences follow. The condition D captures of Table 3.3 are the output of a deterministic generator seeded during boot: the estimators credit them with 0.85 bit per bit, as much as with the radio on, while their physical entropy is at most what the bootloader injected, which those estimators do not measure. And an application that derives a key right after boot with the radio off depends entirely on the stock bootloader's noise; without it, the key could be guessed in a few thousand attempts, the cold-boot problem described by Blanco-Romero et al. [3].

The radio rows answer the other half of Espressif's statement. With the boot noise removed, starting the Wi-Fi is enough for the test to pass: the 1000 outputs are all distinct, no window is shared, and the 289 pairs of restarts that read identical values before the radio was started read 289 different values after it. Timing alone does not explain this. The control waits the same time and reads on the same scheduler tick, with a read instant that varies as much (3824 against 5260 clock cycles among the restarts that share the tick), and it repeats exactly as at boot: 760 distinct outputs, 5632 identical overlaps, about 1700 states. With so few states, the 797 restarts that share a tick in the radio variant would show about 180 repeats by timing alone; they show none. Without the radio the generator receives no noise in its first 350 ms of operation; with the radio it does. What the test does not give is how much entropy per bit the radio delivers: the validated 0.83 bit per bit is the estimators' figure, with their usual bias, not a measurement of the radio noise.

The third chip, f8:b3:b7:2a:09:28, repeats the picture with the same two binaries: the stock bootloader passes (H_r / H_c 0.892 / 0.848, 1000 distinct outputs), and without the boot noise it fails (H_r = 0.016, 746 distinct outputs, 5444 identical overlaps, again 87 groups). More than that, the two chips repeat *each other*. 256 distinct 1024-byte outputs occur on both, covering 352 and 347 of their 1000 restarts; 988 of the third chip's outputs begin, at a whole number of 32-bit words, inside an output of the other, in 8369 overlaps without a single differing word; and the 3000 noise-free outputs of both chips and of a second firmware build fall into the same 87 groups, each holding outputs of all three. With the stock bootloader no output aligns with any other, within or across chips. Without the boot noise, the output of the two chips tested is therefore a deterministic function of a small starting point, and the same function on both: whoever has one chip and the firmware can precompute what another one returns. Two chips of one revision do not establish it for every ESP32.

A dedicated firmware showed what selects that starting point. With the boot noise removed and interrupts off, it reads 1024 bytes, waits a fixed number of instructions and reads again, the wait chosen per restart among four values. For the 204 pairs of restarts whose first read was identical, the second reads were identical when the wait was the same, shifted by exactly one 32-bit word when it was 174 CPU cycles longer (the time `esp_fill_random` spends per word), by exactly 100 words when it was 17 400 cycles longer, and unrelated when it was 2 cycles longer, one APB cycle, in every pair. The generator, without noise, steps with the APB clock from a fixed reset state, whether or not it is read, and its output depends only on the instant reading starts: the 87 groups are the 87 APB-cycle phases of a 174-cycle word, a property of this firmware and clock, not of the chip.

Table 3.5 answers the question left open in Section 3.2. Initialising the Wi-Fi driver, which powers the modem domain but not the radio, injects nothing; starting the radio does, even for the shortest exposure possible, because the PHY calibration inside

Table 3.5: Which source injects noise [MEASURED], third chip, boot noise removed: a first read at boot, one source, and a second read at a fixed 800 ms. A source injects noise if the second reads stop being windows of the fixed sequence (overlaps) and never repeat. 300 restarts per variant, 1000 for the ADC and the boot source, which also pass the restart test of the standard (H_r / H_c 0.864 / 0.877 and 0.840 / 0.842).

Source between the two reads	Distinct	Overlaps	Noise
None (control)	284 of 300	475	no
Wi-Fi initialised, radio not started	284 of 300	461	no
Radio started and stopped (85 to 262 ms on)	300 of 300	0	yes
ADC unit created and deleted, no conversion	274 of 300	0	no
ADC one-shot conversions for 1 ms (17)	300 of 300	0	yes
ADC one-shot conversions for 200 ms (about 4400)	1000 of 1000	0	yes
Boot source switched on by the application, 200 ms	1000 of 1000	0	yes

`esp_wifi_start` already keeps it on for about 85 ms. The boot source works when the application switches it on. And ordinary one-shot conversions of the SAR ADC, which neither the software guide nor the driver sources connect to the generator, inject noise too: seventeen are enough, while creating and deleting the converter without converting is not. The exact repeats of that last variant show it is still noise-free even though, in that build, its outputs share no shifted windows. A restart test of the ESP32 generator, and a comparison of its outputs between chips, have not been found in the literature consulted.

3.7 What a black box can and cannot tell

Every statistical instrument applied to the ESP32 generator gives the same answer. SP 800-22 passes all fifteen tests in all four conditions; the Shannon, autocorrelation, spectral and compression metrics sit where a perfect source would at this length; the IID track of SP 800-90B, read with its calibration, finds the output indistinguishable from a full-entropy binary source; and the non-IID track credits 0.81 to 0.92 bit per bit, with no difference between conditions or between chips that any estimator resolves. None of these figures distinguishes condition D, in which Espressif says the generator is pseudo-random, from condition A, in which both documented sources are active.

The restart test is the one measurement that does, because it asks a physical question rather than a statistical one: does the output repeat when the generator starts again from the same state? With the boot noise removed and the radio off it repeats, word for word, and repeats the same sequence on two different chips; with the radio, the boot source or the ADC converting, it does not. The generator has no noise of its own with the radio and the ADC off, and the radio does inject noise into it. The SP 800-90B figures of condition D are therefore the estimators' verdict on the deterministic continuation of a boot seed: a statistical figure, which cannot be credited as physical entropy. The same holds, less visibly, for the other conditions. The restart test shows that the radio injects

noise, not how much; for conditions A and C it shows that the ADC conversions do reach the generator, but by the manual's own rates the SAR path could not supply one bit of fresh noise per output bit at the rate the generator was read.

Some measurements would still sharpen the picture of the ESP32 itself; they were left undone because none of them can change that conclusion. The SRAM source needs tens of power-ups, at several temperatures, to map the flip probability of each cell: both to bound what the unstable cells contribute and to dimension the error correction of a fuzzy extractor that would turn the stable cells into a device key, an SRAM Physically Unclonable Function (PUF) binding each seed to its chip [65]. A temperature study needs a calibrated sensor and a climatic chamber, over -20 to $+85$ °C. The seeds produced on the device should be captured and analysed, not only reproduced off-line. And the restart test, the one measurement that separated the conditions, should be repeated on a later chip such as the ESP32-S3, whose generator the software guide documents with different read rates (Section 3.1).

What is missing is not a better test but a different kind of knowledge: a model of the source — which physical process produces the randomness, and with which parameters — and a measurement of those parameters that bounds the min-entropy from below. Neither can be obtained from outside the ESP32: its noise sources, internal state and post-processing are undocumented, and the SP 800-90B estimators were designed for raw noise samples, not for the output of a block that may already be conditioned. The second part of this work therefore builds the generator instead of measuring someone else's. Chapter 4 starts from the physics of a ring oscillator, extracts its jitter and turns it, through a validated model, into a lower bound on min-entropy; the chapters that follow build and verify the engine around that source. The ESP32 stays in the work in two other roles: its non-IID figure is the reference of the comparison in Section 8.7, and its cryptographic library is the independent implementation against which the engine is verified.

Chapter 4

Entropy Source: from Transistor to Min-Entropy

This chapter builds the chain that the rest of the thesis rests on: from the physical jitter of a ring oscillator to a defensible figure of min-entropy per output bit, and from that figure to the design parameters of the sampler. Every step is validated before it is used. The statistical estimators are validated against synthetic data of known jitter; the analytic bounds are validated against an exact numerical computation; and the transistor-level bench is validated by measuring its own resolution before anything is measured with it. All figures in this chapter are [SIMULATED] unless stated otherwise; the hardware campaign is reported in Chapter 8.

4.1 The elementary ring oscillator

The entropy source is an Elementary Ring Oscillator (ERO): a free-running ring oscillator RO1 whose output is sampled by a flip-flop clocked by a second ring oscillator RO2 divided by an integer K_D [10, 13]. Between two samples the phase of RO1 accumulates jitter for K_D periods of RO2; when the accumulated jitter is comparable to the period of RO1, the sampled bit becomes unpredictable. The ERO was chosen over higher-throughput alternatives (multi-ring XOR, coherent sampling, transition-effect rings) for one reason: it is the architecture for which a stochastic model with a provable entropy bound exists and for which an embedded jitter measurement method has been published and validated [8]. Throughput is not a requirement here — the engine consumes entropy only to seed a deterministic generator — whereas a demonstrable bound is.

A design decision that follows from the platform is that *the board oscillator is never used as a reference.* The Basys 3 clock source is a MEMS oscillator with a fractional Phase-Locked Loop (PLL) whose datasheet specifies a cycle-to-cycle jitter of up to 95 ps [ESTIMATED] [123], one to two orders of magnitude above the per-period jitter of a ring

oscillator in a 28 nm process (2–4 ps [ESTIMATED] [8]). Sampling RO1 with that clock would measure the phase noise of the PLL, a global and potentially inducible component, rather than the thermal noise of the ring. RO2 therefore samples RO1, and the 100 MHz system clock only drives the synchronous logic.

4.2 Stochastic model and the quality factor

Following Baudet *et al.* [10], the phase $\varphi(t)$ of RO1, measured in periods, is modeled as a Wiener process with drift μ and volatility σ^2. Between two samples separated by Δt the phase advances $\nu = \mu\,\Delta t$ periods on average and accumulates a variance

$$Q = \sigma^2\,\Delta t, \tag{4.1}$$

the *quality factor*. The output bit is $s = 1$ when the fractional phase lies in $[1/2, 1)$. Conditioned on the previous phase φ_0, the bit is Bernoulli with

$$P(s = 1 \mid \varphi_0) = \sum_{k \in \mathbb{Z}} \left[\Phi\!\left(\frac{k + 1 - \varphi_0 - \nu}{\sqrt{Q}} \right) - \Phi\!\left(\frac{k + \frac{1}{2} - \varphi_0 - \nu}{\sqrt{Q}} \right) \right], \tag{4.2}$$

where Φ is the standard normal distribution function; this is the wrapped Gaussian evaluated exactly. Its first harmonic gives the maximum bias $\varepsilon_{\max} \approx (2/\pi)\, e^{-2\pi^2 Q}$, from which Baudet's Corollary 1 yields the well-known expression

$$H(s \mid \varphi_0) = 1 - \frac{4}{\pi^2 \ln 2}\, e^{-4\pi^2 Q} + O\!\left(e^{-6\pi^2 Q}\right). \tag{4.3}$$

4.3 Exact conditional entropy vs. the asymptotic bound

Equation (4.3) is routinely cited as a *lower bound* on entropy per bit. Read carefully, the inequality in the corollary is only that the entropy rate of the sequence is at least the entropy of one bit conditioned on the exact previous phase; the right-hand side is an asymptotic expansion *of* that conditional entropy, not a bound on it. Expanding the binary entropy in the bias, $h_2(\tfrac{1}{2} + \varepsilon) = 1 - \frac{1}{\ln 2} \sum_{n \geq 1} \frac{(2\varepsilon)^{2n}}{2n(2n-1)}$, and averaging over a uniform φ_0 with $\varepsilon = A \sin(2\pi\varphi)$, $A = (2/\pi) e^{-2\pi^2 Q}$, gives

$$H(s \mid \varphi_0) = 1 - \frac{4}{\pi^2 \ln 2}\, e^{-4\pi^2 Q} - \frac{8}{\pi^4 \ln 2}\, e^{-8\pi^2 Q} - \cdots \tag{4.4}$$

All omitted terms are negative, so *any* truncation overestimates the entropy; the next term decays as $e^{-8\pi^2 Q}$, not $e^{-6\pi^2 Q}$. Table 4.1 compares the exact value computed from (4.2) on a grid of 8192 phases with the first- and second-order expansions. Adding the second-order term reduces the error of (4.3) by a factor of 95 at $Q = 0.10$ and 4970 at

$Q = 0.20$, which confirms the derivation.

Table 4.1: Conditional Shannon entropy per bit: exact wrapped-Gaussian computation vs. truncated expansions [SIMULATED].

Q	Exact	Order 2	Order 1 (Eq. 4.3)	Error of order 1
0.02	0.7013194	0.7100952	0.7345213	$+3.3 \times 10^{-2}$
0.05	0.9163001	0.9164920	0.9187783	$+2.5 \times 10^{-3}$
0.10	0.9886728	0.9886733	0.9887174	$+4.5 \times 10^{-5}$
0.15	0.9984319	0.9984319	0.9984327	$+8.5 \times 10^{-7}$
0.20	0.9997823	0.9997823	0.9997823	$+1.6 \times 10^{-8}$

In the operating region ($Q \approx 0.2$) the discrepancy is 10^{-8} and irrelevant; for an under-dimensioned generator ($Q \approx 0.02$) the truncated formula credits 3.3 % of entropy that does not exist. This thesis therefore uses the exact computation as the conservative value and cites (4.3) as what it is: an excellent asymptotic approximation for $Q \gtrsim 0.1$. The same numerical machinery shows that the min-entropy obtained from the first-harmonic bias, $H_\infty = -\log_2(\frac{1}{2}+\varepsilon_{\max})$, agrees with the exact worst-case value to better than 10^{-4} bit over $Q \in [0.05, 0.30]$.

4.3.1 Design target

AIS 20/31 version 3.0 [5] admits two alternative criteria for class PTG.2: Shannon entropy ≥ 0.9998 or min-entropy ≥ 0.98 per internal bit. Solving both by bisection on the exact entropies gives $Q \geq 0.2022$ and $Q \geq 0.2286$ respectively. The min-entropy criterion is the more demanding one and is also the quantity NIST SP 800-90B works with, so it is adopted here: **the design target is $Q \geq 0.23$, with a set point of $Q = 0.30$** ($H_\infty = 0.9951$), the margin covering the reduction of thermal jitter at low temperature, which is the standard adversarial test of Fischer and Lubicz [13].

4.4 Embedded jitter estimator and its two corrections

To choose K_D one must know σ^2, and the method of Fischer and Lubicz [13] measures it from the bits themselves: with $K_D = 1$, the fraction c of pairs (b_j, b_{j+M}) that differ within a block of N pairs estimates the phase accumulated over M samples, and its variance across K blocks grows as $4MQ_{\text{raw}}$. Implemented literally, with a straight-line fit, the estimator *underestimates* Q_{raw} by 13 % to 38 % depending on the jitter level. Two causes were identified on synthetic data with known Q and corrected:

1. **Vertex effect.** c is not linear in the accumulated phase; it is a *triangular* wave of period one and slope ± 2. When the deterministic phase $M\nu \bmod 1$ falls near

a vertex the distribution straddles it and the variance collapses — to 0.38 of its nominal value in one measured case. The vertex is detected from the measured mean itself ($\bar{c} \approx 0$ or 1 is a vertex, $\bar{c} \approx \frac{1}{2}$ the middle of the ramp), so distances M with $\bar{c}$ outside $[0.15, 0.85]$, or for which $3\sqrt{MQ}$ reaches the vertex, are discarded iteratively.

2. **Non-constant estimation noise.** Estimating a fraction from N pairs adds a variance that scales with $\bar{c}(1-\bar{c})$, which changes with M. Fitting it as a constant intercept biases the slope; it is fitted instead as a second regressor, $\mathrm{Var}(c) = Q\,(4M) + \kappa\,\bar{c}(1-\bar{c})/N$.

After both corrections the estimator recovers the injected Q_{raw} within $\pm 2\,\%$ over a range of $150\times$ (Table 4.2), with a bias of $-0.6\,\%$ and a dispersion of $4.0\,\%$ over eight independent seeds for $K = 20\,000$ blocks. The fitted accumulation exponent is 1.009 for a simulator without flicker noise; measured on hardware, the same exponent separates the thermal component ($b = 1$, which counts as entropy) from flicker and global jitter ($b = 2$, which does not).

Table 4.2: Recovery of a known Q_{raw} by the corrected embedded estimator ($K = 20\,000$ blocks of $N = 100$ pairs) [SIMULATED].

Q_{raw} injected	Estimated	Error	M values used
2.0×10^{-7}	1.979×10^{-7}	$-1.07\,\%$	19
5.0×10^{-7}	5.060×10^{-7}	$+1.21\,\%$	18
2.0×10^{-6}	2.037×10^{-6}	$+1.86\,\%$	16
8.0×10^{-6}	8.039×10^{-6}	$+0.48\,\%$	9
3.0×10^{-5}	2.977×10^{-5}	$-0.78\,\%$	5

A practical consequence for the hardware is that the blocks need not be contiguous: each requires only $N + M$ consecutive bits, and independent blocks give a slightly better estimate ($-0.30\,\%$ error with $20\,000$ independent chunks). The capture buffer in the FPGA therefore holds a single chunk of a few kilobits, filled at ring speed and read out slowly, instead of megabits of contiguous samples.

4.5 Transistor-level bench and its resolution

The entropy source is the only block of the engine tied to the technology, and in an integrated circuit it is designed at transistor level. A SPICE bench was built around a five-stage ring with an enabling NAND gate, with every technology-dependent parameter (device models, widths, minimum length, supply) confined to a header so that moving from the generic placeholder models used here to a foundry Process Design Kit (PDK) changes one block of the netlist. Jitter is extracted from the threshold-crossing instants

of the output by the second difference $D(M) = t_{k+2M} - 2t_{k+M} + t_k$, i.e. the Allan variance applied to phase data, whose expectation $2M\sigma^2$ is free of the bias that a simple difference over overlapping windows carries (measured at 5 %), and which is the method used by Benea *et al.* to characterize rings both in Artix-7 and in 28 nm silicon [32]. Estimates at different M are combined with inverse-variance weights proportional to the number of independent windows; without that weighting the dispersion stayed at 15 % regardless of the simulated length.

The bench measures its own resolution first: with all noise sources disabled, the residual jitter is the numerical noise of the simulator. The first attempt gave 0.33 ps, of the same order as the jitter to be measured; it was entirely the start-up transient of the oscillator, and discarding the first twenty cycles brings the floor to **1.38 fs**, three orders of magnitude below the signal. Only then was noise injected. Jitter scales with the square root of the injected spectral density ($\times 2.01$ measured for $\times 4$ in density, against $\times 2.00$ expected) up to the point where the r.m.s. noise current reaches about 80 % of the stage drive current; there the small-signal regime breaks down, the mean period shifts, and the scaling law fails by a factor of five. A validity criterion follows for every campaign, including those on a real PDK: keep injected noise below ~ 40 % of the stage current and verify that the mean period does not move when noise is enabled. With the placeholder models the ring oscillates at 4.3 GHz, which is not physical; the numbers that enter the thesis come from real device models.

The first such models are those of the open IHP SG13G2 PDK (130 nm, 1.2 V core, PSP 103.6 compact model, Apache-2.0 licence) [115], which can be published along with everything else. Two things differ from the placeholder bench. First, the models are Verilog-A and must be compiled to the OSDI interface; this required building ngspice 47 and the OpenVAF-Reloaded compiler in user space, since no package channel offered versions recent enough. Second, and more important, the noise is no longer chosen: ngspice does not inject device noise during a transient, so the density of the current sources is taken from a small-signal noise analysis of the same transistors, an inverter biased at its switching point with input tied to output, where both devices conduct and noise moves the crossing instant. The input-referred noise of that node is exactly the current the transient source must reproduce. Its spectrum is a pure $1/f$ term plus a white floor of 4.13×10^{-24} A^2/Hz, flat from 100 kHz to 10 GHz, with the $1/f$ corner at 19.8 MHz and an effective noise factor $\gamma = 1.04$, above the long-channel 2/3 as expected for a short device. The $1/f$ part of ngspice's noise source was calibrated against a known resistor before use: its density is NAMP$^2/(\pi f)$ to 0.4 %. Results are in Section 8.3.

4.6 Design target and divider selection

With σ/T measured per period, $Q_{\mathrm{raw}} = (\sigma/T)^2$ and the divider follows from $K_D = \lceil Q_{\mathrm{target}}/Q_{\mathrm{raw}} \rceil$. For the range of jitter expected in a 28 nm process the divider lies between 3×10^4 and 5×10^5 and the throughput per ring between 1 and 20 kbit/s. The model reproduces the published design point of Fischer and Lubicz ($K_D \approx 430\,000$ for $\sigma = 5.01$ ps over $T = 7.81$ ns) [13]. Such rates are sufficient here: seeding the deterministic generator with 256 bits of full entropy requires about 512 raw bits at $H_\infty \approx 0.98$, i.e. between 0.03 and 0.5 s per seed, and the engine issues a seed only at start-up and at reseed intervals. If more throughput were ever needed, the clean route is several independent EROs, each with its own bound, rather than a multi-ring XOR whose independence cannot be demonstrated [16].

Chapter 5

Digital Engine

The digital engine turns raw bits from the entropy source into keys and uses them, and it is written so that the same Register-Transfer Level (RTL) runs on the FPGA used for verification and, later, on standard cells. Nothing in it is tied to a vendor: no primitives, no memory macros, no clock-management blocks. The only technology-dependent block of the whole design is the pair of ring oscillators of Chapter 4.

5.1 Architecture and the single-primitive decision

Figure 5.1 summarizes the data path. Raw bits leave the ERO in the clock domain of the sampling ring, cross to the system clock one bit at a time, and feed three consumers: the health tests, the seed collector of the deterministic generator, and a capture buffer used only for characterization. Keys come out of a Counter-mode Deterministic Random Bit Generator (CTR_DRBG); the current key is loaded into an AES-CMAC block that answers challenges; and an Inter-Integrated Circuit (I2C) slave exposes everything as a register map.

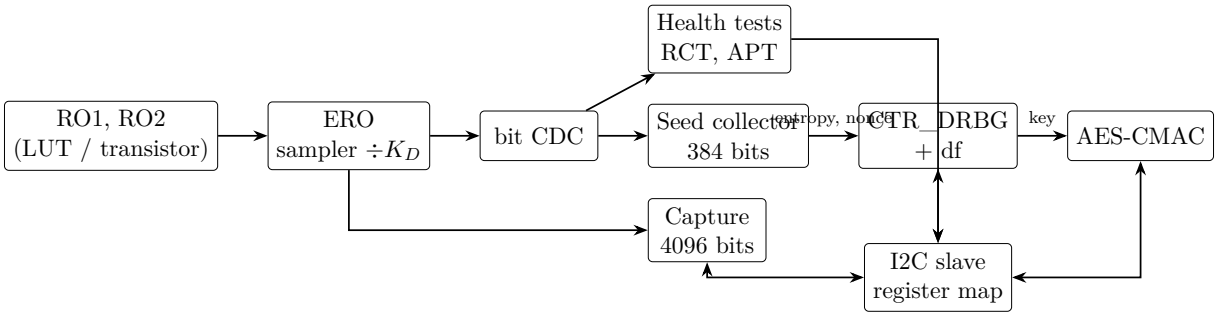


Figure 5.1: Data path of the engine. Only the ring oscillators are technology dependent; every other block is portable RTL.

The decision that shapes the whole engine is to use **one cryptographic primitive, AES-128, for the three jobs**: conditioning the entropy, generating keys and authenticating. The first architecture had a SHA-256 core as conditioner; the synthesis estimates

of Chapter 7 showed that it would take almost half the digital area, and SP 800-90B lists AES-based constructions — CMAC, CBC-MAC and the block-cipher derivation function of SP 800-90A — among its *vetted* conditioning components [6]. Dropping the hash brings the engine from an estimated ∼25 to ∼13 Gate Equivalents (GEs) (thousands), at the price of moving from a CTR_DRBG *without* derivation function, which needs full-entropy input, to one *with* it, which is the common configuration in practice and accepts non-full-entropy input directly. The AES core is encryption-only: counter mode and CMAC never decrypt, and offering decryption as a service to the ESP32 would be pointless since the ESP32 has its own accelerator. Cross-verification is unaffected: the engine encrypts and the ESP32 decrypts with mbedtls.

5.2 AES-128 encryption core

The core is iterative, one round per clock cycle, eleven cycles per block (initial key addition plus ten rounds), with the round keys expanded on the fly from the previous one — forward expansion is all that encryption needs, and it saves storing eleven round keys. A second core with the same interface and a 32-bit datapath (four S-boxes shared with the key schedule, one column per cycle, 51 cycles per block) was written later for the integrated-circuit chapter and is the one the engine now instantiates by default; the one-round core remains the reference the bench compares it against (Chapter 7). At 100 MHz that is 1.16 Gbit/s of core throughput, far above what any I2C link can carry, so pipelining would be wasted area; the iterative form is chosen instead because one round is a pure combinational function that can be checked directly against the standard [78]. The S-box is a 256-entry table synthesized as logic, MixColumns is written as the usual `xtime` network, and the state follows the column-major convention of FIPS 197 with byte 0 on the left, so that hexadecimal strings in the standard map one-to-one onto the VHDL vectors.

The core was verified against the five official vectors available for AES-128 encryption (FIPS 197 appendices B and C.1, and the three ECB blocks of SP 800-38A F.1.1) and against a chain of one thousand encryptions to expose any state left over between blocks. A check that had not been planned turned out to be the most convincing: the encryption of the all-zero block under the SP 800-38A example key gave `7df76b0c...546f`, which is exactly the intermediate value L published by NIST in its CMAC examples [124].

5.3 AES-CMAC

AES-CMAC [92, 93] is the primitive of the challenge-response: the ESP32 sends a nonce, the engine returns its tag under the current key. The block computes the sub-keys K_1, K_2 once when a key is loaded (one encryption of the zero block and two conditional left

shifts with the polynomial constant `0x87`) and then runs CBC-MAC over the message, applying K_1 to a complete last block or padding with `0x80` and applying K_2 otherwise. It was chosen over HMAC-SHA-256 because it reuses the only primitive and costs one encryption per block against two hash compressions. The block passes the four vectors of RFC 4493 (empty message, one block, 40 and 64 bytes), which exercise both padding paths.

5.4 CTR_DRBG with derivation function

The generator implements CTR_DRBG with AES-128 and the block-cipher derivation function of SP 800-90A [89]. Its input format is fixed on purpose: 256 bits of raw entropy and 128 bits of nonce, so that $S = L \,\|\, N \,\|\, E \,\|\, \mathit{nonce} \,\|\, \texttt{0x80} \,\|\, 0\ldots$ is exactly four blocks and the derivation function becomes a closed sequence of fourteen encryptions; a variable-length input would cost a byte-oriented datapath for no benefit. Instantiation (or reseed) costs 14 AES operations, 183 cycles; generating 256 bits costs four, 53 cycles. The state (Key, V) is never exposed; the reseed counter is readable so that the host can enforce a reseed policy far below the 2^{48} limit of the standard.

Verification proceeded in two steps. First, a bit-exact reference model was written in Python — with its own AES, independent from the VHDL — and run against the CAVP vectors: 960 AES-128 cases with and without derivation function, with and without reseed, all passing. Second, that model generated the expected outputs for the fixed input layout of the hardware, and the VHDL reproduced them: two consecutive generations, a reseed followed by a generation, the reseed counter, and the fact that a new instantiation erases the previous state.

5.5 Health tests

The two tests approved by SP 800-90B run continuously on the raw bits [6]. The Repetition Count Test fails when $C_{\mathrm{RCT}} = 1 + \lceil -\log_2 \alpha / H \rceil$ identical samples arrive in a row; the Adaptive Proportion Test counts, in a window of $W = 1024$ samples (the binary-source value; 512 is for non-binary sources), how many repeat the first one, and fails at $C_{\mathrm{APT}} = 1 + \mathrm{CRITBINOM}(W, 2^{-H}, 1-\alpha)$. For the design target $H = 0.98$ and $\alpha = 2^{-20}$ these are $C_{\mathrm{RCT}} = 22$ and $C_{\mathrm{APT}} = 596$; both are generics, because H is set from the *measured* min-entropy, not assumed. The proportion test also checks the complement of the count, which the standard permits for binary sources: a source stuck at the value opposite to the window's first sample would otherwise score zero and pass. Alarms are sticky until explicitly cleared, and while any alarm is raised nothing leaves the engine: the sequencer refuses to seed, to generate a key and to produce random output. Two further rules close the gap between a continuous test and a 384-bit seed. A start-up test (SP 800-90B §4.3)

must pass first: one complete proportion window, 1024 samples, closed without alarm after reset or after the alarms are cleared, and clearing them restarts the window. And a seed is accepted only once the window that holds its last bit has closed clean; the seed is shorter than the window, so otherwise it would enter the generator before the proportion test had judged it.

Four more rules came out of reviewing the sequencer against the register map rather than against the standard. The raw-bit capture buffer records the same bits the seed is collected from, so a capture started during a seeding left the seed readable in test mode; capture is now a test-mode command that is refused while the engine is busy, and seeding is refused while a capture is in progress. The request counter of the Deterministic Random Bit Generator (DRBG) no longer wraps: at its 32-bit ceiling the engine refuses to generate until it is reseeded and says so in a diagnostic bit, which is the hardware end of the reseed policy the host is still expected to run well before that. A zeroization command clears the key, the generator state and counter, the challenge, the random register and the collected seed, and loads the null key into the authenticator; it is honoured even under alarm, and the engine must be seeded again afterwards. And the sampling divider resets to its maximum rather than to a small value, so a host that forgets to configure it seeds at the slowest rate and the most entropy per bit, never below the characterized setting.

The testbench injects the failure modes the tests exist for: a healthy source passes twenty windows (worst run 13, worst proportion 540); a stuck source trips the repetition test at exactly sample 22; a source biased to 70 % trips the proportion test at 713 without tripping the repetition test; and a source stuck at the complement is caught. The first "healthy" source of the bench — a shift register seeded with a single one — was itself caught by the tests, with a start-up run of 27 zeros and a first window at 76 %. The detector was right and the bench was wrong; the source now starts from a dense seed and is warmed up.

5.6 Capture path and clock-domain crossings

Raw bits are produced in the clock domain of RO2 and consumed in the system domain. The bit-serial path to the health tests and the seed collector uses a toggle synchronizer: the source registers the bit and inverts a flag; the destination synchronizes the flag through three flip-flops and reads the bit on the flag's edge. It requires each bit to last at least eight system cycles, i.e. $K_D \geq 64$ for a ∼700 MHz sampling ring at 100 MHz, which the generator mode satisfies by orders of magnitude. The characterization path does not cross bit by bit: the capture buffer is written in the ring domain at full rate into a 4096-bit memory and read out in the system domain afterwards, with the two directions of the handshake carried by toggle flags. Writing and reading never overlap, so no asynchronous FIFO is needed, and the buffer size follows from Section 4.4: one chunk of $N + M_{\max}$ bits

is all the estimator needs.

5.7 I2C register interface and command sequencer

The product interface is I2C, as in commercial secure elements: two pins, addressable, and the bus the ESP32 uses for its peripherals. The slave implements the register-pointer protocol with auto-increment, a five-sample glitch filter on both lines as the specification requires, and open-drain outputs. It does not stretch the clock: at 400 kHz there are 250 system cycles per bit, plenty for any register access, and long operations (seeding, generation) are issued as commands and polled through a status register, which is how commercial devices behave. The slave was tested with a bus-functional master; two real defects surfaced — the pointer did not auto-increment on multi-byte writes, and the most significant bit of each read byte was driven one clock edge late, so every read would have been shifted — and were fixed.

The register map (Appendix A) exposes identification, status and control, the divider and capture-page configuration, the health-test diagnostics, the reseed counter, the challenge and its tag, a 256-bit random-output register and, *only while a test pin is asserted*, the current key and the capture buffer. The random-output register is filled by its own command with a separate generation of the DRBG, never by the one that produced the key: an earlier version of the design left the 256 bits of the key-generating call in that register, readable without the test pin, and thus leaked the key. The review that caught it is described in Chapter 6. In the FPGA the test pin is a switch; in an integrated circuit it would be a fuse blown at production, because exposing either the key or the raw entropy is a vulnerability. A small sequencer serializes the commands: seeding collects 384 raw bits, waits until the health tests have evaluated all of them, aborts if a health alarm rises meanwhile, and instantiates or reseeds the generator; generation takes 256 bits, keeps the first 128 as the key, discards the rest and reloads the CMAC sub-keys; a separate command fills the random-output register from an independent generation; authentication runs the CMAC over the challenge block.

The whole engine was exercised as the ESP32 will exercise it: a bus-functional I2C master reads the identification, configures the divider, seeds, generates, reads the key in test mode, writes two challenges and reads their tags, generates again, captures raw bits and verifies that the key is unreadable without the test pin — sixteen checks. What a VHDL bench cannot know is whether the tags are *correct*, because the key came from ring-oscillator noise; the bench dumps key, challenges and tags to a file and a Python script recomputes the CMACs with the independent AES. They match. That is the same cross-check the ESP32 performs on hardware with mbedtls.

Chapter 6

Verification

6.1 Strategy: vectors, simulation, independent verifier

A cryptographic engine that is verified only against itself proves nothing: an error in the S-box would be reproduced identically in the model used to check it. The verification of this work therefore rests on three legs that do not share an implementation.

1. **Official vectors.** Every primitive is checked against the test vectors published with its standard: FIPS 197 and SP 800-38A for AES, RFC 4493 and the NIST example file for CMAC, and the CAVP response files for CTR_DRBG (with and without derivation function, with and without reseed). These are known-answer tests and they settle correctness of the algorithm.

2. **Independent reference models.** A bit-exact model of the generator was written in Python with its own AES, derived from the standard without looking at the VHDL. It is validated against the CAVP vectors first, and only then used to generate expected values for the fixed input layout of the hardware and to recompute the tags of the system-level bench. Two implementations that agree are worth more than one that passes.

3. **An independent verifier in the loop.** On hardware the ESP32 recomputes with mbedtls, and with its own AES accelerator, every result the engine produces, so that the final check is performed by a third, unrelated implementation over a real bus.

Two working rules apply throughout, both learned in the black-box study of the ESP32 that opens this work: the output of a compilation, simulation or measurement is never filtered before being read in full — a filter hides exactly the warning that mattered — and

failures are recorded whole, never discarded. Twice in this work a filter on the simulator's log hid a result, and both times the fix was to remove the filter.

6.2 Reference models in Python

`ctr_drbg_ref.py` implements AES-128 encryption, the block-cipher derivation function, CBC-MAC and CTR_DRBG with and without derivation function, in pure Python and from the text of SP 800-90A. The AES asserts the two FIPS 197 vectors at import; the DRBG is exercised against the CAVP response files following the CAVS procedure (instantiate, optional reseed, two generate calls, compare the second): **960 AES-128 cases, all passing**. One defect was found in the file parser rather than in the model — a section header must close the pending test case before the parameters change, otherwise the last case of every parameter block is evaluated with the next block's parameters — and it is worth recording because its symptom, a single failure at count 14 of each block, looked exactly like an algorithmic error.

A second script, `gen_drbg_vectores.py`, drives the validated model through the sequence the hardware supports (instantiate with 32 bytes of entropy and 16 of nonce, generate twice, reseed, generate) and emits a VHDL package of expected values; if the input layout of the hardware ever changes, that script is the only place to change it. A third, `verifica_top.py`, recomputes AES-CMAC from the same AES and validates itself against RFC 4493 before checking anything.

6.3 Testbenches and results

All VHDL is simulated with GHDL 6 under a single runner that analyses the sources in dependency order and executes every bench, printing the simulator's output unfiltered. Table 6.1 summarizes the seven benches and their state at the time of writing.

Some of what the benches found deserves to be stated, because a verification chapter that reports only passes has not verified much.

Ring model. The LUT primitives used to simulate the ring were first modeled with a strict unknown-propagation rule: any unknown input gave an unknown output. At the start of a simulation every signal is unknown, so the ring never left that state and never started. A real NAND with its enable input at zero outputs one regardless of the other input; the model now evaluates every combination compatible with the known inputs and resolves the output when they all agree. Without this the entire system-level bench would have been impossible.

Table 6.1: Testbenches, what they check and their result (GHDL 6.0, VHDL-2008).

Bench	Checks	Result
Ring oscillator	start/stop, $f = 1/(2Nt_{pd})$, zero floor, $\sigma_T = \sqrt{2N}\,\sigma_{\text{stage}}$	6/6
I2C slave	addressing, foreign address, multi-byte write, repeated-start read; bus under abuse: sub-filter glitches, stop and repeated start mid-byte, bytes after a foreign address, pointer across stop and wrap, master lost with SDA held, stop after read-ACK, 1 MHz and 100 kHz; 1 property	49/49
AES-128 cores (both)	FIPS 197 B, C.1; SP 800-38A F.1.1 (3 blocks); 1000-block chain; zeroization mid-block; one-round-per-cycle and 32-bit cores agree	18/18
AES-CMAC	RFC 4493 examples 1–4; repeatability	5/5
CTR_DRBG	instantiate, 2×generate, reseed, counter, re-instantiate, refuse unseeded, zeroize at rest and mid-operation, saturating request counter	18/18
Health tests	healthy source, stuck, 70 % biased, stuck-at-complement, sticky/clear	15/15
Whole engine	ESP32 sequence over I2C: id, seed (divider below minimum refused, divider frozen while busy), key, random register free of the key, 2 challenges, 2nd key, capture (seed never in the buffer), key hidden, exhaustion, zeroization and residues; 11 properties, 6 covers	64/64

I2C slave. Two genuine design errors: the register pointer did not advance on multi-byte writes, so consecutive bytes overwrote the same address; and on reads the most significant bit was driven one clock edge too late, shifting every byte. The first was fixed by incrementing the pointer one state after the write strobe, so the address is still valid during the strobe; the second by loading the shift register and driving its first bit on the same falling edge that releases the acknowledge. A third defect was in the bench itself: initializing the register memory from the stimulus process created a second driver on the same signal and every conflicting bit resolved to unknown. The slave is the only door into the chip, so the bench was later extended with the bus under abuse: spurious pulses shorter than the input filter in the middle of a byte, on SDA with SCL high (a start followed by a stop, if not filtered) and on SCL in its low phase; a stop and a repeated start in the middle of a byte; bytes sent after a foreign address; the pointer surviving a stop and wrapping at the end of the map; a master that disappears with the clock stopped while the slave is holding SDA low in its acknowledge; a master that issues a stop after acknowledging a read byte, which I2C itself makes impossible when the next byte starts with a zero because the slave is holding the line; and the bus at 1 MHz and 100 kHz with a 20 MHz system clock. The bench counts the write strobes to the register bank, so no aborted transaction may write, and a property checked at every clock states that the slave never drives SDA while idle. The bus-hang guard releases the line in the configured

number of cycles in both stuck cases, and nothing abnormal writes.

Health tests. The "healthy" source of the first bench, a shift register seeded with a single one, started with a run of 27 zeros and a first window at 76 %; the detector fired. The detector was right.

Key leak through the random register. The most serious finding did not come from a bench but from reading the design again before handing it over. The command that generated the key wrote the full 256-bit output of the DRBG into the random-output register, which is readable without the test pin; the first 128 of those bits *were* the key. The system bench even asserted that equality as a "check". The fix separates the two: key generation leaves nothing readable, and a new command fills the random register from an independent generation, after the DRBG has updated its state. The bench now asserts the opposite, that the register contains the key in neither half, and the Python cross-check does the same.

Whole engine. With the rings at their physical speed the system bench, which drives almost a millisecond of I2C traffic, did not finish in twenty minutes of CPU: every ring transition costs two random draws in the jitter model. In that bench alone the stage delay is set ten times larger, with the same relative jitter, and this is stated in the bench. The eighteen checks pass, and the key, the two challenges and their tags are written to a file that `verifica_top.py` checks: both tags equal the CMAC recomputed by the independent Python AES, the readable random-output register contains the key in neither of its halves, and two generations give different keys. This is the first point at which a key born from ring-oscillator noise, conditioned, generated and used to authenticate, is confirmed by an implementation that shares nothing with the VHDL. Two of the eighteen checks were added when two obligations that the register map had placed on the host were moved into the hardware: a seed command issued with the divider below its minimum is now refused, and a write to the divider register while a command is running is ignored. A rule that only the documentation enforces is a rule that a careless host breaks; the bench now exercises both. The security rules of the sequencer are also written as PSL properties inside the bench, checked on every clock of the whole simulation rather than only at the steps that provoke them: no generation and no seeding while an alarm is raised, no seed collection before the start-up test, no generation with the request counter exhausted, capture and seeding mutually exclusive and capture only in test mode, no authentication without a valid key, zeroization clearing the key in the same cycle, and the engine never staying busy forever; four cover directives confirm that the bench reaches the states those properties guard. A mutant of the sequencer that drops the alarm check on key generation is caught both by the step that expects the refusal and by the property. The bench also

had to be told to stop: with the rings running after reset, a simulation whose stimulus has finished never runs out of events, and the first re-run consumed forty-five minutes of CPU after its last check had passed.

6.4 ESP32 verifier firmware

The firmware is an ESP-IDF 5 application for the classic ESP32 that talks to the engine through the register map of Appendix A and offers a console over the UART. Its commands map onto the verification plan: `id` reads identification and status; `sembrar` and `generar` issue the long commands and poll the busy flag; in test mode `generar` also reads back the key (provisioning) and checks that it equals the first sixteen bytes of the generated block; `reto N` performs N challenge-response rounds with nonces from `esp_random()`, recomputing each tag with `mbedtls_cipher_cmac` and counting matches and mismatches, the mismatches being printed in full; `crudo N` dumps N chunks of raw bits and `esp N` dumps blocks of the ESP32's own generator, both in the framed, CRC32-protected format of the ESP32 capture campaigns (Section 3.6), so that the same statistical pipeline processes the two sources. CMAC is not enabled by default in the ESP-IDF build of mbedtls and is switched on in the project defaults; the console runs at 115 200 baud, the only rate that proved reliable with the board's USB bridge in those campaigns, and the log level is set to errors only so that log lines cannot corrupt the frames. The register access layer (map, command issue, real-time polling, rejection check) is a component shared with the application of Section 6.5. The firmware builds with ESP-IDF 5.3.5 into a 231 kB image; running it awaits the hardware.

Both firmwares are also exercised on the host, without a board: the bench includes the firmware source unchanged, links it against a software model of the engine that implements the register map as a second, independent reading of the specification, and runs the sequence with the same mbedtls 3.6.7 tree that ESP-IDF ships. The model's AES and CMAC validate themselves against FIPS 197 and RFC 4493 before any check, so that a model fault cannot pass for a firmware fault. The bench checks counts, not just outcomes: that a rejected command is not mistaken for a quick one, that a bus failure is counted as a bus failure and not as a cryptographic mismatch, that nothing reaches the PC while an alarm is raised, and that the timeouts last what they claim.

6.5 The engine in a product: signed readings

The verifier judges the engine; a second firmware *uses* it, as a product would. The product is a generic energy meter on an ESP32-C3, the RISC-V part of the small Wi-Fi devices, with the engine on the same I2C bus as the meter's own peripherals. Two uses, both fed by the engine's entropy. First, the device generates its own ECDSA P-256 key pair:

mbedtls draws the private scalar through a callback that issues one `aleatorio` command per 32-byte block, two blocks per key, and the private key never leaves the chip; what leaves is the public key, or a PKCS#10 certificate request signed with it, which is what a device would send to the manufacturer's CA so that nobody, the manufacturer included, ever holds the private key. Second, every reading (device identifier, sequence number, time, accumulated energy, power; 24 bytes) is signed, so that whoever receives it checks integrity and origin without trusting the transport or the offline store in between.

Three decisions carry the design. The key's strength is bounded by what generated it: the engine's DRBG is CTR_DRBG with AES-128, 128 bits of security, which is what P-256 offers, so a larger curve would gain nothing from this seed. Signatures use deterministic ECDSA (RFC 6979): the per-signature nonce is derived from the key and the message, two signatures of the same reading are identical (the bench checks it), and a poor generator at signing time cannot leak the key, which is the classic ECDSA failure; the engine is therefore needed only to *create* the key, and a meter whose engine is in alarm keeps signing with the key it has, while it cannot make a new one. And the entropy callback fails closed: if the engine rejects the command (unseeded, alarm, start-up test not passed) or the bus fails, mbedtls aborts and no other source is substituted; a failed generation leaves the previous key in service and never a half-made one. A monotonic sequence number lets the receiver reject replayed readings, which a valid signature alone does not.

The host bench runs the meter firmware against the engine model and hands its console output to a PC script that verifies the frames with a different library (`cryptography`, not mbedtls). The bench plants one tampered reading (one bit of the energy field) and one replayed reading among eight valid ones, and the script must find exactly those two; the certificate request must parse, carry a valid self-signature and the device's public key. All checks pass. The firmware builds with ESP-IDF 5.3.5 for the ESP32-C3, and it also runs on Espressif's QEMU emulation of that chip: without an engine every command fails closed, and with the engine model linked into the firmware in place of the I2C driver (a build option, never for a board) the whole sequence runs on the emulated RISC-V core with the target build of mbedtls, and the PC script accepts its frames; the public key the emulated chip produces equals the one the host bench produces from the same engine blocks.

The same build was then flashed onto a real ESP32-C3 module over USB-serial and driven by a script that logs every line and times every command. On the silicon, seeding the model takes tens of milliseconds, generating the P-256 key and storing it takes about 0.17 s, a certificate request 0.2 s and one signed reading about 0.2 s including the serial round trip; the private key survives physical resets (the enable pin pulsed from the serial adapter), the boot counter kept in flash advances on each, the public key read back after a reset is unchanged and the first reading of the next boot exceeds the last one of the

previous boot in sequence, so the anti-replay rule holds across real power cycles. Two things the emulator could not show appeared at once. The console read its commands with a buffered-line call over a non-blocking input, and over a real USB-serial link the bytes of a command arrive with gaps: a single command reached the firmware as four fragments, each rejected as unknown; the console now accumulates characters up to the line end. And the new key generated after forgetting the old one was identical to it, because the engine model draws its "randomness" from a seedless pseudo-random function that repeats the same sequence at every boot; this is not a defect of the meter but the live demonstration of what the engine is for. The PC verifier accepts every frame of the board log. What remains unverified is the engine itself on a board: it exists only in simulation until the FPGA bring-up, and the model's entropy is an AES with a fixed seed, not a measurement.

6.6 FPGA bring-up

Pending hardware. The constraints the rings need are known from the literature and from the vendor documentation — `DONT_TOUCH` on the cells and nets of each ring (unlike `KEEP`, it survives implementation), `ALLOW_COMBINATORIAL_LOOPS` on one net of each loop, `set_disable_timing` to break the loop for static timing analysis, and one placement block per ring so that identical rings are neither merged nor placed side by side — and the first bitstream has a single purpose: to read the frequency counters and confirm that every ring oscillates. Nothing about entropy is claimed before that.

Chapter 7

Path to an Integrated Circuit

The engine is designed to end up as a dedicated integrated circuit that serves keys to a microcontroller, and the FPGA is its verification vehicle, not its destination. This chapter takes the same VHDL that passes the testbenches of Chapter 6, synthesizes it on an open standard-cell library, and states what a fabrication run would cost and what would have to change in the entropy source. A tape-out is explicitly out of scope of this thesis — not for cost or area, as will be seen, but for schedule — and everything here is labelled as what it is: a lower bound on area from logic synthesis, with the growth factors to core area taken from published data.

7.1 Standard-cell synthesis flow

The flow is deliberately reproducible with free tools. Yosys reads the VHDL through the GHDL front end, synthesizes each block flattened, maps flip-flops and logic onto the liberty file of the target library and reports cell count and area. The library is the open PDK of IHP, SG13G2, a 130 nm BiCMOS process with a maintained standard-cell library, compiled SRAM macros, I/O cells and OpenROAD support, chosen because it is the one open PDK that is alive, European and reachable through the same Europractice membership the university already holds. The gate equivalent is read from the liberty file itself rather than from memory: 1 GE = area of `sg13g2_nand2_1` = 7.2576 μm^2. No clock constraint is applied: Yosys does not use one for area, so the figures do not depend on a target frequency. The ring oscillators are not synthesized; they are the one block designed at transistor level (Chapter 4).

7.2 Area per block in gate equivalents

Table 7.1 gives the result for each block. The digital engine — everything except the capture buffer — is about **46 kGE**. Synthesized on their own, the authenticator and the

generator each carry their own one-round-per-cycle encryption core, 16 kGE apiece, and that is how their rows in the table are measured; in the whole engine a single 32-bit core serves both, because the sequencer never lets them run at the same time (Section 7.3), so the rows do not add up to the engine's figure. Roughly: the shared 32-bit AES 6.7 kGE, the authenticator without its core 7, the generator without it 22, health tests 1.3, sampler, crossing and resets 0.4, I2C 1.1, and some 7 kGE of top-level sequencer, registers and core arbitration. The capture buffer is reported for completeness but must not be read as part of the product: its 46.8 kGE are a 4096-bit memory mapped onto flip-flops because the flow has no SRAM macros. In the FPGA it occupies a block RAM at no logic cost; in silicon it would be a compiled SRAM or, more sensibly, a much smaller buffer, because it is a test path that is disabled at production.

Table 7.1: Logic synthesis on IHP SG13G2, flattened, no routing. 1 GE = 7.2576 μm^2. Lower bound on area [SIMULATED].

Block	Cells	Flip-flops	Area (μm^2)	kGE
AES-128 core, one round per cycle (20 S-boxes, clear input)	10 571	262	116 550	16.06
AES-128 core, 32-bit datapath (4 S-boxes, 51 cycles/block)	3 813	266	48 434	6.67
AES-CMAC (with its one-round AES)	13 289	912	169 382	23.34
CTR_DRBG with df (with its one-round AES)	19 895	2 357	277 293	38.21
Health tests (RCT, APT)	588	86	9 137	1.26
ERO sampler and 24-bit divider	127	28	2 371	0.33
Bit clock-domain crossing	11	7	408	0.06
Reset synchronizer (one per clock domain)	3	3	147	0.02
I2C slave (with bus-hang guard)	435	89	7 729	1.07
Capture buffer (4096-bit RAM as flip-flops)	12 526	4 140	339 386	46.76
Whole engine, rings as wires, one shared one-round AES	39 879	8 175	739 061	101.83
Whole engine, one shared 32-bit AES	32 954	8 179	671 576	**92.53**

The numbers are internally consistent, and that consistency is how an error in the flow was caught: before flattening, the report returned for the authenticator and for the generator the area of their AES sub-module alone. An authenticator weighing exactly as much as a bare AES is impossible, so the report, not the design, was wrong. After flattening, the authenticator adds to the AES 650 flip-flops (sub-keys, chaining state, key) and 7.6 kGE, and the generator adds 2095 flip-flops and 22 kGE, about half of which are its registers. The table is the synthesis of the engine at register-map version 3, which

added the zeroization command, a request counter that saturates instead of wrapping, the capture exclusion rules and the reset synchronizers; the generator grew by 1.0 kGE for the saturating counter and the clear path, and the whole engine by 0.8 kGE, to 115.7 kGE with two cores; sharing the core took it to 101.8, and the 32-bit core to the 92.5 kGE of the table. The flow for this run read the VHSIC Hardware Description Language (VHDL) through GHDL's own Verilog netlist output and the `pip`-installable build of Yosys rather than the GHDL plugin; on the unchanged AES core the two paths agree within 0.9 %.

7.3 Optimizations and their expected savings

The synthesis is naive on purpose — the RTL was written to be verifiable, not small — and it points at three optimizations, in order of return on effort:

1. **One shared AES core — done.** The top-level sequencer already serializes commands: the generator and the authenticator never run at the same time. The core now lives in the top level and is handed to whichever block raises its start pulse; a property in the system bench checks that the two never do so in the same cycle. Measured saving: **14.1 kGE** (115.7 to 101.6), a little under the 15.8 estimated, because the arbitration costs about half a kGE and the clear input that the review required on the core and on the generator costs another. A generic keeps the two-core arrangement available.

2. **Compact S-box datapath — done.** Twenty independent S-box tables synthesize to 16 kGE for the one-round core; area-optimized designs in the literature range from 2.4 kGE for an 8-bit datapath at 226 cycles per block [81] to about 5 kGE for 32-bit datapaths. Speed is irrelevant here, so a second core was written with a 32-bit datapath: four S-boxes shared between SubBytes and the key schedule's SubWord, one MixColumns, ShiftRows applied as a wiring permutation of the state register at the start of each round, one column per cycle, 51 cycles per block instead of 11. It passes the same official vectors, the thousand-block chain and the zeroization test, and the bench requires its output to equal the one-round core's on every block. Measured: **6.67 kGE against 16.06** for the core (a saving of 9.4, against the 10 estimated, the difference being the column multiplexers and the extra phase register), and **92.5 against 101.8 kGE** for the whole engine in the same synthesis run. The one-round core remains available through a generic.

3. **Fewer registers in the generator.** Of its 2355 flip-flops, the captured entropy and nonce (384), the seed material (256), the output (256) and the temporaries of the derivation function can be reduced by consuming the seed as a stream and sharing temporaries; roughly 800 flip-flops, about 4 kGE.

The two measured optimizations took the digital engine from 69 kGE (two one-round cores) to 46; the third would leave it at **≈42 kGE**. That is above the 13 to 25 kGE the state of the art anticipated for engines of this kind, and the table says why: the generator without its core is 22 kGE, half of it registers, and the top-level sequencer and registers another 7; reaching the published range would mean rethinking the generator's storage rather than its cipher, which this work leaves stated rather than done. At the published growth factor of ≈2.35 from cell area to core area for an AES on this same library, 42 kGE is about 0.72 mm^2 of core, which still fits several times over in the smallest academic block (Section 7.5). The first two savings are measurements; the third is an estimate and is listed as future work, not claimed.

7.4 What the ring oscillator needs in silicon

The digital blocks port with the standard flow. The entropy source does not, and the literature that has done the crossing — notably a 2024 study that characterizes the same ring model on an Artix-7 and on a 28 nm FD-SOI circuit [32] — makes three points that the chip design must absorb.

First, the ring is rebuilt from inverter cells rather than look-up tables, and its jitter changes in value and in character: flicker noise gains weight in newer nodes, and the study finds, against expectation, that counting it as a legitimate source can raise throughput for the same entropy. The stochastic model and the estimator of Chapter 4 carry over unchanged; the measured σ does not. Second, process variation makes every die different, so the frequency and jitter of each ring must be calibrated per chip; the published all-digital designs that survive voltage and temperature sweeps do it with a configurable ring and a tuning loop [104]. Third, and most important for a security device, injecting a frequency on the supply locks the rings and removes the jitter: a 500 mV signal on the rail reduced the key space of a commercial secure microcontroller from 2^{64} to about 3300 [1], and a few hundred microwatts of radiated field destroyed the statistics of a fifty-ring generator [105]. The same calibration loop, together with supply filtering, is the published defence, and the health tests of Section 5.5 are the detector of last resort. None of this is in the FPGA prototype; it is the work that a silicon implementation adds.

7.5 Cost and schedule of an academic shuttle

Fabrication is more accessible than the size of the word suggests. The University of Sevilla and IMSE-CNM are members of Europractice, whose academic price list is public: a mini@sic block in UMC 180 nm costs 3 430 €, one in TSMC 65 nm 3 691 €, and IHP SG13G2 is offered at 5 110 €/mm^2 with a free multi-project run for open-source designs under 2 mm^2 [107]. An engine of 0.3–0.4 mm^2 fits many times over in the smallest block

of any of them; the chip would be limited by its pads, not its logic. The institute's own hardware-security group has already taken ring-oscillator primitives from Xilinx 7-series FPGAs to characterized 65 nm silicon, which makes the continuation of this work a matter of joining an existing line rather than opening one.

What does not fit in a master's thesis is the schedule: from tape-out to packaged samples takes months to over a year, the price does not include packaging, and the characterization of the returned dies is a project in itself. For comparison, a commercial secure element such as the ATECC608B costs 0.90 $ in single units and ships with a NIST entropy source validation [7]; two of its direct competitors do not mention SP 800-90 anywhere in their datasheets. The honest position of this work is therefore not to compete on price but on *transparency of the entropy*: every bit of a key produced by this engine can be traced to a measured jitter and a stated bound, which is precisely what a black-box part cannot offer.

Chapter 8

Results and Comparison

This chapter collects what the work establishes and separates it, without softening, from what it does not. The engine has been modelled, written, verified in simulation against official vectors and synthesized on a real standard-cell library; it has not yet run on the Field-Programmable Gate Array (FPGA). Every number below therefore carries its provenance tag, and the entropy campaign — the one result that requires silicon on the bench — is stated as a protocol to execute, not as data.

8.1 Status against the objectives

Table 8.1 maps the objectives of Section 1.3 onto evidence.

The minimum defensible result declared in Chapter 1 — O1–O3 together with O5 — is reached on its modelling and design side. What O5 still lacks is the bring-up, not the design.

8.2 Transistor-level noise of the ring

The ring was simulated with a current noise source of controlled spectral density injected at the switching node, sweeping the density and measuring the standard deviation of the rising-edge crossings over 500 periods. Table 8.2 gives the result for a five-stage ring with a 10 fF load.

Three readings, and the third is the one that matters.

The numerical floor is far enough below the signal. With the noise source switched off the measurement returns 1.38 fs, which is the resolution of the simulation itself, not physics. It sits about 870 times below the smallest jitter to be measured, so the floor does not contaminate the sweep. Reaching it required care: the first attempt returned 0.33 ps, of the same order as the quantity under study, and the cause was the ring's start-up

Table 8.1: Objectives and the evidence supporting each.

Obj.	State	Evidence
O1	met	Ring at transistor level on the open IHP SG13G2 PDK, with the noise of its own device models (Section 8.3); foundry PDK pending
O2	met	Jitter extracted from the waveforms; thermal and $1/f$ separated (bench and impulse sensitivity function); estimator validated on synthetic data with known Q
O3	met	Stochastic model closed: exact wrapped-Gaussian computation, design point $Q \geq 0.2286$ (Section 8.4)
O4	met	Engine written in portable VHDL-2008; seven testbenches pass against FIPS-197, SP 800-38B, RFC 4493 and Cryptographic Algorithm Validation Program (CAVP) vectors
O5	partial	Verifier firmware compiles and the protocol is closed; the board-level run is pending
O6	pending	Requires the FPGA on the bench (Section 8.10)
O7	met	Synthesis on IHP SG13G2, per-block area and optimization path (Chapter 7)
O8	partial	Structural comparison closed (Section 8.6); the ESP32 side of the numerical comparison is measured under SP 800-90B (Section 3.6); the engine side is simulated under the same battery (Section 8.8) and its measurement needs O6

Table 8.2: Period jitter of the five-stage ring against injected noise density [SIMULATED]. $\sigma_{\text{rel}} = \sigma/T$.

Noise density (A^2/Hz)	Crossings	T (ps)	σ (fs)	σ_{rel}
none (numerical floor)	500	230.30	1.38	6.0×10^{-6}
1×10^{-20}	501	230.37	1200	5.2×10^{-3}
4×10^{-20}	507	229.98	2416	1.1×10^{-2}
1.6×10^{-19}	509	227.77	25588	1.1×10^{-1}

transient being included in the statistics. Discarding the first twenty cycles removed it. A measurement floor that coincides with the signal is indistinguishable from a result, which is precisely why the floor was characterized before the sweep and not after.

The scaling law holds where it should. Thermal jitter accumulates as the square root of the noise density, so quadrupling the density must double σ. From 1×10^{-20} to 4×10^{-20} the measurement gives a factor of 2.01 against a predicted 2.00. That agreement is the evidence that what is being measured is the injected noise and not an artefact of the solver.

And it breaks at the top of the sweep, visibly. At 1.6×10^{-19}, sixteen times the reference density, σ grows by a factor of 21.3 where the law predicts 4.0, and the mean period drops from 230.4 to 227.8 ps. A zero-mean noise source cannot shift a mean period. The point is therefore outside the regime the model describes — the perturbation is large enough to change the operating point of the inverters — and it is reported rather than dropped, because the boundary of validity of a model is itself a result. Only the two central points are used for anything downstream.

8.3 The ring with the transistors of an open PDK

The same five-stage ring was simulated with the low-voltage transistors of the IHP SG13G2 open PDK ($W_n = 0.5$ µm, $W_p = 1.2$ µm, $L = 0.13$ µm, 1.2 V, 27 °C, typical corner, 1 fF per node as a pre-layout estimate of wiring), with the noise density measured on the same devices (Section 4.5) injected at every node. Table 8.3 gives the result over 400 ns, about 1070 periods per point. The simulator draws a new noise realization on every run and cannot be made to repeat one (in ngspice 47 the Gaussian generator behind the transient noise source is seeded from the process identifier, and none of its seed options reaches it), so the uncertainty of the jitter was obtained by repetition: the PDK thermal point was simulated eight times.

Table 8.3: Five-stage ring on IHP SG13G2 [SIMULATED]. The noise density is not assumed: it is the white floor of the transistors' own noise model at the switching point. The thermal point is the mean of eight independent realizations, ± one standard deviation of the mean; the other rows are single realizations. Power is the average over whole periods.

Injected noise	T (ps)	σ (fs)	σ/T	Exponent	P (µW)
none (numerical floor)	366.04	0.68	1.8×10^{-6}	0.42	143.0
PDK thermal, 4.13×10^{-24} A^2/Hz	365.97	176.7 ± 2.0	4.83×10^{-4}	0.99	143.0
thermal ×4	365.98	351.0	9.59×10^{-4}	0.92	143.0
thermal + $1/f$ of the PDK	365.97	183.3	5.01×10^{-4}	1.01	143.0

The resolution is stated first: 0.68 fs, 262 times below the jitter that the device noise produces, and with an exponent of 0.42 that marks it as numerical noise rather than a random walk. The eight thermal realizations spread by 3.3 % in σ (169.0 to 186.5 fs), so a single run carries that uncertainty and the mean carries 1.2 %. Quadrupling the density multiplies the jitter by 1.99 ± 0.06 against the expected 2, and the mean period does not move, so the bench stays in the linear regime that the placeholder bench left at its highest point. The accumulation exponent averages 0.99: white noise, as the stochastic model assumes. As an independent check, Abidi's closed form for ring oscillators [125], evaluated with quantities measured at the same bias point, predicts $\sigma/T = 1.2 \times 10^{-3}$; the simulation gives 0.41 times that, the same order. The long-channel form of that formula, which goes through the threshold voltage, does not apply here: at the switching point the devices sit only 0.07 V above threshold.

With $\sigma/T = 4.83 \times 10^{-4}$ the divider for $Q \geq 0.2286$ is $K_D = 981\,095$ (958 895 to 1 004 347 within one standard deviation), a sampling interval of 359 μs with an identical sampling ring, and 2.79 kbit/s per ERO; if the independent jitter of the sampling ring is also credited, K_D halves and the rate doubles, but that requires the two rings not to share noise through supply or substrate, which only silicon can show. One design consequence was acted on immediately: the divider counter had 20 bits, so the largest K_D was $2^{20} = 1\,048\,576$, a 7 % margin (4 to 9 %) over the requirement in the typical corner at room temperature. It now has 24 bits (four flip-flops, 0.04 kGE), and the full-engine bench and its cross-check were re-run with it.

Corners and temperature. The thermal point was repeated in the slow, typical and fast corners at -40, 27 and 125 °C, with the device noise recomputed at each point and four realizations per point [SIMULATED]. Cold is the worst case, not heat: σ/T rises with temperature in all three corners, because thermal noise follows kT while the period changes little, so the required divider falls by a factor of 2.5 to 3.5 from -40 to 125 °C. The slow corner is worse than the fast one, and the two combine worse than their product: the slow corner at -40 °C ($\sigma/T = 3.15 \times 10^{-4}$, period 438.6 ps) needs $K_D = 2\,306\,215$. There 2^{20} gives $H_\infty = 0.78$ and 2^{21} only 0.970, so a fixed divider valid over the whole range at the nominal 1.2 V is 2^{22}, with $H_\infty = 0.9995$ at the worst point and 0.51 to 0.82 kbit/s depending on the corner. Supply is the third factor and the strongest. At -40 °C in the slow corner, lowering the supply by 10 % to 1.08 V lengthens the period by 34 % while σ hardly moves, so the required divider grows by a factor of 1.65 to $K_D = 3\,808\,445$ (3.71 to 3.91×10^6 at $\pm 1\sigma$, eight realizations); at 1.32 V it shrinks by a factor of 0.79. The asymmetric corners at -40 °C (sf, fs) need 1.32 and 1.55×10^6, close to tt (1.47) and below ss; at 1.08 V, tt, sf and fs need 1.93, 1.57 and 1.74×10^6, half of ss. At the worst point, 2^{22} still gives $H_\infty = 0.9873$, but with a 10 % margin on K_D, close to the 7 % judged too thin for 2^{20} at the typical point; 2^{23} gives 0.9999 and 0.20 to 0.41 kbit/s, about 1.9 s

for the 384 bits of a seeding in the worst case. Device mismatch, drawn per transistor from the PDK statistical models with a new noise realization in every run, spreads the period by 1.3 % over 48 dies at the typical point and by 1.6 % over 15 at the worst point, but leaves σ/T where it was: its die-to-die spread (2.6 and 3.2 %) is no larger than the run-to-run spread of the noise alone. A sixteenth worst-point run was discarded because its crossing count does not fit in the simulated time, most likely double crossings of a slow edge; the crossing detector has since gained hysteresis and records any such event with its waveform. The worst of the 15 dies needs $K_D = 4.49 \times 10^6$, above 2^{22} ($H_\infty = 0.973$), and 2^{23} still covers it ($H_\infty = 0.9996$). The divider register resets to 2^{24}, so a master that does not write it is already on the safe side; a lower setting has to come from the coldest corner and the lowest supply the product must work at.

Flicker noise. Adding the $1/f$ noise of the PDK leaves 400 ns unchanged: σ rises by 3.8 %, within the spread between realizations, and the separation $E[D^2] = a_1 M + a_2 M^2$, validated beforehand on synthetic phases (thermal recovered to 0.1 % with and without $1/f$), finds a_2 compatible with zero. Amplifying the $1/f$ source to make it visible does not work, and the reason is instructive: the source is broadband, and amplified a thousandfold its density near the oscillation frequency is already seven times the thermal floor, so the ring converts it as white noise and the linear term grows by the expected factor while the quadratic term does not scale. The answer comes instead from the impulse sensitivity function of Hajimiri and Lee [126], measured by injecting 1 % of the node charge at 40 phases (linear to 0.1 % between 0.5 and 2 %). It is nearly symmetric, $|\Gamma_{dc}/\Gamma_{rms}| \approx 0.03$–$0.05$, so low-frequency noise injected uniformly over the period is barely converted. But the transistors are not symmetric: measured separately, the p-channel device has three times the $1/f$ noise of the n-channel one, and each injects noise mainly while it conducts, during one of the two opposite lobes. The two limits bracket the truth: with stationary noise, as in the bench, the $1/f^3$ corner is 36 kHz and flicker equals thermal jitter after 27 000 periods; with each device confined to its lobe, the corner is 5.3 MHz and the crossover 185 periods. In both cases, at the operating divider flicker dominates the accumulated variance, by a factor between 36 and 5 300. Two consequences follow. The jitter that sets the entropy bound must be measured at short accumulation, below about a hundred periods for this ring, and separated from the quadratic term — inferring it from the total accumulated jitter at K_D would credit entropy that is not there. And the stationary bench may overestimate the thermal variance by up to a factor of two, which would double K_D; the 24-bit counter covers it, and the tool that settles it is a cyclostationary transient-noise analysis, a concrete use of a commercial simulator.

8.4 Operating point: divider and throughput

The stochastic model fixes the target. For the PTG.2 thresholds of AIS 20/31 v3.0, the min-entropy criterion ($H_\infty \geq 0.98$) demands $Q \geq 0.2286$, and is more demanding than the Shannon criterion ($H \geq 0.9998$, $Q \geq 0.2022$); the design therefore targets the former, with $Q = 0.30$ as the working set-point, which yields $H_\infty = 0.9951$. Two properties of the model were established rather than assumed: the truncated series of equation 14 in [10] *overestimates* entropy and is thus not a lower bound, so the exact wrapped-Gaussian computation is used instead; and the first-harmonic derivation of min-entropy agrees with the exact one to six digits over the whole range of interest.

Since $Q_{\text{gen}} = K_D \cdot Q_{\text{raw}}$, the divider follows from the jitter that the campaign measures. Table 8.4 gives the operating points for the jitter range expected of this technology [8], assuming RO2 at 500 MHz.

Table 8.4: Divider and resulting throughput per ring for $Q \geq 0.2286$ [ESTIMATED].

σ/T per period	Q_{raw}	K_D	Throughput
7.1×10^{-4}	5×10^{-7}	457 200	1.09 kbit/s
1.4×10^{-3}	2×10^{-6}	114 300	4.37 kbit/s
2.8×10^{-3}	8×10^{-6}	28 575	17.5 kbit/s

These bracket the published design point of Fischer and Lubicz [13] ($K_D \approx 430\,000$ for $\sigma = 5.01$ ps on $T = 7.81$ ns), which is the sanity check that the model is being applied as its authors intended.

The architectural consequence is stated plainly: a single ERO delivers kbit/s, not Mbit/s. That is the price of demonstrable entropy, and it is affordable here because the engine's product is a 256-bit seed for the DRBG, not a bit stream. More throughput means more independent rings, each with its own bound, not a faster ring.

8.5 Functional verification, consolidated

Chapter 6 describes the strategy; this section records the totals. Every figure in Table 8.5 was obtained by running the suite, not by reading a previous log.

Two of these deserve a sentence. The 960 CAVP cases are the strongest single piece of evidence in the work: the generator is not merely self-consistent, it reproduces the outputs that NIST publishes for CTR_DRBG with AES-128, in both the derivation-function and the raw-entropy configurations. And the health-test cutoffs, $C_{RCT} = 22$ and $C_{APT} = 596$, had been computed by hand from the standard and were never checked; deriving them again from the formulas of SP 800-90B [6] confirms both. The same check quantifies the cost of the complement test added to the Adaptive Proportion Test (APT), which is an addition of this work and not of the standard: it raises the false-alarm rate per window

Table 8.5: Verification totals [SIMULATED].

Suite	What it checks	Result
VHDL testbenches (GHDL)	rings, ERO, capture, health tests, I2C, Advanced Encryption Standard (AES), Cipher-based Message Authentication Code (CMAC), CTR_DRBG, top level	7/7 pass
`ctr_drbg_ref.py`	CAVP vectors, with and without derivation function	960/960
`ero_model.py`	stochastic model, exactness and bounds	24/24
`jitter_estimator.py`	jitter recovery on synthetic data of known Q	20/20
`cortes_salud.py`	health-test cutoffs against SP 800-90B formulas	6/6
`verifica_top.py`	cross-check of the top-level bench with an independent AES	5/5

from $2^{-20.15}$ to $2^{-20.14}$, a 0.8 % increase, because the complementary tail lies 5.7 standard deviations away. The cutoff therefore remains valid as it stands and the addition is free in practice.

8.6 Comparison with commercial secure elements

The engine occupies the same functional slot as a commercial secure element: a small companion device that generates and holds keys for a host microcontroller over a two-wire bus. Table 8.6 places it beside the three parts that dominate the I2C secure-element market.

The honest reading of that table has two halves.

On capability and assurance the commercial parts win, and it is not close. Two of them carry CC EAL6+ evaluations, which represent an amount of third-party scrutiny — including physical attack resistance — that no academic project reproduces. They offer asymmetric cryptography, certified provisioning flows and manufacturing support. This work offers symmetric key generation and authentication, and carries no certification at all.

On what the thesis is actually about, the comparison is more nuanced than "open versus closed", and it pays to be exact. Of the three, only the ATECC608B has its entropy source validated under SP 800-90B: the NIST record names ring oscillators and states 0.5071 bit of entropy per sample [102]. That is a real, third-party figure, and it must be credited. What no vendor publishes — not Microchip either — is the stochastic model behind the figure, the jitter measurement that supports it, or the circuit itself; the datasheet of one competitor claims compliance without a validation record and the other does not mention SP 800-90 at all. Their randomness is trusted because the vendor and the evaluation scheme are trusted. The engine described here makes the opposite

Table 8.6: The engine against commercial secure elements. Certification data from the manufacturers' datasheets and from the NIST validation records [7, 102, 98, 99].

	ATECC608B	SE050	OPTIGA Trust M	This work
Bus	I2C	I2C	I2C	I2C
Certification	JIL High; NIST ESV #E46 for the entropy source	CC EAL6+ (up to OS)	CC EAL6+ (hardware)	none
Entropy source	ring oscillators, per the ESV record	"TRNG, SP 800-90B" in datasheet	not stated	ERO, published
Entropy claim	0.5071 bit/sample, NIST-validated	not published	not published	bound derived from measured jitter
Stochastic model	not published	not published	not published	published
Key export	not possible	not possible	not possible	test mode only
Inspectable	no	no	no	yes, RTL released

trade: it has no certificate, and it can show where each bit of entropy comes from, what physical measurement bounds it, and what the bound becomes when the temperature or the divider changes. That is the same distinction that the black-box study of the ESP32 generator in this work runs into from the other side (Section 3.6) — a black box can be tested statistically but cannot be bounded — and it is the reason this engine was built as a white box.

Which of the two is preferable depends on the question being asked. For shipping a product, the certified part. For knowing what the randomness in that product is worth, only the open one answers.

8.7 The ESP32 reference

Chapter 3 characterizes the generator of the ESP32 under four operating conditions and on three chips; the comparison of O8 takes its figures from there.

Three consequences follow for O8. First, the figure this thesis carries forward as the ESP32 reference is the non-IID one, 0.81 to 0.92 bit per bit with a mean of 0.85 over the thirty-six captures of conditions A–D, because it is what an evaluator would credit without granting the IID assumption, with the restart test as the reminder that for condition D it is a statistical figure and not physical entropy; the engine's raw bits, when the campaign of Section 8.10 produces them, must be run through the same battery at the same N, and compared estimator by estimator, since the same conservative bias will

act on them; the calibration of the IID tests at that length applies to them unchanged. Second, the Shannon figure of 0.99999 bit per bit obtained in the first campaign and the min-entropy figures above are not in contradiction — $H_\infty \leq H$ always — but they are different currencies: crediting 256 bits of entropy costs 256 output bits in the first and up to 316 in the second, and only the second is the one key generation may use. Third, and independent of any number, the measurement is of a black box: the source and post-processing behind `esp_fill_random` are undocumented, and the SP 800-90B estimators are designed for raw noise, not for an output that may already be conditioned; the restart test shows how far that can go, a stream of pseudo-random output that every estimator scores like noise. What the ESP32 cannot supply, whatever the estimators return, is the bound that Section 8.4 derives for the engine from a physical measurement. That asymmetry is the comparison.

8.8 The simulated source under the same battery

Before the campaign, the engine side of O8 can be run in simulation through exactly the pipeline that produced the ESP32 figures: same binaries, options and length. The bits come from the stochastic model of the ERO driven by the jitter of the open-PDK ring (Section 8.3) at three points — tt at 27 °C, ss at −40 °C and 1.08 V, and the worst chip with mismatch at that point — for dividers from 2^{14} to 2^{24}, three seeds each, 10^6 bits per sequence [SIMULATED]. As a reference, uniform independent bits of the same length were run through the same battery. Figure 8.1 sets the exact min-entropy of the model against the non-IID estimate.

Four readings follow. First, at 10^6 bits the battery saturates: uniform bits score 0.839 ± 0.021, and every point where the model gives $H_\infty \geq 0.97$ scores between 0.82 and 0.87, always limited by the compression estimator. Above that ceiling the battery tells nothing apart, so the ESP32, at 0.85, and the engine at the recommended 2^{23}, at 0.84 to 0.85 in the three cases, are indistinguishable at this length; the figures must not be compared as if they differed. Second, at low entropy the battery *overestimates*: at 2^{14} the model gives $H_\infty \approx 0$ and the battery 0.07 to 0.11, and up to 2^{19} at the worst point 0.10 to 0.32 against 0 to 0.25. The source is then nearly deterministic, its phase advancing by a known step with little noise, and generic predictors do not exploit that structure fully; the model does, because it grants the attacker the previous phase. A divider chosen too small would not have been caught by the battery alone, which is the case AIS 31 makes for a stochastic model, here in the design's own numbers. Third, the IID track accepts data that is not full-entropy: at 2^{21} the worst chip ($H_\infty = 0.79$) and the ss point (0.85) pass the three IID tests in five sequences of six, and the IID credit, the byte most-common-value estimate, would be 0.86 to 0.90 bit per bit, above the model; at 2^{20} ($H_\infty \approx 0.5$) the χ^2 test rejects every sequence. This supports carrying the non-IID figure for the ESP32 and

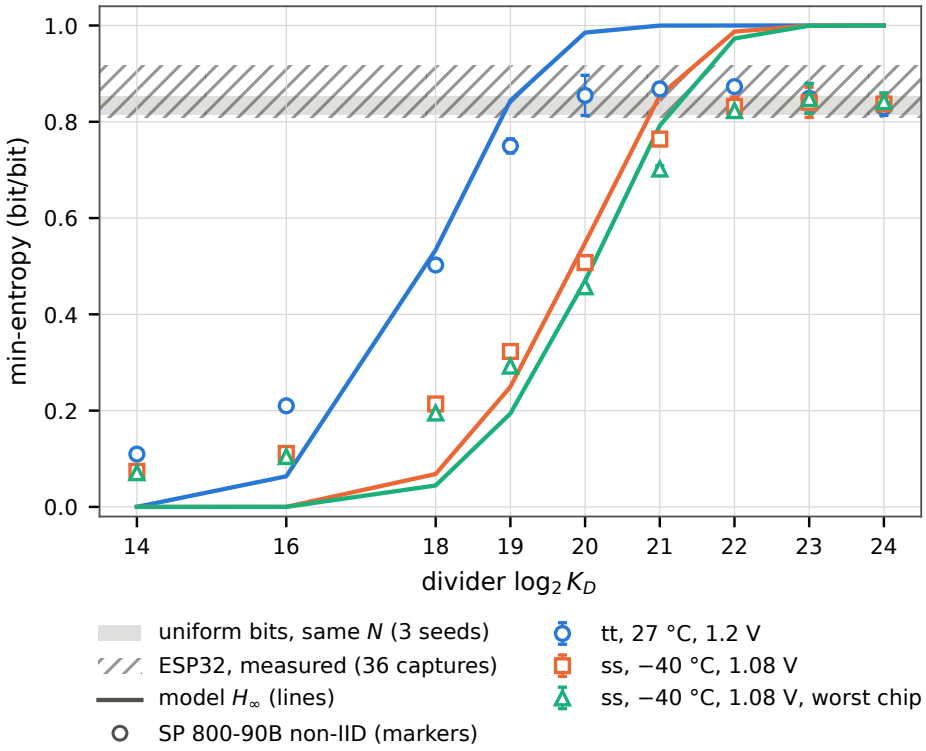


Figure 8.1: Min-entropy of the simulated source against the divider [SIMULATED]: the stochastic model (lines) and the SP 800-90B non-IID estimate on 10^6 bits (markers, mean and standard deviation of three seeds). The grey band is the estimate for uniform independent bits of the same length; the hatched band spans the thirty-six ESP32 captures of conditions A–D [MEASURED].

applying the same rule to the engine. Fourth, between those regimes, for H_∞ from about 0.5 to 0.95, the estimate sits below the model, as a conservative estimator should.

A longer record raises the ceiling, but for every source alike. At the lengths of the two long radio-off ESP32 captures, 3.52×10^6 and 8.38×10^6 bits, the divider 2^{23} was rerun in the three cases with uniform bits as reference, three seeds each [SIMULATED]. The twelve sequences with $H_\infty \approx 1$ average 0.881 (0.860 to 0.925) and 0.912 (0.889 to 0.935); the ESP32 captures score 0.867 and 0.911 [MEASURED], inside those spreads at both lengths, and the ideal bits score 0.869 ± 0.003 and 0.926 ± 0.016. The limiting estimator is still compression, or the t-tuple in three sequences, whose bias shrinks only slowly with N. At 8.38×10^6 bits the IID track in bytes accepts all twelve sequences, as it does the ESP32 capture. The comparison of objective O8 therefore cannot rest on a difference between SP 800-90B figures: above its ceiling the battery only says that both sources are above it. What separates them is what stands behind the figure, a physical model and a divider derived from it on the engine side and none on the ESP32 side, which without boot noise repeats sequences across chips (Table 3.4).

These bits contain only white Gaussian jitter. Flicker noise, coupling between rings, a shared supply and slow drift are absent, and they are what the battery is least able to see and the campaign must measure. Three seeds make the spreads indicative only. The reading in Spanish, with the byte and IID tables, is in `docs/fuente_simulada_90b.md`.

8.9 Uncertainty budget

Five quantities carry the conclusions, and each has a different and declared kind of uncertainty.

Simulated jitter. Resolution floor 1.38 fs [SIMULATED], about 870 times below the smallest quantity measured. The scaling check against the expected square-root law agrees to 0.5 % in the valid regime, and the regime boundary is documented in Section 8.2. On the open PDK the floor is 0.68 fs, 262 times below the jitter, whose run-to-run spread is 3.3 % (1.2 % for the mean of eight), and the noise density is no longer assumed but taken from the device models. What remains approximate is how that noise is applied: stationary injection may overestimate the thermal variance by up to a factor of two against a cyclostationary model, and pre-layout wiring is a 1 fF estimate. Process corners and temperature were covered (ss, tt, ff at -40 to 125 °C), and at -40 °C also the asymmetric corners, a ±10 % supply and the tt, sf and fs corners at low supply, and device mismatch at the typical and the worst point; ff at low supply and low supply when hot were not. The figure is therefore a simulated value with a factor-of-two band, not the jitter of a fabricated device.

Recovered Q. The estimator reproduces a known Q with a bias below 2 % and a dis-

persion of 4.0 % over eight seeds with 20 000 blocks [SIMULATED]. Without the corrections applied here a plain straight-line fit biases the result by −18 %, so the correction is not cosmetic.

Area. A lower bound [SIMULATED]: cells only, flattened, without routing, fill or clock tree. Published growth factors from cell area to core area on this library are ≈ 2.35 for an AES, and the optimization savings of Chapter 7 are [ESTIMATED] and explicitly not claimed as measured.

Min-entropy of the ESP32 reference. Measured [MEASURED] on thirty-eight captures of 10^6 to 8.4×10^6 bits, nine per condition, three on each of three chips of the same model identified by MAC address. The IID-track figure is at the resolution ceiling of the MCV estimator for that N (0.9963 at 10^6 bits), so it is a floor on the ceiling rather than a measurement of the source, and the IID verdicts are read against the measured false-failure rate of each test at that length, not against the nominal one. The non-IID figure is set by estimators whose conservative bias on nearly uniform data exceeds the spread between conditions; an analysis of variance finds no condition effect ($p \geq 0.045$ on every estimator) and no chip effect ($p \geq 0.032$, against 0.0045 after correction). Three devices show that the reference does not hinge on one of them; they do not bound the spread across a production population. The restart test was run on two chips, in condition D, with the Wi-Fi started but not associated, and on the second chip with each source switched on and off in turn; Bluetooth and an associated radio were not tested. It bounds the entropy that the stock bootloader injects only from below: no repeat in 1000 restarts means more than about 1.7×10^5 equally likely starting states, some 17 bits, at 95 % confidence. The size of the generator's internal state is not documented.

Min-entropy of the built device. Not yet bounded. The model, the estimator and the cutoffs are ready; the number requires the campaign of Section 8.10. No value is offered here, and none should be inferred from the simulated jitter, nor from the simulated source of Section 8.8, whose bits carry only the noise the model puts in.

8.10 The pending campaign

The one result that silicon must supply is the min-entropy of the rings as built, and the procedure is fixed in advance so that it cannot be tuned after seeing the data.

1. **Confirm the rings oscillate.** Read the frequency of each ring with the embedded counter before trusting any entropy measurement. A ring removed by the synthesizer, or two rings that have locked to each other, would otherwise be indistinguishable from a working source.

2. **Measure the jitter** with $K_D = 1$ by the method of Fischer and Lubicz [13], sweeping the pair distance M and fitting the linear region, with at least three rings so that the global component separates from the local one [31]. Fit $\sigma^2(t) = at + bt^2$ and keep only the thermal term.

3. **Fix** K_D from the measured jitter for $Q \geq 0.2286$ with margin, and recompute the health-test cutoffs for the measured H_∞ rather than the assumed 0.98, using `cortes_salud.py`.

4. **Collect raw bits** at that operating point and run the NIST SP 800-90B estimators, contrasting the empirical estimate with the model's bound. Agreement supports the model; disagreement is a result about the model's hypotheses, and either way it is reported.

5. **Repeat across placement** — rings in separated regions versus adjacent ones — to quantify the coupling that the literature warns about, which is the measurement the architecture of this engine was chosen to make possible.

Until step 4 closes, the engine's claim is precise and limited: the chain from physical noise to key material is designed, implemented and verified against the standards, and the bound it will produce is computable from a measurement that has not yet been taken.

Chapter 9

Conclusions and Future Work

9.1 Conclusions

9.2 Contributions

9.3 Future work

Bibliography

[1] A. T. Markettos and S. W. Moore, "The frequency injection attack on ring-oscillator-based true random number generators," in *Cryptographic Hardware and Embedded Systems – CHES 2009*, ser. LNCS, vol. 5747. Springer, 2009, pp. 317–331. [Online]. Available: https://www.iacr.org/archive/ches2009/57470316/57470316.pdf

[2] D. Hurley-Smith and J. Hernandez-Castro, "Certifiably biased: An in-depth analysis of a common criteria EAL4+ certified TRNG," *IEEE Transactions on Information Forensics and Security*, vol. 13, no. 4, pp. 1031–1041, 2018.

[3] J. Blanco-Romero, F. Almenares Mendoza, D. Díaz-Sánchez, and A. Marín-López, "Entropy bootstrapping for wireless embedded systems," arXiv:2607.08865, Sep. 2026. [Online]. Available: https://arxiv.org/abs/2607.08865

[4] National Institute of Standards and Technology, "Decision to revise NIST SP 800-22 Rev. 1a," Apr. 2022. [Online]. Available: https://www.nist.gov/news-events/news/2022/04/decision-revise-nist-sp-800-22-rev-1a

[5] M. Peter and W. Schindler, "A proposal for functionality classes for random number generators, version 3.0," Bundesamt für Sicherheit in der Informationstechnik (BSI), Mathematical-technical reference of AIS 20/31, Sep. 2024. [Online]. Available: https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Certification/Interpretations/AIS_31_Functionality_classes_for_random_number_generators_e_2024.pdf

[6] M. S. Turan, E. Barker, J. Kelsey, K. A. McKay, M. L. Baish, and M. Boyle, "Recommendation for the entropy sources used for random bit generation," NIST, Special Publication SP 800-90B, Jan. 2018. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/90/b/final

[7] *ATECC608B CryptoAuthentication Device Summary Data Sheet*, Microchip Technology Inc., 2021. [Online]. Available: https://ww1.microchip.com/downloads/aemDocuments/documents/SCBU/ProductDocuments/DataSheets/ATECC608B-CryptoAuthentication-Device-Summary-Data-Sheet-DS40002239B.pdf

[8] O. Petura, U. Mureddu, N. Bochard, V. Fischer, and L. Bossuet, "A survey of AIS-20/31 compliant TRNG cores suitable for FPGA devices," in *26th International Conference on Field Programmable Logic and Applications (FPL 2016)*, 2016, pp. 1–10. [Online]. Available: https://hal.science/hal-01382955/document

[9] D. Lubicz and V. Fischer, "Recommendations for the design and validation of a physical true random number generator integrated in an electronic device, version 1.0," IACR Cryptology ePrint Archive, Report 2024/301, Feb. 2024. [Online]. Available: https://eprint.iacr.org/2024/301.pdf

[10] M. Baudet, D. Lubicz, J. Micolod, and A. Tassiaux, "On the security of oscillator-based random number generators," *Journal of Cryptology*, vol. 24, no. 2, pp. 398–425, 2011. [Online]. Available: https://eprint.iacr.org/2009/299.pdf

[11] W. Killmann and W. Schindler, "A design for a physical RNG with robust entropy estimators," in *Cryptographic Hardware and Embedded Systems – CHES 2008*, ser. LNCS, vol. 5154. Springer, 2008, pp. 146–163.

[12] Y. Ma, J. Lin, T. Chen, C. Xu, Z. Liu, and J. Jing, "Entropy evaluation for oscillator-based true random number generators," in *Cryptographic Hardware and Embedded Systems – CHES 2014*, ser. LNCS, vol. 8731. Springer, 2014, pp. 544–561.

[13] V. Fischer and D. Lubicz, "Embedded evaluation of randomness in oscillator based elementary TRNG," in *Cryptographic Hardware and Embedded Systems – CHES 2014*, ser. LNCS, vol. 8731. Springer, 2014, pp. 527–543. [Online]. Available: https://www.iacr.org/archive/ches2014/87310109/87310109.pdf

[14] B. Sunar, W. J. Martin, and D. R. Stinson, "A provably secure true random number generator with built-in tolerance to active attacks," *IEEE Transactions on Computers*, vol. 56, no. 1, pp. 109–119, 2007.

[15] K. Wold and C. H. Tan, "Analysis and enhancement of random number generator in FPGA based on oscillator rings," *International Journal of Reconfigurable Computing*, vol. 2009, p. Article ID 501672, 2009.

[16] N. Bochard, F. Bernard, V. Fischer, and B. Valtchanov, "True-randomness and pseudo-randomness in ring oscillator-based true random number generators," *International Journal of Reconfigurable Computing*, vol. 2010, pp. Article ID 879 281, 1–13, 2010.

[17] P. Kohlbrenner and K. Gaj, "An embedded true random number generator for FPGAs," in *Proc. ACM/SIGDA 12th International Symposium on Field Programmable*

Gate Arrays (FPGA 2004), 2004, pp. 71–78. [Online]. Available: https://people-ece.vse.gmu.edu/~kgaj/publications/conferences/GMU_FPGA_2004.pdf

[18] B. Valtchanov, V. Fischer, and A. Aubert, "Enhanced TRNG based on the coherent sampling," in *3rd International Conference on Signals, Circuits and Systems (SCS 2009)*, 2009, pp. 1–6.

[19] A. Peetermans, V. Rožić, and I. Verbauwhede, "Design and analysis of configurable ring oscillators for true random number generation based on coherent sampling," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 14, no. 2, pp. 7:1–7:20, 2021. [Online]. Available: https://cosicdatabase.esat.kuleuven.be/backend/publications/files/journal/3280

[20] ——, "A highly-portable true random number generator based on coherent sampling," in *29th International Conference on Field Programmable Logic and Applications (FPL 2019)*, 2019, pp. 218–224. [Online]. Available: https://lirias.kuleuven.be/retrieve/0ee069b3-afb3-43e3-9fe3-9e6d0b90ae46

[21] M. Varchola and M. Drutarovský, "New high entropy element for FPGA based true random number generators," in *Cryptographic Hardware and Embedded Systems – CHES 2010*, ser. LNCS, vol. 6225. Springer, 2010, pp. 351–365. [Online]. Available: https://www.iacr.org/archive/ches2010/62250341/62250341.pdf

[22] F. Bernard, P. Haddad, V. Fischer, and J. Nicolai, "From physical to stochastic modeling of a TERO-based TRNG," *Journal of Cryptology*, vol. 32, no. 2, pp. 435–458, 2019.

[23] A. Cherkaoui, V. Fischer, L. Fesquet, and A. Aubert, "A very high speed true random number generator with entropy assessment," in *Cryptographic Hardware and Embedded Systems – CHES 2013*, ser. LNCS, vol. 8086. Springer, 2013, pp. 179–196. [Online]. Available: https://www.iacr.org/archive/ches2013/80860154/80860154.pdf

[24] B. Yang, V. Rožić, M. Grujić, N. Mentens, and I. Verbauwhede, "ES-TRNG: A high-throughput, low-area true random number generator based on edge sampling," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2018, no. 3, pp. 267–292, 2018. [Online]. Available: https://tches.iacr.org/index.php/TCHES/article/view/7276

[25] V. Rožić, B. Yang, W. Dehaene, and I. Verbauwhede, "Highly efficient entropy extraction for true random number generators on FPGAs," in *Proc. 52nd Annual Design Automation Conference (DAC 2015)*, 2015, pp. 116:1–116:6.

[26] CEA-Leti, "Opentrng: open-source reference ptrngs based on ring oscillators (ero, muro, coso) — hardware documentation and repository," https://opentrng.org/docs/hardware.html ; https://github.com/opentrng/ptrng, 2025.

[27] W. Schindler and W. Killmann, "Evaluation criteria for true (physical) random number generators used in cryptographic applications," in *Cryptographic Hardware and Embedded Systems – CHES 2002*, ser. LNCS, vol. 2523. Springer, 2003, pp. 431–449.

[28] P. Haddad, Y. Teglia, F. Bernard, and V. Fischer, "On the assumption of mutual independence of jitter realizations in P-TRNG stochastic models," in *Design, Automation & Test in Europe (DATE 2014)*, 2014, pp. 1–6.

[29] M.-J. O. Saarinen, "On entropy and bit patterns of ring oscillator jitter," arXiv:2102.02196 [cs.CR], v4, Oct. 2021. [Online]. Available: https://arxiv.org/abs/2102.02196

[30] V. Fischer, F. Bernard, N. Bochard, and M. Varchola, "Enhancing security of ring oscillator-based TRNG implemented in FPGA," in *International Conference on Field Programmable Logic and Applications (FPL 2008)*, 2008, pp. 245–250.

[31] D. Lubicz and M. Skórski, "Improved phase noise measurement for multi-ring oscillator based TRNG (arxiv title: Quantifying jitter transfer for differential measurement: Enhancing security of oscillator-based TRNGs)," arXiv:2410.08259 [cs.CR], Oct. 2024. [Online]. Available: https://arxiv.org/abs/2410.08259

[32] L. Benea, M. Carmona, V. Fischer, F. Pebay-Peyroula, and R. Wacquez, "Impact of the flicker noise on the ring oscillator-based TRNGs," in *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2024.

[33] D. Lubicz and N. Bochard, "Towards an oscillator based TRNG with a certified entropy rate," *IEEE Transactions on Computers*, vol. 64, no. 4, pp. 1191–1200, 2015.

[34] W. Killmann and W. Schindler, "A proposal for: Functionality classes for random number generators, version 2.0," Bundesamt für Sicherheit in der Informationstechnik (BSI), Tech. Rep., Sep. 2011. [Online]. Available: https://cosec.bit.uni-bonn.de/fileadmin/user_upload/teaching/15ss/15ss-taoc/01_AIS31_Functionality_classes_for_random_number_generators.pdf

[35] Bundesamt für Sicherheit in der Informationstechnik, "Transition policy to version 3.0 of the mathematical/technical reference, part of

AIS 20/31, version 1.0," BSI, Tech. Rep., Apr. 2025. [Online]. Available: https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Certification/Interpretations/AIS_20_31_Transition_Policy_Version_3_0.pdf

[36] W. Schindler, "The new AIS 20/31," Presentación, European Cyber Week 2024, Rennes, Nov. 2024. [Online]. Available: https://www.creachlabs.fr/sites/default/files/public/media/document/2024-12/ecw2024-rng-puf-schindlerwerner-20241120.pdf

[37] National Institute of Standards and Technology, "Sp800-90b_entropyassessment: C++ implementation of the min-entropy assessment methods of sp 800-90b," GitHub repository, 2018. [Online]. Available: https://github.com/usnistgov/SP800-90B_EntropyAssessment

[38] L. E. Bassham, A. L. Rukhin, J. Soto, J. R. Nechvatal, M. E. Smid, E. B. Barker, S. D. Leigh, M. Levenson, M. Vangel, D. L. Banks, N. A. Heckert, J. F. Dray, and S. Vo, "A statistical test suite for random and pseudorandom number generators for cryptographic applications," National Institute of Standards and Technology, NIST Special Publication 800-22 Rev. 1a, Apr. 2010.

[39] M.-J. O. Saarinen, "SP 800-22 and GM/T 0005-2012 tests: Clearly obsolete, possibly harmful," IACR Cryptology ePrint Archive, Report 2022/169, 2022. [Online]. Available: https://eprint.iacr.org/2022/169

[40] A. Iwasaki, "Analysis of NIST SP800-22 focusing on randomness of each sequence," arXiv:1710.01441, 2017. [Online]. Available: https://arxiv.org/abs/1710.01441

[41] "Public comments on SP 800-22 rev. 1a (crypto publication review project)," National Institute of Standards and Technology, 2021. [Online]. Available: https://csrc.nist.gov/csrc/media/Projects/crypto-publication-review-project/documents/initial-comments/sp800-22r1a-initial-public-comments-2021.pdf

[42] D. Hurley-Smith and J. Hernandez-Castro, "Great expectations: A critique of current approaches to random number generation testing & certification," in *Security Standardisation Research (SSR 2018)*, ser. LNCS, vol. 11322. Springer, 2018, pp. 143–163.

[43] "ESP-IDF programming guide: Random number generation (ESP32)," Espressif Systems, 2026. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/random.html

[44] *ESP32 Technical Reference Manual, Version 5.8, Chapter 18: Random Number Generator (RNG)*, Espressif Systems, 2025. [Online].

Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf

[45] J. Castillo, P. Aran Vila, F. Palacio, B. Garrido, S. Hernández, and A. Cirera, "Automated framework for testing random number generators for IoT security applications using NIST SP 800-22," *IoT*, vol. 7, no. 1, p. 26, 2026.

[46] P. Hidayatulloh, N. S. Qolbiyah, A. Nugroho, H. Mukhtar, I. Purnama, and F. Simanjuntak, "Performance evaluation of ESP32 random number generator for LoRa communication security," in *2024 International Conference on Radar, Antenna, Microwave, Electronics, and Telecommunications (ICRAMET)*, 2024, pp. 44–49.

[47] N. Heninger, Z. Durumeric, E. Wustrow, and J. A. Halderman, "Mining your Ps and Qs: Detection of widespread weak keys in network devices," in *Proceedings of the 21st USENIX Security Symposium*. USENIX Association, 2012, pp. 205–220. [Online]. Available: https://www.usenix.org/conference/usenixsecurity12/technical-sessions/presentation/heninger

[48] D. E. Eastlake 3rd, J. I. Schiller, and S. Crocker, "Randomness requirements for security," RFC 4086, BCP 106, IETF, 2005. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4086

[49] J. Tillmanns, J. Classen, F. Rohrbach, and M. Hollick, "Firmware insider: Bluetooth randomness is mostly random," in *14th USENIX Workshop on Offensive Technologies (WOOT '20)*, 2020. [Online]. Available: https://arxiv.org/abs/2006.16921

[50] "ESP-IDF source: components/bootloader_support/src/bootloader_random.c," Espressif Systems, 2026. [Online]. Available: https://github.com/espressif/esp-idf/blob/master/components/bootloader_support/src/bootloader_random.c

[51] "ESP-IDF source: components/mbedtls/port/esp_hardware.c (mbedtls_hardware_poll)," Espressif Systems, 2026. [Online]. Available: https://github.com/espressif/esp-idf/blob/master/components/mbedtls/port/esp_hardware.c

[52] Espressif Systems, "Mbed TLS — ESP-IDF programming guide v5.4 (ESP32)," https://docs.espressif.com/projects/esp-idf/en/v5.4/esp32/api-reference/protocols/mbedtls.html, 2025.

[53] R. Salles and R. Farias, "TLS protocol analysis using IoTST — an IoT benchmark based on scheduler traces," *Sensors*, vol. 23, no. 5, p. 2538, 2023.

[54] KaeLL and Espressif Systems, "Clarification on the HW RNG (IDFGH-13813), espressif/esp-idf issue #14665," 2024. [Online]. Available: https://github.com/espressif/esp-idf/issues/14665

[55] Espressif esp-idf community, "boot: Disabling RNG early entropy source? (IDFGH-5906), espressif/esp-idf issue #7598," 2021. [Online]. Available: https://github.com/espressif/esp-idf/issues/7598

[56] "ESP-IDF programming guide: Random number generation (ESP32-C3)," Espressif Systems, 2026. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/stable/esp32c3/api-reference/system/random.html

[57] M. Cheich, "Just how random is the ESP32 random number generator?" 2025. [Online]. Available: https://www.programmingelectronics.com/esp32-random-number-generator/

[58] AndroidCrypto, "ESP32: How to generate true random numbers ("true RNG")," 2024. [Online]. Available: https://medium.com/@androidcrypto/esp32-how-to-generate-true-random-numbers-true-rng-951fcda58e9b

[59] STMicroelectronics, "AN4230: Introduction to random number generation validation using the NIST statistical test suite for STM32 MCUs and MPUs." [Online]. Available: https://www.st.com/resource/en/application_note/an4230-introduction-to-random-number-generation-validation-using-the-nist-statistical-test-suite-for-stm32-mcus-and-mpus-stmicroelectronics.pdf

[60] K. Ngo and E. Dubrova, "Side-channel analysis of the random number generator in STM32 MCUs," in *Proceedings of the Great Lakes Symposium on VLSI 2022 (GLSVLSI '22)*. ACM, 2022. [Online]. Available: https://dl.acm.org/doi/10.1145/3526241.3530324

[61] "nRF52840 product specification: RNG — random number generator," Nordic Semiconductor. [Online]. Available: https://infocenter.nordicsemi.com/topic/ps_nrf52840/rng.html

[62] "RP2040 datasheet," Raspberry Pi Ltd. [Online]. Available: https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf

[63] DMO, "RP2040 randomness and ring oscillator," 2022. [Online]. Available: https://dmo.page/2022/10/02/rp2040-randomness-and-ring-oscillator/

[64] S. C. Russell, ""well, that was unexpected...": The Raspberry Pi's hardware random number generator," 2013. [Online].

Available: https://scruss.com/blog/2013/06/07/well-that-was-unexpected-the-raspberry-pis-hardware-random-number-generator/

[65] D. E. Holcomb, W. P. Burleson, and K. Fu, "Power-up SRAM state as an identifying fingerprint and source of true random numbers," *IEEE Transactions on Computers*, vol. 58, no. 9, pp. 1198–1210, 2009. [Online]. Available: https://ieeexplore.ieee.org/document/4674345/

[66] J. Guajardo, S. S. Kumar, G.-J. Schrijen, and P. Tuyls, "FPGA intrinsic PUFs and their use for IP protection," in *Cryptographic Hardware and Embedded Systems — CHES 2007*, ser. Lecture Notes in Computer Science, vol. 4727. Springer, 2007, pp. 63–80.

[67] V. van der Leest, E. van der Sluis, G.-J. Schrijen, P. Tuyls, and H. Handschuh, "Efficient implementation of true random number generator based on SRAM PUFs," in *Cryptography and Security: From Theory to Applications*, ser. Lecture Notes in Computer Science. Springer, 2012, vol. 6805.

[68] staniond, "esp32_puflib: SRAM-based physical unclonable function on the ESP32 microcontroller family." [Online]. Available: https://github.com/staniond/esp32_puflib

[69] ESP32 Forum users, "Wake-up-stub — uninitialized SRAM (SRAM PUF)." [Online]. Available: https://esp32.com/viewtopic.php?t=24384

[70] R. Impagliazzo, L. A. Levin, and M. Luby, "Pseudo-random generation from one-way functions," in *Proceedings of the 21st Annual ACM Symposium on Theory of Computing (STOC '89)*. ACM, 1989, pp. 12–24.

[71] Y. Dodis, R. Gennaro, J. Håstad, H. Krawczyk, and T. Rabin, "Randomness extraction and key derivation using the CBC, cascade and HMAC modes," in *Advances in Cryptology — CRYPTO 2004*, ser. Lecture Notes in Computer Science, vol. 3152. Springer, 2004, pp. 494–510.

[72] H. Krawczyk, "Cryptographic extraction and key derivation: The HKDF scheme," in *Advances in Cryptology — CRYPTO 2010*, ser. Lecture Notes in Computer Science, vol. 6223. Springer, 2010. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-642-14623-7_34

[73] M. P. Pawlowski, A. Jara, and M. Ogorzalek, "Harvesting entropy for random number generation for Internet of Things constrained devices using on-board sensors," *Sensors*, vol. 15, no. 10, pp. 26 838–26 865, 2015.

[74] J. Zhang and M. Wu, "Random number generation based on heterogeneous entropy sources fusion in multi-sensor networks," *Sensors*, vol. 23, no. 20, p. 8497, 2023.

[75] C. Foreman, R. Yeung, and F. J. Curchod, "Statistical testing of random number generators and their improvement using randomness extraction," *Entropy*, vol. 26, no. 12, p. 1053, 2024. [Online]. Available: https://arxiv.org/abs/2403.18716

[76] P. Kietzmann, T. C. Schmidt, and M. Wählisch, "A guideline on pseudorandom number generation (PRNG) in the IoT," *ACM Computing Surveys*, vol. 54, no. 6, pp. 112:1–112:38, 2021.

[77] Raspberry Pi Forums, "Cryptographically secure pseudorandom number generator (RP2040, pico_rand)." [Online]. Available: https://forums.raspberrypi.com/viewtopic.php?t=362076

[78] National Institute of Standards and Technology, "Advanced encryption standard (AES)," NIST, Federal Information Processing Standards Publication FIPS 197-upd1, May 2023. [Online]. Available: https://csrc.nist.gov/pubs/fips/197/final

[79] T. Good and M. Benaissa, "AES on FPGA from the fastest to the smallest," in *Cryptographic Hardware and Embedded Systems — CHES 2005*, ser. Lecture Notes in Computer Science, vol. 3659. Springer, 2005, pp. 427–440.

[80] P. Chodowiec and K. Gaj, "Very compact FPGA implementation of the AES algorithm," in *Cryptographic Hardware and Embedded Systems — CHES 2003*, ser. Lecture Notes in Computer Science, vol. 2779. Springer, 2003, pp. 319–333.

[81] A. Moradi, A. Poschmann, S. Ling, C. Paar, and H. Wang, "Pushing the limits: A very compact and a threshold implementation of AES," in *Advances in Cryptology — EUROCRYPT 2011*, ser. Lecture Notes in Computer Science, vol. 6632. Springer, 2011. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-642-20465-4_6

[82] A. Satoh, S. Morioka, K. Takano, and S. Munetoh, "A compact Rijndael hardware architecture with S-Box optimization," in *Advances in Cryptology — ASIACRYPT 2001*, ser. Lecture Notes in Computer Science, vol. 2248. Springer, 2001, pp. 239–254.

[83] D. Canright, "A very compact S-box for AES," in *Cryptographic Hardware and Embedded Systems — CHES 2005*, ser. Lecture Notes in Computer Science, vol. 3659. Springer, 2005, pp. 441–455. [Online]. Available: https://www.iacr.org/archive/ches2005/032.pdf

[84] J. Boyar and R. Peralta, "A new combinational logic minimization technique with applications to cryptology," in *Experimental Algorithms (SEA 2010)*, ser. Lecture Notes in Computer Science, vol. 6049. Springer, 2010, pp. 178–189. [Online]. Available: https://www.nist.gov/publications/new-combinational-logic-minimization-technique-applications-cryptology

[85] H. Hadipour, "AES-VHDL: VHDL implementation of AES algorithm," https://github.com/hadipourh/AES-VHDL, 2026.

[86] J. Strömbergson, "secworks/aes: Verilog implementation of the symmetric block cipher AES as specified in NIST FIPS 197," https://github.com/secworks/aes, 2026.

[87] National Institute of Standards and Technology, "Secure hash standard (SHS)," NIST, Federal Information Processing Standards Publication FIPS 180-4, Aug. 2015. [Online]. Available: https://csrc.nist.gov/pubs/fips/180-4/upd1/final

[88] J. Strömbergson, "secworks/sha256: Hardware implementation of the SHA-256 cryptographic hash function," https://github.com/secworks/sha256, 2026.

[89] E. Barker and J. Kelsey, "Recommendation for random number generation using deterministic random bit generators," NIST, Special Publication SP 800-90A Rev. 1, Jun. 2015. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/90/a/r1/final

[90] J. Woodage and D. Shumow, "An analysis of NIST SP 800-90A," in *Advances in Cryptology — EUROCRYPT 2019*, ser. Lecture Notes in Computer Science, vol. 11476. Springer, 2019. [Online]. Available: https://eprint.iacr.org/2018/349

[91] E. Barker, J. Kelsey, K. McKay, A. Roginsky, and M. S. Turan, "Recommendation for random bit generator (RBG) constructions," NIST, Special Publication SP 800-90C, Sep. 2025. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/90/c/final

[92] M. Dworkin, "Recommendation for block cipher modes of operation: The CMAC mode for authentication," NIST, Special Publication SP 800-38B (incl. updates as of 10-06-2016), Oct. 2016. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/38/b/upd1/final

[93] J. Song, R. Poovendran, J. Lee, and T. Iwata, "The AES-CMAC algorithm," IETF, RFC 4493, Jun. 2006. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4493

[94] National Institute of Standards and Technology, "CAVP testing: Block ciphers — AES test vectors (KAT_AES.zip, aesmmt.zip, aesmct.zip)," https://csrc.nist.gov/Projects/cryptographic-algorithm-validation-program/block-ciphers, 2026.

[95] ——, “CAVP testing: Block cipher modes — CMAC test vectors (cmactestvectors.zip),” https://csrc.nist.gov/projects/cryptographic-algorithm-validation-program/cavp-testing-block-cipher-modes, 2026.

[96] ——, “CAVP testing: Random number generators — DRBG test vectors (drbgtestvectors.zip),” https://csrc.nist.gov/projects/cryptographic-algorithm-validation-program/random-number-generators, 2026.

[97] ——, “ACVP-Server: NIST automated cryptographic validation protocol server and sample JSON test vectors,” https://github.com/usnistgov/ACVP-Server, 2026.

[98] *SE050 Plug & Trust Secure Element — Product Data Sheet*, NXP Semiconductors, 2023. [Online]. Available: https://www.nxp.com/docs/en/data-sheet/SE050-DATASHEET.pdf

[99] *OPTIGA Trust M Datasheet — SLS 32AIA010MH/S/K/L/M*, Infineon Technologies AG, 2024. [Online]. Available: https://www.infineon.com/assets/row/public/documents/30/49/infineon-optiga-trust-m-sls32aia-datasheet-en.pdf

[100] *STSAFE-A110: Authentication, State-of-the-Art Security for Peripherals and IoT Devices — Datasheet*, STMicroelectronics, 2019. [Online]. Available: https://www.st.com/resource/en/datasheet/stsafe-a110.pdf

[101] DigiKey Electronics, “Precios y disponibilidad de ATECC608B, SE050, OPTIGA Trust M y STSAFE-A110,” https://www.digikey.com/, 2026.

[102] National Institute of Standards and Technology, “Entropy source validation certificate #e46: ECC608 NRBG entropy source,” https://csrc.nist.gov/projects/cryptographic-module-validation-program/entropy-validations/certificate/46, 2023.

[103] I. Kuon and J. Rose, “Measuring the gap between FPGAs and ASICs,” in *Proc. ACM/SIGDA 14th Int. Symp. on Field Programmable Gate Arrays (FPGA ’06)*, 2006, pp. 21–30.

[104] K. Yang, D. Blaauw, and D. Sylvester, “An all-digital edge racing true random number generator robust against PVT variations,” *IEEE Journal of Solid-State Circuits*, vol. 51, no. 4, pp. 1022–1031, 2016.

[105] P. Bayon, L. Bossuet, A. Aubert, V. Fischer, F. Poucheret, B. Robisson, and P. Maurine, “Contactless electromagnetic active attack on ring oscillator based true random number generator,” in *Constructive Side-Channel Analysis and Secure Design (COSADE 2012)*, ser. LNCS, vol. 7275. Springer, 2012, pp. 151–166.

[106] EUROPRACTICE, "Active EUROPRACTICE members," https://www.europractice.stfc.ac.uk/membership/membership_list.cfml, 2026.

[107] EUROPRACTICE IC Service, "MPW schedules and prices 2026," https://europractice-ic.com/schedules-prices-2026/, 2026.

[108] ——, "MPW fabrication and the mini@sic programme," https://europractice-ic.com/services/fabrication/, 2026.

[109] IHP – Leibniz-Institut für innovative Mikroelektronik, "Open source PDK for SG13G2," https://www.ihp-microelectronics.com/services/research-and-prototyping-service/fast-design-enablement/open-source-pdk, 2026.

[110] Instituto de Microelectrónica de Sevilla (IMSE-CNM, CSIC–Universidad de Sevilla), "Áreas de investigación: Hardware security," http://www.imse-cnm.csic.es/es/areas/hardware-sec.php, 2026.

[111] M. C. Martínez-Rodríguez, L. F. Rojas-Muñoz, E. Camacho-Ruiz, S. Sánchez-Solano, and P. Brox, "Efficient RO-PUF for generation of identifiers and keys in resource-constrained embedded systems," *Cryptography*, vol. 6, no. 4, p. 51, 2022. [Online]. Available: https://doi.org/10.3390/cryptography6040051

[112] P. Ortega-Castro, L. F. Rojas-Muñoz, J. M. Mora-Gutiérrez, P. Brox, and M. C. Martínez-Rodríguez, "VLSI integration of a RO-based PUF into a 65 nm technology," in *IEEE Nordic Circuits and Systems Conference (NorCAS)*, 2024. [Online]. Available: https://doi.org/10.1109/NorCAS64408.2024.10752474

[113] P. Ortega-Castro, E. Camacho-Ruiz, J. M. Mora-Gutiérrez, P. Brox, and M. C. Martínez-Rodríguez, "Robust and scalable cell-based 65-nm CMOS RO-PUF implementation," *IEEE Open Journal of the Solid-State Circuits Society*, 2026. [Online]. Available: https://doi.org/10.1109/OJSSCS.2026.3689804

[114] J. M. Mora-Gutiérrez, C. J. Jiménez-Fernández, and M. Valencia-Barrero, "ASIC design and power characterization of standard and low power multi-radix Trivium," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 67, no. 11, pp. 2682–2686, 2020. [Online]. Available: https://doi.org/10.1109/TCSII.2020.2969242

[115] IHP GmbH, "IHP-Open-PDK: Open source PDK for the IHP SG13G2 130 nm BiCMOS process," https://github.com/IHP-GmbH/IHP-Open-PDK, 2026.

[116] SkyWater Technology and Google, "SkyWater SKY130 PDK documentation: Foundry provided standard cell libraries," https://skywater-pdk.readthedocs.io/en/main/contents/libraries/foundry-provided.html, 2026.

[117] GlobalFoundries, Google, and FOSSi Foundation, "GF180MCU open PDK: standard cell libraries for the GlobalFoundries 180 nm MCU process," https://github.com/fossi-foundation/gf180mcu-pdk, 2026.

[118] Silicon Integration Initiative (Si2) and Silvaco (Nangate), "Open cell and free PDK libraries: Nangate 45 nm open cell library," https://si2.org/open-cell-and-free-pdk-libraries/, 2026.

[119] J.-P. Kaps, W. Diehl, M. Tempelmeier, E. Homsirikamol, K. Gaj *et al.*, "Hardware implementations of NIST lightweight cryptographic candidates: A first look," in *NIST Lightweight Cryptography Workshop 2019*, 2019. [Online]. Available: https://csrc.nist.gov/CSRC/media/Events/lightweight-cryptography-workshop-2019/documents/papers/hardware-implementations-of-nist-lwc-candidates-lwc2019.pdf

[120] J. von Neumann, "Various techniques used in connection with random digits," in *Monte Carlo Method, National Bureau of Standards Applied Mathematics Series*. Washington, D.C.: U.S. Government Printing Office, 1951, vol. 12, pp. 36–38.

[121] S. Zhu, Y. Ma, T. Chen, J. Lin, and J. Jing, "Analysis and improvement of entropy estimators in NIST SP 800-90b for non-IID entropy sources," *IACR Transactions on Symmetric Cryptology*, vol. 2017, no. 3, pp. 151–168, 2017.

[122] J. Kelsey, K. A. McKay, and M. Sönmez Turan, "Predictive models for min-entropy estimation," in *Cryptographic Hardware and Embedded Systems – CHES 2015*, ser. LNCS, vol. 9293. Springer, 2015, pp. 372–393.

[123] Digilent Inc., *Basys3 FPGA Board Reference Manual (Revised August 12, 2014; applies to Basys3 rev. C)*, Digilent, 2014. [Online]. Available: https://www.amd.com/content/dam/amd/en/documents/university/aup-boards/XUPBasys3/documentation/Basys3_rm_8_22_2014.pdf

[124] National Institute of Standards and Technology, "Block cipher modes of operation — CMAC mode for authentication: Example values (CMAC-AES128/192/256)," https://csrc.nist.gov/CSRC/media/Projects/Cryptographic-Standards-and-Guidelines/documents/examples/AES_CMAC.pdf, 2016.

[125] A. A. Abidi, "Phase noise and jitter in CMOS ring oscillators," *IEEE Journal of Solid-State Circuits*, vol. 41, no. 8, pp. 1803–1816, 2006.

[126] A. Hajimiri and T. H. Lee, "A general theory of phase noise in electrical oscillators," *IEEE Journal of Solid-State Circuits*, vol. 33, no. 2, pp. 179–194, 1998.

[127] The ngspice team, "ngspice-47: Mixed-level/mixed-signal circuit simulator," https://ngspice.sourceforge.io/, 2026.

[128] Á. Bűrmen and P. Kuthe, "OpenVAF-Reloaded: a Verilog-A compiler producing OSDI models," https://github.com/OpenVAF-Reloaded/OpenVAF, 2026.

Appendix A

Register Map

This appendix is the interface contract between the engine and its host, version 0x03 of the RTL. The same text, with the history of each rule, is kept in the repository as `docs/mapa_registros.md`; the RTL (`rtl/top/motor_top.vhd`), the host model (`firmware/host_check/motor_sim.c`) and the access layer of the firmware (`firmware/componentes/motor_i2c`) follow it.

The engine answers at 7-bit I2C address 0x30. A write sets an 8-bit register pointer that auto-increments on every data byte, read or written. The slave never stretches the clock: long commands are launched by writing `CONTROL` and the host polls `STATUS` until `busy` clears. Undefined addresses read as 0x00 and writes to read-only addresses are ignored. Register, bit and command names are translated here; the repository uses the Spanish ones (`CLAVE`, `RETO`, `sembrar`, ...).

Rules enforced by the hardware. A refused command does not raise `busy` and sets `DIAG.rejected`.

- With a health alarm set, `seed`, `reseed`, `generate` and `random` are refused; `authenticate` is still served, because it uses the key already generated and draws no entropy.
- No seeding before the start-up test has passed (SP 800-90B §4.3 [6]). The seed enters the DRBG only after the APT window that contains its last bit closes without alarm; an alarm or a `clear_alarms` during collection aborts the seeding and leaves the previous state untouched. A seeding takes at most $384 + 1024$ raw bits.
- `seed` and `reseed` are refused with `CFG_KD` below the minimum (generic `G_KD_MIN`, 6 by default), below which the bit-level clock-domain crossing loses bits silently and the health tests cannot tell.
- A capture and a seeding exclude each other, so the capture buffer never holds bits of a seed.

Table A.1: Registers. R: read, W: write.

Address	Name	Access	Content
0x00	ID	R	0xA5
0x01	VERSION	R	0x03
0x02	STATUS	R	Table A.2
0x03	CONTROL	W	commands, Table A.4; self-clearing
0x04	CFG_KD	R/W	divider exponent, $K_D = 2^{\text{value}}$, effective range 0–24 (larger values act as 24). Resets to 24. Written only while the engine is idle; a write during a command is ignored
0x05	CFG_PAG	R/W	page of the capture buffer, 0–3
0x06–0x07	RCT_MAX	R	longest run seen by the Repetition Count Test (RCT), low byte first
0x08–0x09	APT_MAX	R	largest count seen by the APT, low byte first
0x0A–0x0D	REQUESTS	R	reseed counter of the DRBG, 32 bits, low byte first. Saturates at $2^{32} - 1$
0x0E	DIAG	R	Table A.3
0x10–0x1F	KEY	R	current key, 16 bytes. **Test mode only**
0x20–0x2F	CHALLENGE	W	16-byte nonce for challenge-response
0x30–0x3F	TAG	R	CMAC_{KEY}(CHALLENGE), 16 bytes; reads as zeros while `tag_ready` is 0
0x40–0x5F	RANDOM	R	32 bytes from the last `random` command, a DRBG generation independent of the key
0x80–0xFF	CAPTURE	R	128 bytes of raw bits, page CFG_PAG. **Test mode only**

Table A.2: STATUS (0x02).

Bit	Name	Meaning
0	seeded	the DRBG holds a valid state
1	busy	a command is running; do not send another
2	rct_alarm	repetition count test fired (stuck source)
3	apt_alarm	adaptive proportion test fired (biased source)
4	key_valid	a key is loaded in the authenticator
5	capture_full	the capture buffer holds a chunk
6	tag_ready	TAG belongs to the current CHALLENGE
7	test_mode	the test pin is active

Table A.3: DIAG (0x0E).

Bit	Name	Meaning
0	started	the start-up test has passed: one full APT window (1024 samples) without alarm since reset or since the last `clear_alarms`
1	rejected	the last command written to CONTROL was ignored, or, for a seeding, aborted. Cleared by the next accepted command
2	exhausted	the request counter is at $2^{32} - 1$; `generate` and `random` are refused until the engine is reseeded

Table A.4: CONTROL (0x03). One command per write.

Bit	Command	Action
0	seed	collect 384 raw bits (256 of entropy, 128 of nonce), wait until the health tests have judged all of them, and instantiate the DRBG from scratch
1	generate	draw 256 bits; the first 128 become KEY and are loaded into the authenticator, the rest are discarded. Nothing becomes readable
2	authenticate	TAG $\leftarrow \mathrm{CMAC}_{\mathrm{KEY}}$(CHALLENGE)
3	clear_alarms	clear the health alarms; restarts the start-up window
4	capture	fill the capture buffer with raw bits. Test mode only, engine idle, no capture in progress
5	reseed	as `seed`, but on top of the current state; requires a seeded DRBG
6	random	draw 256 bits into RANDOM
7	clear_all	zeroization: KEY, the whole DRBG state and its intermediates (including the AES core), CHALLENGE, RANDOM, TAG and the collected seed to zero, and the zero key loaded into the authenticator. Accepted whenever the engine is idle, also with an alarm

- The request counter does not wrap: at $2^{32} - 1$ the engine refuses to generate until reseeded (`DIAG.exhausted`). The reseeding policy below that ceiling is the host's.

Priority. If several command bits arrive in one write, only the highest-priority one is served: `clear_all`, then `seed`/`reseed` (`reseed` if both), `generate`, `random`, `authenticate`, `capture`. `clear_alarms` is served alongside any of them, but it must not share a write with a seeding: it restarts the start-up window, so the seeding is accepted and aborted on the next cycle.

Typical host sequence.

1. Read ID (0xA5) and VERSION (0x03).
2. Write `CFG_KD` with the divider of the characterization, or leave the reset value.
3. Poll DIAG until `started`; write `seed`; poll STATUS until `busy` clears; check `DIAG.rejected` $= 0$, `seeded` $= 1$ and no alarm.
4. Write `generate`, poll, check `DIAG.rejected`. A key now exists.
5. Bench provisioning only, in test mode: read KEY.
6. Challenge-response: write CHALLENGE, write `authenticate`, poll, read TAG and compare it with the CMAC computed by the host.
7. Random for the host: write `random`, poll, read RANDOM.

Test mode. The `test_mode` pin, synchronized by two flip-flops, enables reading KEY and CAPTURE and the `capture` command. On the FPGA it is a board switch; on an integrated circuit it would be a fuse blown at production.

Appendix B

Official Test Vectors Used

Table B.1 lists where every reference value used by the benches comes from, and how each check closes. Official vectors are used where a standard publishes them. Where the hardware's input layout differs from any published case, the expected values come from a reference model in Python that is itself checked against the official vectors first, and written independently of the VHDL.

The CAVP files are kept in the repository (`tests/vectors/drbg/`), so the validation does not depend on the NIST site staying as it is. The entropy estimates of Chapter 8 are not test vectors: they come from the NIST SP 800-90B tool [37], run unchanged.

Table B.1: Reference values and their origin. Counts are checks passed in the last run (`results/tb_motor_top.log`, 177 in total, with 11 of 11 PSL assertions and 6 of 6 cover points).

Block	Source of the expected values	Where checked
AES-128 (both cores)	FIPS 197 [78], Appendix B and Appendix C.1; SP 800-38A, F.1.1 ECB-AES128, three blocks; a thousand chained encryptions repeated; Appendix C.1 again after `clear`	`tb_aes_enc`, 18 checks, the two cores with the same stimuli and required to agree
AES-CMAC	RFC 4493 [93] (the examples of SP 800-38B [92]): the four messages of 0, 16, 40 and 64 bytes under key `2b7e1516...`, and the 16-byte case repeated; the subkeys K_1 and K_2 of the same example	`tb_aes_cmac`, 7 checks
CTR_DRBG, reference model	CAVP `drbgtestvectors` [96], `CTR_DRBG_no_reseed.rsp` and `CTR_DRBG_pr_false.rsp`, AES-128 with and without derivation function: $4 \times 240 = 960$ cases	`analysis/ctr_drbg_ref.py`, 960 of 960
CTR_DRBG, RTL	the reference model above, with the hardware's input layout (256-bit entropy, 128-bit nonce, no personalization): `analysis/gen_drbg_vectores.py` writes `drbg_vectores_pkg.vhd`	`tb_ctr_drbg`, 18 checks, including zeroization and counter saturation
Whole engine	the key comes from ring noise, so the bench dumps key, challenge and tag, and the CMAC is recomputed with the Python model	`analysis/verifica_top.py`, 5 of 5
Firmware against the engine model	CMAC and ECDSA of the target's mbedtls 3.6, compiled for the PC; signed readings checked with the `cryptography` library	`run_host.py`, 0 failures; `verifica_lecturas.py`
Health-test cut-offs	SP 800-90B §4.4 [6] formulas for $H = 0.98$ and $\alpha = 2^{-20}$: RCT $C = 22$, APT $W = 1024$, $C = 596$	`analysis/cortes_salud.py`; behaviour in `tb_health_tests`, 15 checks

Appendix C

Reproduction Guide

Everything in this work that does not need the boards is rerun on every push by a continuous-integration workflow (`.github/workflows/verifica.yml`) on a clean Ubuntu machine that is not the author's: seven jobs, no credentials. The repository is public: `https://github.com/mmggjj/TFM_ENCRY`. The commands below are the ones the workflow runs, so a green run reproduces them. Outputs are never filtered: the scripts print the full tool output and fail on any error.

Table C.1: What the workflow reproduces.

Job	Command	Result
Model and estimators	`python analysis/ero_model.py`, `jitter_estimator.py`, `ctr_drbg_ref.py`, `cortes_salud.py`, `fuzz_verifica_lecturas.py`, `verifica_lecturas.py` on the board log	self-checks, 960 CAVP cases, board readings valid
VHDL benches	`python rtl/run_tb.py`, then `python analysis/verifica_top.py`	7 benches, PSL report as artefact
Synthesis	`python synth/sintetiza.py -verilog`	area per block in kGE (Chapter 7)
Firmware on the PC	`python firmware/host_check/run_host.py -mbedtls <dir>`, and again with `-sanear`	0 failures, with ASan and UBSan
ESP-IDF build	`idf.py set-target`, `build`, `size`, and `idf.py -B build_sim -DMOTOR_SIMULADO=1 build`, for the verifier (ESP32) and the meter (ESP32-C3)	4 images
Thesis	`tectonic tfm_main.tex` and `tfm_main_es.tex`	both PDFs as artefacts

Tools. Python 3.12 with `numpy`, `scipy` and `cryptography`; GHDL from the Ubuntu packages (5.0.1 with the mcode back-end on the author's machine) for the benches, which must be run with VHDL-2008; Yosys through `yowasp-yosys`, or the OSS CAD Suite with the GHDL plug-in, and the IHP SG13G2 standard-cell library of the open PDK [115]; mbedtls 3.6.2 from its release tarball (a git clone lacks the generated sources); ESP-IDF 5.3.5, the workflow using the `espressif/idf:v5.3.5` container; tectonic 0.15. The GHDL 7.0 development build with the LLVM back-end crashes on the PSL properties of `tb_motor_top` that use external names, and should not be used.

Outside the workflow. Three parts need software or hardware the workflow does not have:

- *Transistor-level ring.* ngspice 47 [127] with OpenVAF-Reloaded [128] to compile the PSP models of the PDK; `python analysis/anillo_ihp_repeticiones.py`, with `-esquina`, `-temp`, `-vdd` and `-mismatch` for corners, supply and mismatch. ngspice cannot fix the noise realization, so every run is a new one and the uncertainty comes from repeating. One 400 ns realization takes about 30 minutes.
- *SP 800-90B estimates of the ESP32.* The NIST tool [37] built from source; `python analysis/entropia_90b.py`, then `resumen_90b.py`, on the captures kept in `results/`; the restart test with `reinicios_90b.py`. New captures need an ESP32 and `analysis/campana_rng.py`.
- *FPGA.* Vivado (free tier, Artix-7 included) and `synth/basys3/build.tcl`; installation and programming are described in `docs/instalacion_vivado_windows.md`.

First ESP32 campaign. The analysis of the first campaign (Sections 3.3 to 3.5) reruns without the board from the files kept in `results/esp32_rng_agosto`, with `numpy`, `scipy`, `matplotlib` and `cryptography`:

- `python analysis/esp32_rng/nist_sp800_22.py` runs the self-test of the SP 800-22 battery (operating-system data, zeros, alternating pattern).
- `python analysis/esp32_rng/harvester_from_sources.py` writes the 1000 seeds of the off-line validation to `seeds_rep1.bin`, from `puf_boot1.bin` and `condD_rep1.bin`.
- `python analysis/esp32_rng/run_campaign.py` runs `nist_analysis.py`, `entropy_analysis.py`, `temperature_correlation.py`, `harvester_validation.py` and the off-line mode of `mqtt_tls_demo.py`. Each writes its CSV and figures into the data directory, and `run_campaign.py` copies the figures into `memoria/figures` under their own names; the thesis uses only

`nist_heatmap.pdf`, as `esp32_nist_heatmap.pdf`. The four scripts also run one by one, with `-d <data directory>` and `-figdir <directory>`. Rerun on 2026-10-31 on a copy of the data, they reproduce the CSV files kept, value for value.

- `python analysis/esp32_rng/make_result_tables.py` writes the result tables as LaTeX into `tablas_generadas.tex`, and `python analysis/esp32_rng/figuras_memoria.py` draws Figures 3.3, 3.4 and 3.6 into `memoria/figures`.

A new capture needs an ESP32 with `firmware/esp32_rng` built and flashed (`idf.py set-target esp32`, `build`, `-p <port> flash`; ESP-IDF 5.3, console at 115 200 baud). Then `python analysis/esp32_rng/capture_rng.py -port <port> -reps <n>` resets the board through RTS and collects n valid sequences per condition, with the SRAM patterns (`puf_boot<k>.bin`), the seeds and `capture_log.csv`; by default it writes into `results/esp32_rng_agosto`, so `-o <directory>` keeps new data apart. A reset through RTS keeps the SRAM powered and does not give a new power-up pattern: for the clean series, unplug the board, plug it in again and run `python analysis/esp32_rng/capture_puf.py -port <port>` once per power-up, which adds one `puf_pc<N>.bin`. The later campaigns use the same firmware with `analysis/campana_rng.py`, which keeps the whole serial stream and numbers the repetitions from `-primera-rep`.